

GRENOBLE-INP ENSIMAG

INGÉNIERIE DES SYSTÈMES
D'INFORMATION

2A ISI

GRENOBLE-INP ENSIMAG

INFORMATION SYSTEMS
ENGINEERING

2A ISI



**ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET DE MATHÉMATIQUES
APPLIQUÉES DE GRENOBLE**

PROJET GoMile

Rapport

Réalisé par :

DUONG DANG KHOA DANG
DIAKITE ALPHA OUSMANE
JOUNI YOUSSEF
MAOUDE SAMIR
OUDRHIRI IDRISSE SAFWANE

CLIENT :

GoMile Entreprise

2026–2027

Table des matières

1	Introduction	7
2	Manuel d'utilisateur	9
1	App mobile	9
1.1	But de l'application	9
1.2	Installation et première ouverture	9
1.3	Inscription (par étapes)	11
1.4	Écran principal — Tableau de bord	11
1.5	Liste des missions	13
1.6	Focus mission et navigation	15
1.7	Détail d'une mission	17
1.8	Portefeuille (Wallet)	19
1.9	Profil et vérification KYC	19
2	Plugin WooCommerce	20
2.1	Parcours de test	21
2.2	Résultat attendu	23
3	Plateforme web	23
3.1	Lancement local	24
3.2	Déploiement en production	24
3.3	Accueil et choix de l'espace	25
3.4	Connexion commerçant	26
3.5	Inscription commerçant	26
3.6	Vérification de l'adresse e-mail	27
3.7	Tableau de bord marchand	28
3.8	Gestion des magasins	28
3.9	Création d'une boutique	29
3.10	Suivi des commandes	30
3.11	Gestion des clés API	30
3.12	Inscription livreur web	31

4	Vérifications du front web	33
4.1	Tests Cypress	33
4.2	Typecheck et qualité du code	34
4.3	Build de validation	34
4.4	Technologies utilisées	35
5	Plateforme web-Admin	36
5.1	Connexion à l'espace administrateur	36
5.2	Navigation dans le tableau de bord	36
5.3	Consultation et gestion des livreurs	37
5.4	Consultation des commerçants	39
5.5	Traitement des demandes de retrait	40
5.6	Renvoi du handshake d'une commande	41
5.7	Création d'un nouveau compte	42
3	Back-end	44
1	Stack technique et choix de conception	44
1.1	Routage et géocodage : OpenRouteService	44
1.2	Stockage des fichiers : Amazon S3	44
1.3	SMS : Twilio	45
1.4	Hébergement : Raspberry Pi 4 avec Cloudflare Tunnel	45
1.5	Autres choix techniques	47
2	Architecture abstraite	47
2.1	Vue d'ensemble des couches applicatives	47
3	Architecture de déploiement	48
3.1	Vue d'ensemble de l'infrastructure	48
3.2	Configuration Nginx	48
3.3	Gestion des processus (systemd)	49
3.4	Déploiement continu (webhook GitLab)	49
4	Pipeline CI/CD	50
5	Schéma de base de données	50
5.1	Groupe 1 : Authentification et utilisateurs	51
5.2	Groupe 2 : Commandes et livraison	51
5.3	Groupe 3 : Magasins et intégrations API	52
5.4	Groupe 4 : Finance et KYC	52
5.5	Référence des énumérations	53
6	Modèle du domaine	53
7	Couche BFF (Backend-for-Frontend)	55
7.1	Rôle et motivations	55

7.2	Architecture des cookies d'authentification	55
7.3	Flux de requête authentifiée (proxyApiRequest)	55
7.4	Déduplication du rafraîchissement (token-refresh.ts)	56
7.5	Factories de routes (route-factories.ts)	57
7.6	Flux de connexion et déconnexion via BFF	57
7.7	Contrats partagés (shared/)	57
8	Authentification et autorisation	58
8.1	Inscription commerçant	58
8.2	Inscription livreur — spécificités	58
8.3	Connexion et rotation des tokens	59
8.4	Réinitialisation du mot de passe (3 étapes)	60
8.5	Guards et décorateurs	60
9	Cycle de vie d'un token JWT	61
10	Cycle de vie d'une commande	63
10.1	Flux métier complet	63
10.2	Annulation de commande	63
11	Machines à états	65
11.1	Statut de commande	65
11.2	Disponibilité du livreur	66
11.3	Statut d'abonnement	67
12	Système de handshake	67
12.1	Principe	67
12.2	Flux de vérification et limitation des tentatives	68
12.3	Verrouillage définitif	68
12.4	Expiration et récupération du code	69
13	Module livreur	69
13.1	Mise à jour de position	69
13.2	Basculement de disponibilité	69
13.3	Missions disponibles	69
13.4	Refus de mission	70
13.5	Portefeuille et retraits	70
14	KYC et vérification des documents	71
15	Module administrateur	71
16	Abonnements et facturation (Stripe)	72
17	Tarification et calcul d'itinéraire	73
17.1	Formule de calcul	73
18	Upload de fichiers (S3)	73

19	Événements temps réel (WebSocket)	73
20	Notifications	74
20.1	Push (Expo)	74
20.2	SMS (Twilio)	74
21	Système d'emails	74
22	Webhooks sortants	75
23	Tâches planifiées (cron jobs)	75
24	Gestion des erreurs	76
25	Variables d'environnement	77
26	Couverture de tests	78
26.1	Stratégie générale	78
26.2	Périmètre des tests end-to-end	78
26.3	Périmètre des tests unitaires	79
26.4	Résultats de couverture	80
26.5	Analyse et justification	80
4	Application Web	82
1	Implémentation de la partie frontend administrateur	82
1.1	Structure générale du code	82
1.2	Routage et organisation des pages	83
1.3	Composants réutilisables	84
1.4	Composants métier spécialisés	85
1.5	Couche de services frontend : <code>api-admin.ts</code>	85
1.6	Typage TypeScript	86
1.7	Fonctions utilitaires	87
1.8	Authentification et gestion de session	88
1.9	Gestion des états d'interface	88
1.10	Diagramme de cas d'utilisation	88
1.11	Diagrammes de séquence système	89
2	Partie commercial	94
2.1	Objectif et périmètre	95
2.2	Diagramme de cas d'utilisation	95
2.3	Conception de l'espace commerçant	96
2.4	Parcours d'inscription marchand et vérification e-mail	97
2.5	Connexion marchand	98
2.6	Tableau de bord commerçant	99
2.7	Magasins	100
2.8	Flux de création d'une boutique	100

2.9	Commandes	101
2.10	Flux de suivi et d'annulation d'une commande	102
2.11	Clés API	103
2.12	Flux de création d'une clé API de boutique	104
2.13	Flux de handshake	105
2.14	Conception de la partie livreur web	106
5	Application Mobile Livreur	108
0.1	Rôle de la partie mobile	108
0.2	Architecture générale et technologies	108
0.3	Parcours d'inscription	109
0.4	Connexion et vérification email	111
0.5	Réinitialisation du mot de passe	111
0.6	Diagramme d'état global authentification	114
0.7	Décisions UX abandonnées ou déplacées	114
0.8	Navigation et écrans post-connexion	115
0.9	Écran enfant : Détails Mission (MissionDetailsScreen)	117
0.10	Diagramme de flux : Navigation Tableau de Bord → Détails Mission → Focus	119
0.11	Diagramme d'état : Cycle de vie d'une mission acceptée	120
0.12	Écran 2 : Missions (MissionsScreen)	121
0.13	Écran enfant : Focus Mission (MissionFocusScreen)	122
0.14	Diagramme de flux : Acceptation mission → Focus → Vérification codes	125
0.15	Diagramme d'état : Transition états mission dans MissionFocusScreen	126
0.16	Écran 3 : Portefeuille (WalletScreen)	127
0.17	Diagramme de flux : Wallet (solde → retrait → confirmation) . . .	128
0.18	Écran 4 : Profil (ProfileScreen)	128
0.19	Diagramme d'état : Profil et transitions KYC	130
0.20	Écran enfant : KYC (KYCScreen)	131
0.21	Services et stores partagés	136
6	Plugin WooCommerce	140
1	Rôle dans l'architecture GoMile	140
2	Fonctionnement général	140
3	Choix techniques	142
4	Sécurité et synchronisation	143
5	Documentation technique	144

7	Utilisation de l'intelligence artificielle pendant le projet	146
8	Conclusion	148
1	Annexe : vocabulaire technique	149



Chapitre 1

Introduction

Contexte du projet

GoMile est une plateforme de livraison développée dans le cadre de notre projet spécialité de 2A à l'École Nationale Supérieure d'Informatique et de Mathématiques Appliquées de Grenoble (ENSIMAG), filière Ingénierie des Systèmes d'Information. Ce projet a été réalisé pour le client *GoMile Entreprise* par une équipe de cinq étudiants : Duong Dang Khoa Dang, Alpha Ousmane Diakite, Youssef Jouini, Samir Maoude et Safwane Oudrhiri Idrissi.

L'objectif de GoMile est de mettre en relation des commerçants ayant besoin de livrer des colis avec des livreurs indépendants disponibles à proximité. La plateforme coordonne l'ensemble du cycle de livraison : création de mission côté commerçant, attribution et exécution par le livreur, validation par code, et reversement des gains sur un portefeuille numérique.

Architecture du système

Le système GoMile repose sur plusieurs composants complémentaires :

- **Application mobile livreur** (React Native / Expo) : permet au livreur de s'inscrire, de passer une vérification d'identité (KYC), de consulter et d'accepter des missions, de naviguer jusqu'au point de livraison et de gérer ses gains depuis un portefeuille intégré.
- **Interface web commerciale** : espace destiné aux commerçants pour créer et suivre leurs missions de livraison.
- **Interface web d'administration** : tableau de bord permettant à l'équipe GoMile de superviser l'ensemble de la plateforme, de valider les comptes livreurs et de gérer

les litiges.

- **Backend et API REST** : cœur métier du système, exposant l'ensemble des fonctionnalités aux différents clients via une API unifiée.

Structure du rapport

Ce document constitue le rapport technique complet du projet. Il est organisé en quatre grandes parties.

Le **manuel utilisateur** présente comment faire les installations nécessaires à l'exécution du projet, ainsi que le guide-user des interfaces web et appli mobile.

La **documentation de conception** détaille les choix d'architecture et d'implémentation pour chaque composant : le backend et son API, les interfaces web administrateur et commerciale, le plugin ainsi que l'application mobile et ses briques techniques.

La section **tests et validation** rend compte de la stratégie de test adoptée ainsi que les résultats obtenus.



Chapitre 2

Manuel d'utilisateur

1 App mobile

1.1 But de l'application

Cette application mobile s'adresse aux livreurs : elle permet de s'inscrire, de se connecter, de recevoir et d'accepter des missions, de suivre un trajet, de gérer son portefeuille et son profil.

1.2 Installation et première ouverture

Procédure rapide depuis la racine du projet (développeur/testeur) :

1. Installer `pnpm` si nécessaire :
 - Option simple : `npm install -g pnpm`
 - Ou activer Corepack et préparer pnpm : `corepack enable && corepack prepare pnpm@latest -activate`
2. Installer toutes les dépendances (depuis la racine du dépôt) : `pnpm install`.
3. Lancer l'application mobile en mode LAN (réseau local) si vous voulez utiliser un simulateur sur PC ou avec l'application Expo GO :
 - `corepack pnpm run mobile:lan` ou `corepack pnpm run mobile:tunnel` pour le mode tunnel
 - Puis entrez le caractère 's' pour activer la simulation, sinon Expo vous donne un lien illisible
4. Installer Expo Go sur le téléphone OU configurer un émulateur sur le PC. Expo Go est recommandé pour les tests réels (plus simple et fiable).

Après ces étapes, ouvrir l'application via Expo (QR code ou lien) ; l'utilisateur arrive sur l'écran de connexion.

20:13

BIENVENUE

Email

votre@email.com

Mot de passe

.....

Mot de passe oublié?

SE CONNECTER

OU

DEVENIR LIVREUR

FIGURE 2.1 – Écran de connexion — saisie email / mot de passe ou démarrer l'inscription

1.3 Inscription (par étapes)

L'inscription se déroule en plusieurs étapes : informations de base, identité (photo, nom, prénom, date de naissance, téléphone) et logistique (adresse, véhicule). Les captures ci-dessous montrent les étapes.

The figure displays three sequential screenshots of a mobile application's registration process, each with a green header bar and a status bar at the top showing the time and signal strength.

- Step 1/3: IDENTITÉ - Informations de base (1/3)**
This screen features a profile picture placeholder with a large Greek letter alpha (α). Below it are input fields for 'Prénom' (containing 'Test Prenom'), 'Nom' (containing 'Test Nom'), 'Email' (containing 'Ensimag@gmail.com'), 'Mot de passe', and 'Confirmation de Mot de passe'. A small note at the bottom states: 'Le numéro de téléphone, la date de naissance et le genre seront demandés à l'étape suivante.' At the bottom are 'RETOUR' and 'CONTINUER' buttons.
- Step 2/3: IDENTITÉ - Complète ton identité (2/3)**
This screen also has the alpha profile picture placeholder. It includes input fields for 'Numéro de téléphone' (0785231458), 'Date de naissance' (2000-03-01), and 'Genre' (with 'Homme', 'Femme' selected, and 'Autre' as an option). At the bottom are 'RETOUR' and 'CONTINUER' buttons.
- Step 3/3: LOGISTIQUE - Logistique (3/3)**
This screen contains input fields for 'Adresse de résidence' (1 Avenue Thiers 33100 Bordeaux), 'Ville de livraison' (Bordeaux), and 'Rayon (km)' (25). Below these is a 'Mode de transport' section with buttons for 'Velo', 'Moto' (highlighted with a blue border), 'Voiture', and 'Utilitaire'. At the bottom are 'RETOUR' and 'CONTINUER' buttons.

FIGURE 2.2 – Étapes d'inscription 1/3, 2/3, 3/3

1.4 Écran principal — Tableau de bord

Une fois connecté et validé, l'application ouvre le tableau de bord. Il affiche la carte, le solde rapide du jour et les missions proposées à proximité.

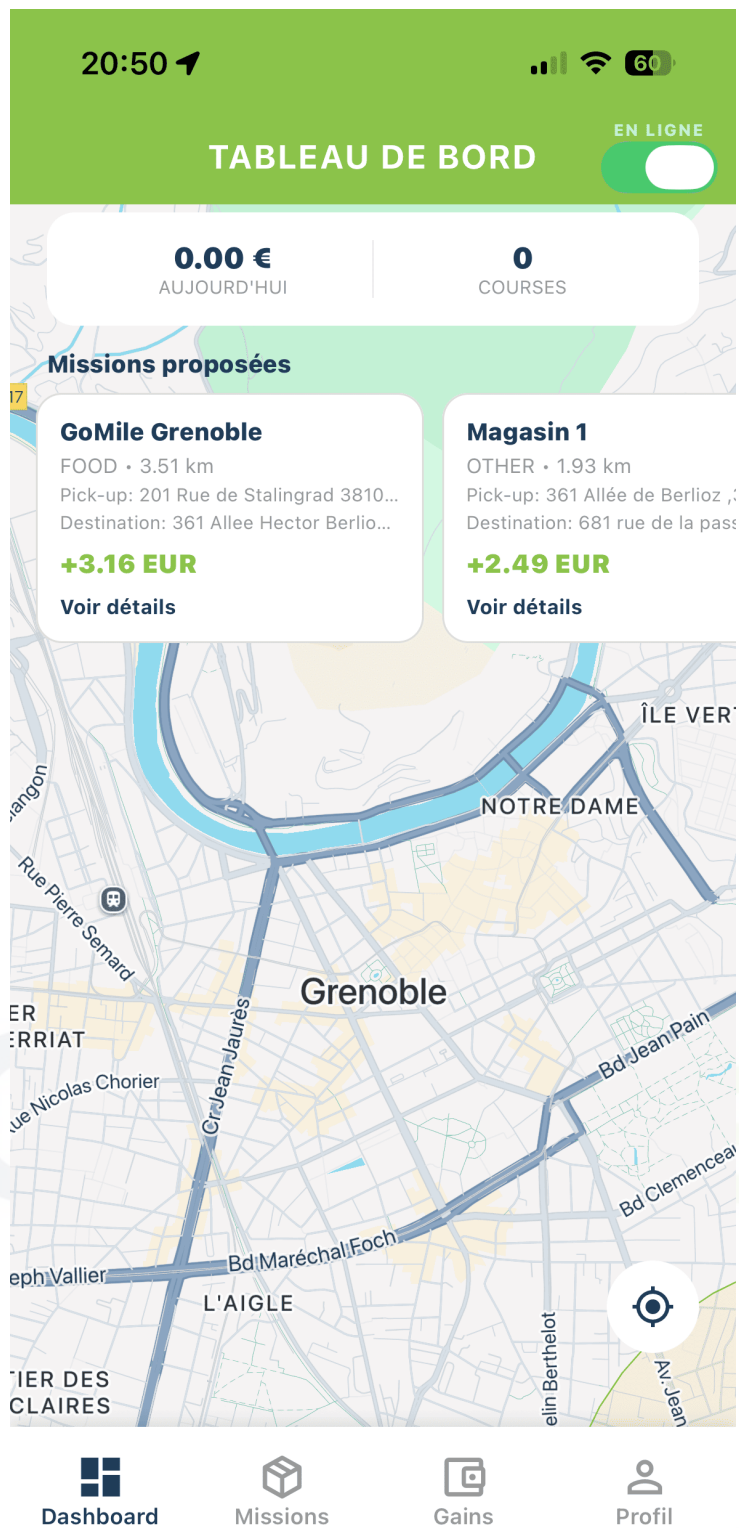


FIGURE 2.3 – Tableau de bord — carte et cartes de missions proposées

1.5 Liste des missions

L'onglet "Missions" regroupe : notifications, missions en cours et missions à proximité. Chaque carte propose un aperçu rapide et un bouton *Voir les détails*.





FIGURE 2.4 – Écran Missions — liste et état

1.6 Focus mission et navigation

Lorsqu'une mission est sélectionnée, l'écran Mission Focus montre le tracé sur la carte, les points pickup / delivery et les actions principales (valider identification commerçant, valider code client, terminer mission).



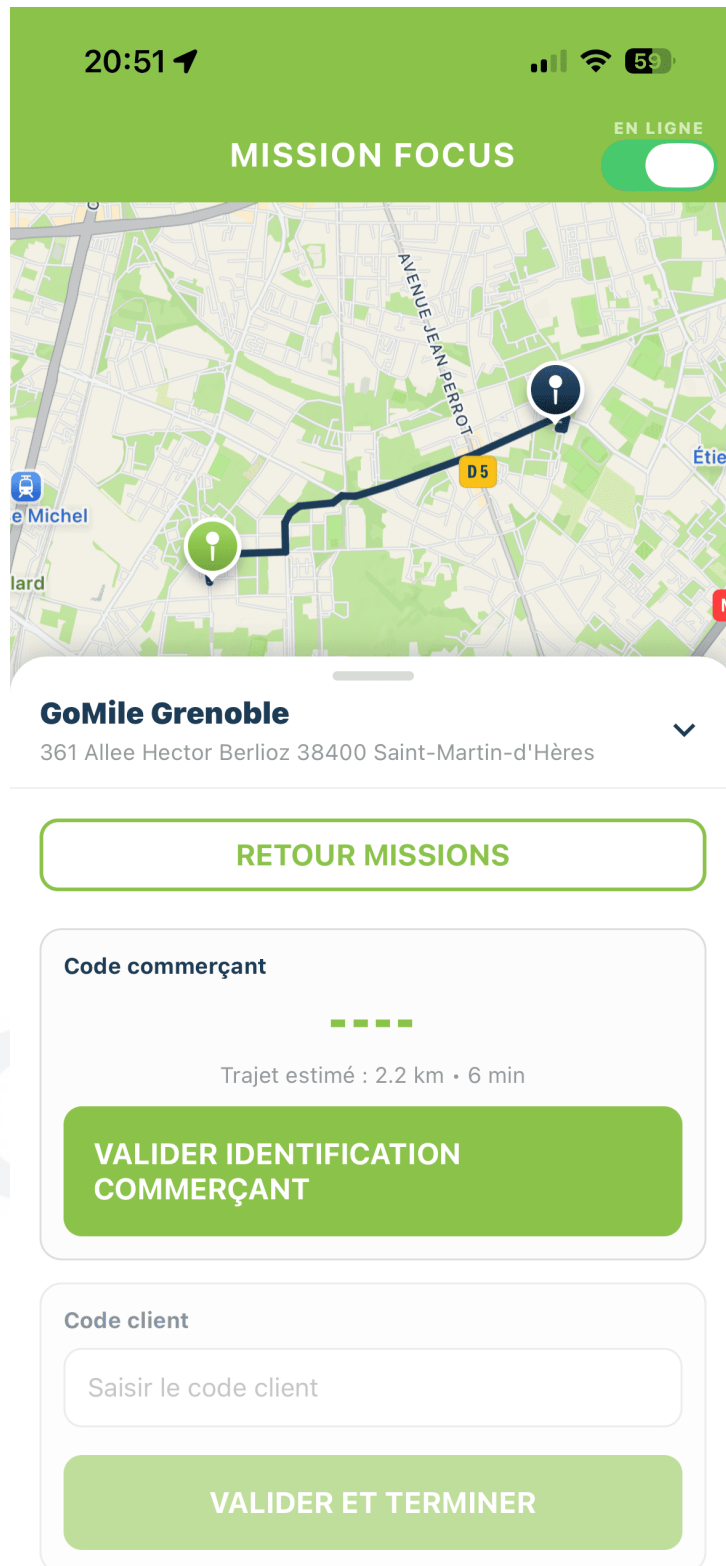


FIGURE 2.5 – Mission Focus — itinéraire et actions métier

1.7 Détail d'une mission

La fiche détail expose les informations métier : adresses, type (FOOD, OTHER...), prix proposé, distance et temps estimé. Les boutons d'acceptation/refus sont clairement identifiés.



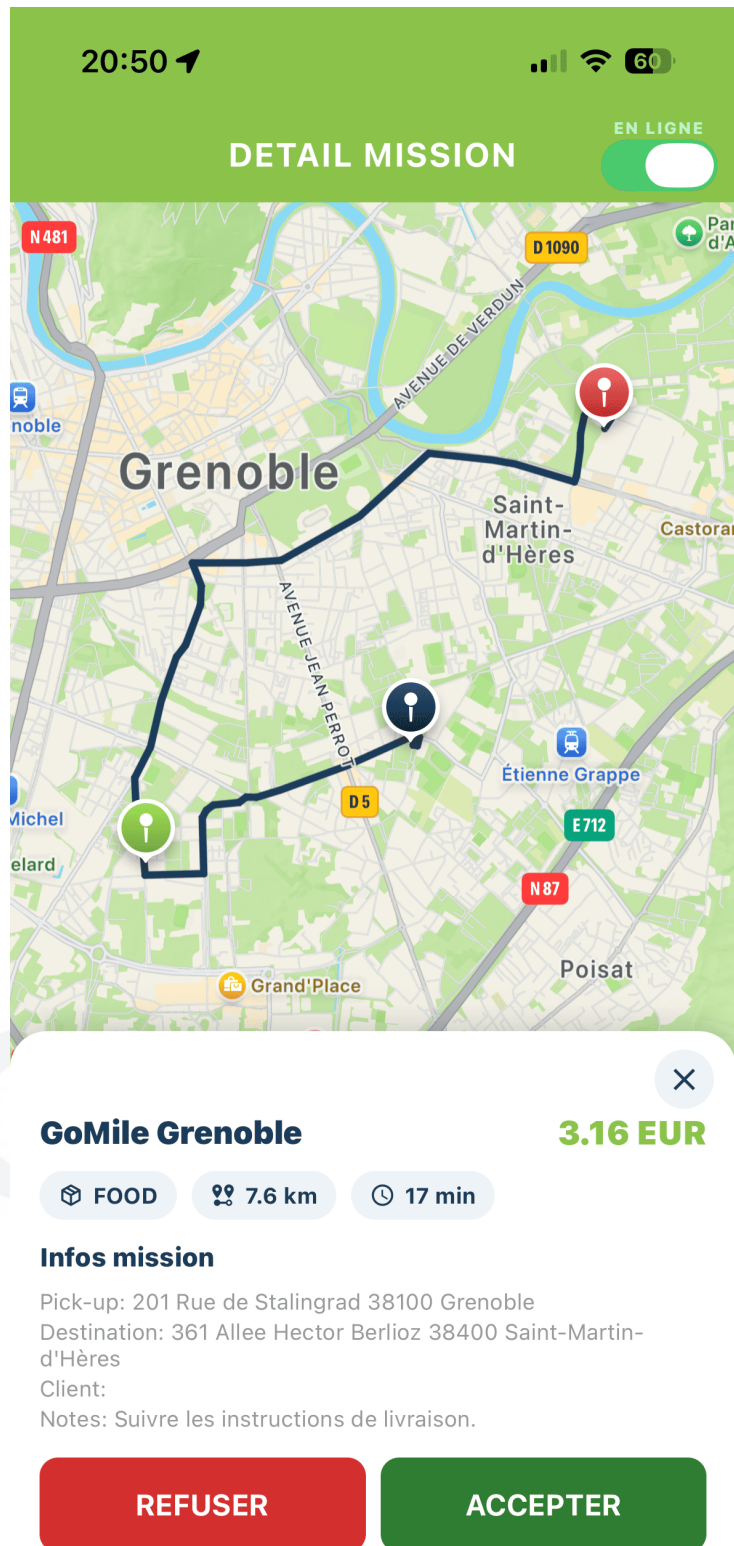


FIGURE 2.6 – Détail mission — infos et actions

1.8 Portefeuille (Wallet)

L'onglet "Gains" montre le solde disponible et les montants en attente. L'utilisateur peut demander un virement depuis le modal de retrait.

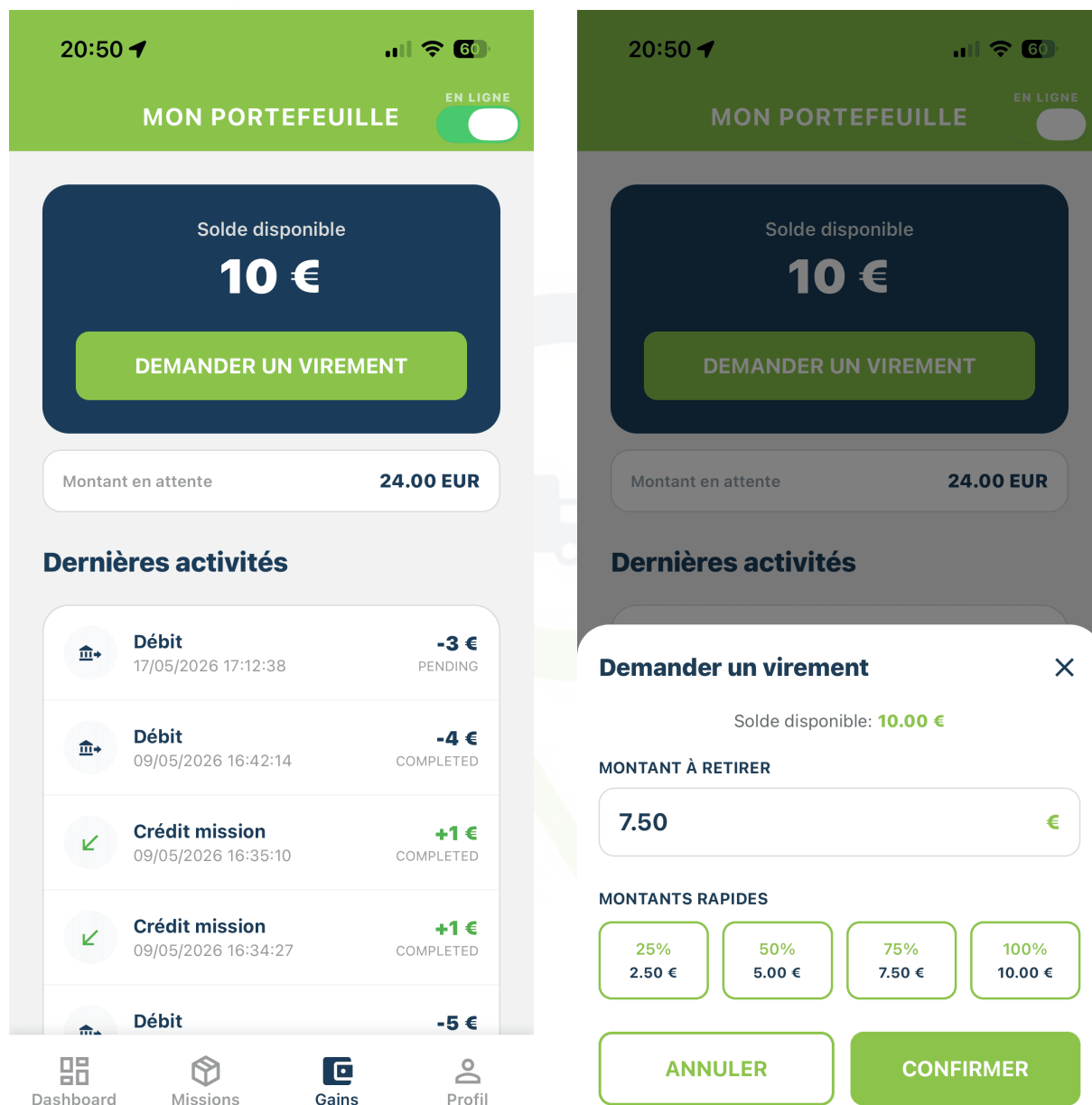


FIGURE 2.7 – Portefeuille — solde et modal de demande de virement

1.9 Profil et vérification KYC

L'écran Profil permet de consulter et modifier les informations du livreur, choisir le véhicule de la session et suivre l'état KYC (non soumis / en attente / approuvé / rejeté). La vignette KYC s'affiche en tête de section.

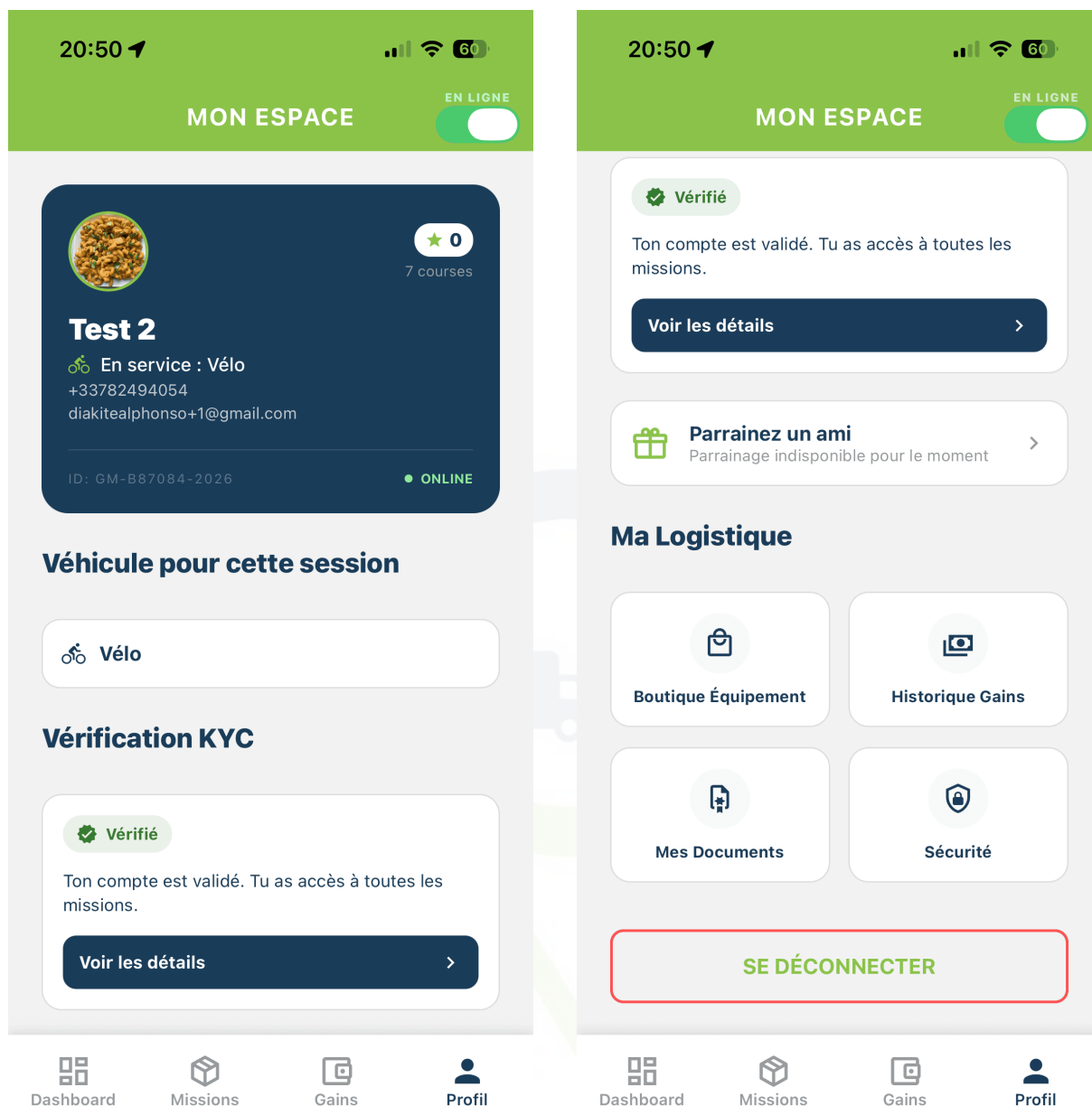


FIGURE 2.8 – Profil — résumé et section KYC / logistique

2 Plugin WooCommerce

Le plugin peut être testé directement sur la boutique de démonstration :

<https://store.pgn2vid.com/>

Le test consiste à effectuer un parcours d'achat complet et à vérifier que la livraison GoMile apparaît dans le panier puis dans la page de validation de commande.

2.1 Parcours de test

Pour tester le plugin, suivre les étapes suivantes :

1. Ouvrir la boutique de test : <https://store.pgn2vid.com/>.
2. Choisir un ou plusieurs produits dans la boutique.

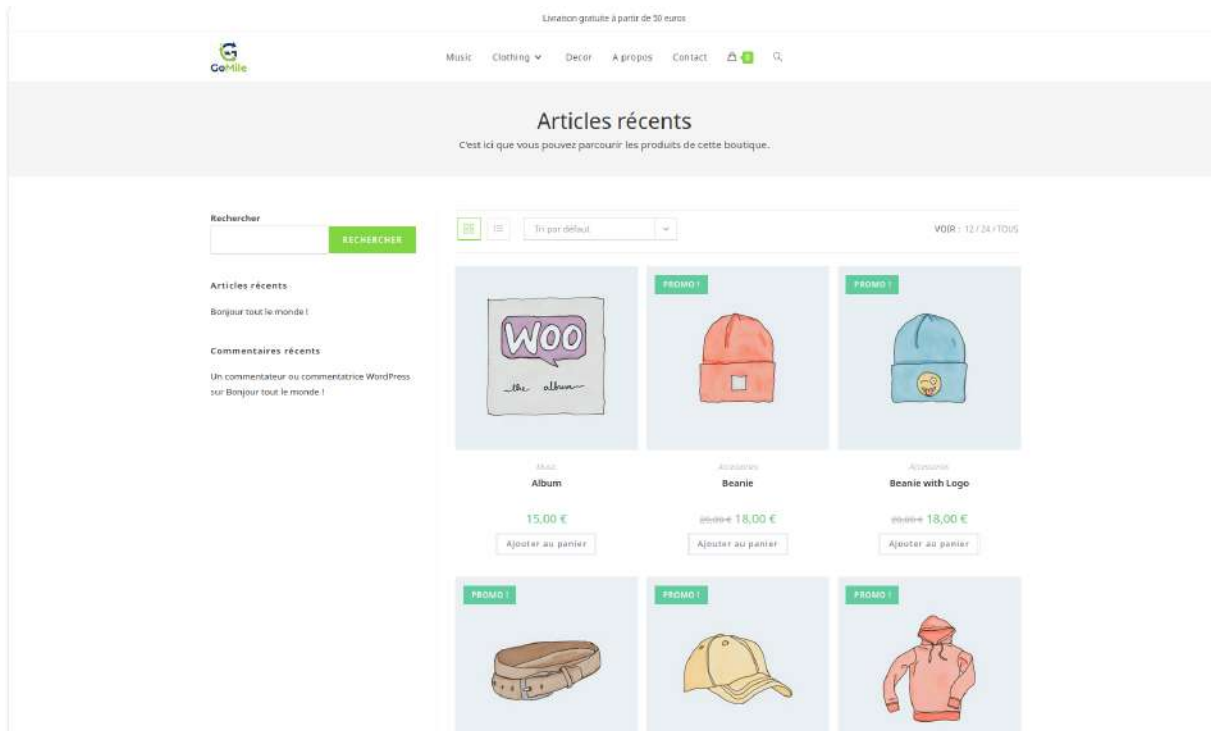


FIGURE 2.9 – Boutique de démonstration utilisée pour tester le plugin WooCommerce.

3. Ajouter les produits au panier.
4. Ouvrir la page du panier.
5. Vérifier que la ligne **Livraison Gomile** apparaît dans le total du panier.

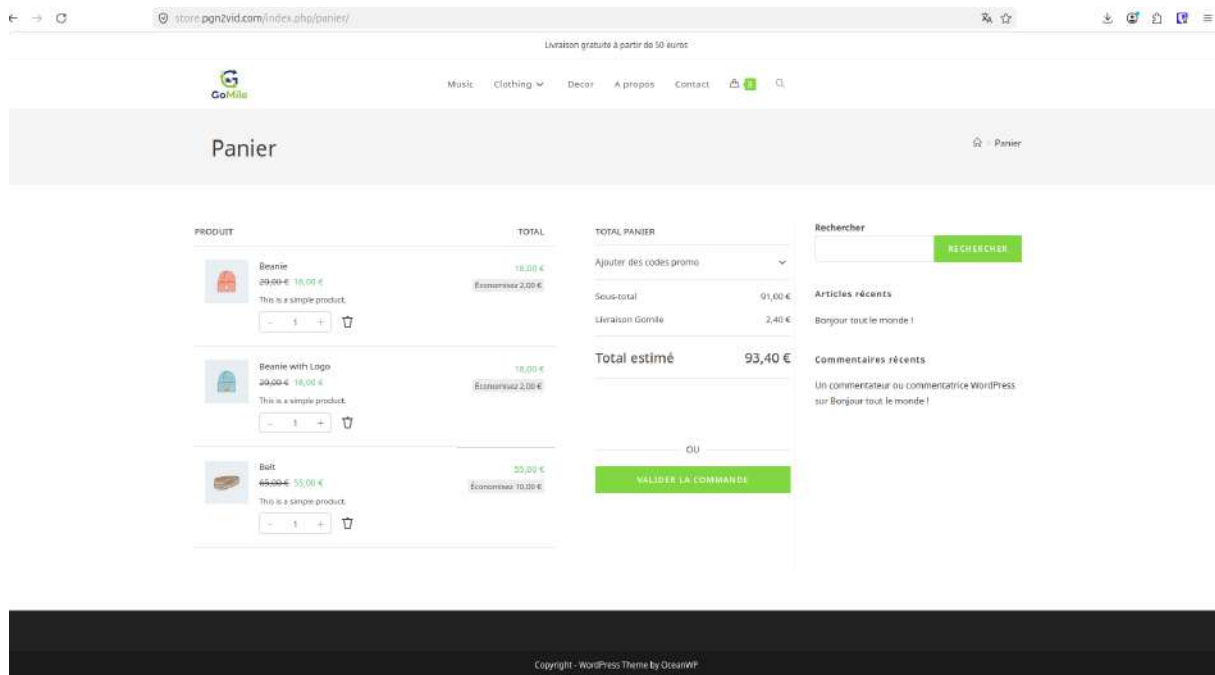


FIGURE 2.10 – Affichage de la livraison GoMile dans le panier de la boutique de test.

6. Cliquer sur le bouton de validation de commande.
7. Renseigner les informations de livraison demandées.
8. Vérifier que l'option **Livraison Gomile** est proposée dans les options de livraison.
9. Finaliser la commande avec le moyen de paiement de test disponible.

The screenshot shows a web browser window with the URL `store.pgn2vid.com/index.php/commander/`. The page title is "Validation de la commande". The main content area is divided into several sections:

- Validation de commande express:** A black button with the Google Pay logo.
- Coordonnées:** A form field for "Adresse e-mail" containing `john.doe@gmail.com`.
- Adresse de livraison:** A form field containing "John Doe", "361 allée de Berlinz, 38400 SAINT-MARTIN-D'HERES, France", and "+33620651777". A "Modifier" button is next to it.
- Options de livraison:** A radio button selected for "Livraison GoMile" with a price of "2,40 €".
- Options de paiement:** A section for "Carte de crédit/débit" with a "link" button and a "Utiliser un autre moyen de paiement" button.

The sidebar on the right, titled "Résumé de la commande", lists items and their prices:

- Beanie: 20,00 € (discounted to 18,00 €)
- Beanie with logo: 20,00 € (discounted to 18,00 €)
- Belt: 65,00 € (discounted to 55,00 €)

The sidebar also shows a "Sous-total" of 91,00 €, a "Livraison GoMile" of 2,40 €, and a "Total" of 93,40 €. A "Stripe" logo is visible in the bottom right corner.

FIGURE 2.11 – Validation de commande avec la méthode de livraison GoMile sélectionnée.

2.2 Résultat attendu

Lors du passage de commande, le plugin calcule les frais de livraison via l'API GoMile, puis ajoute le montant obtenu au total de la commande. Après validation, la commande WooCommerce est transmise à l'API GoMile afin de créer une livraison.

La commande peut ensuite être suivie côté administration WordPress grâce aux notes de commande et aux informations de livraison stockées par le plugin. Les mises à jour de statut envoyées par GoMile sont reçues par webhook et reportées dans WooCommerce.

3 Plateforme web

La partie web de GoMile sert aux commerçants pour gérer leurs boutiques, leurs commandes et leurs clés d'intégration, et aux livreurs pour créer leur dossier d'inscription. L'interface administrateur permet de contrôler les comptes commerçants et livreurs, et vérifier les documents KYC des livreurs.

- `src/app` : pages de l'application et routes internes Next.js
- `src/app/merchant` : interface commerçant
- `src/app/driver` : parcours d'inscription livreur
- `src/app/admin` : interface d'administration

- `src/lib` : fonctions partagées, proxy API et gestion de session
- `src/components` : composants réutilisables de l'interface
- `cypress` : scénarios de tests end-to-end
- `public` : images et ressources statiques.

3.1 Lancement local

Pour utiliser l'interface web en local, il faut lancer trois éléments : la base de données, l'API et le front web.

1. Démarrer PostgreSQL et Redis :

```
docker compose -f infra/docker-compose.yml up -d
```

Si vous rencontrez un problème de permission, exécutez la même commande avec `sudo` :

```
sudo docker compose -f infra/docker-compose.yml up -d
```

2. Préparer la base de données :

```
pnpm --filter api exec prisma generate
pnpm --filter api exec prisma migrate dev
```

3. Lancer l'API NestJS :

```
pnpm --filter api start:dev
```

4. Lancer le front Next.js :

```
pnpm --filter web dev
```

3.2 Déploiement en production

En production, le front web n'est pas lancé comme en local avec la commande de développement. Le dépôt fournit un script de déploiement qui prépare l'application et redémarre les services du serveur :

```
bash infra/deploy.sh
```

Le script suit ce flux :

- il s'exécute depuis la racine du projet
- il récupère la dernière version du code avec `git pull`
- il installe les dépendances avec `pnpm install`
- il prépare l'API avec Prisma et les migrations de base de données
- il compile l'API, puis le front Next.js avec `pnpm -filter web build`
- il redémarre les services `gomile-api` et `gomile-web`
- le front tourne ensuite en production sur le port 3001 du serveur
- l'URL publique du front est `https://gomile.delivery`, exposée avec Nginx.

En résumé, pour mettre en ligne une nouvelle version du front, il suffit de lancer le script de déploiement sur le serveur. Le script s'occupe de récupérer le code, construire l'application et relancer le service web.

3.3 Accueil et choix de l'espace

La première interface oriente l'utilisateur vers l'espace adapté à son profil. Un commerçant rejoint l'espace marchand pour gérer ses magasins et ses commandes. Un livreur accède au parcours d'inscription livreur pour créer son dossier.



FIGURE 2.12 – Accueil web et choix entre l'espace commerçant et l'espace livreur

3.4 Connexion commerçant

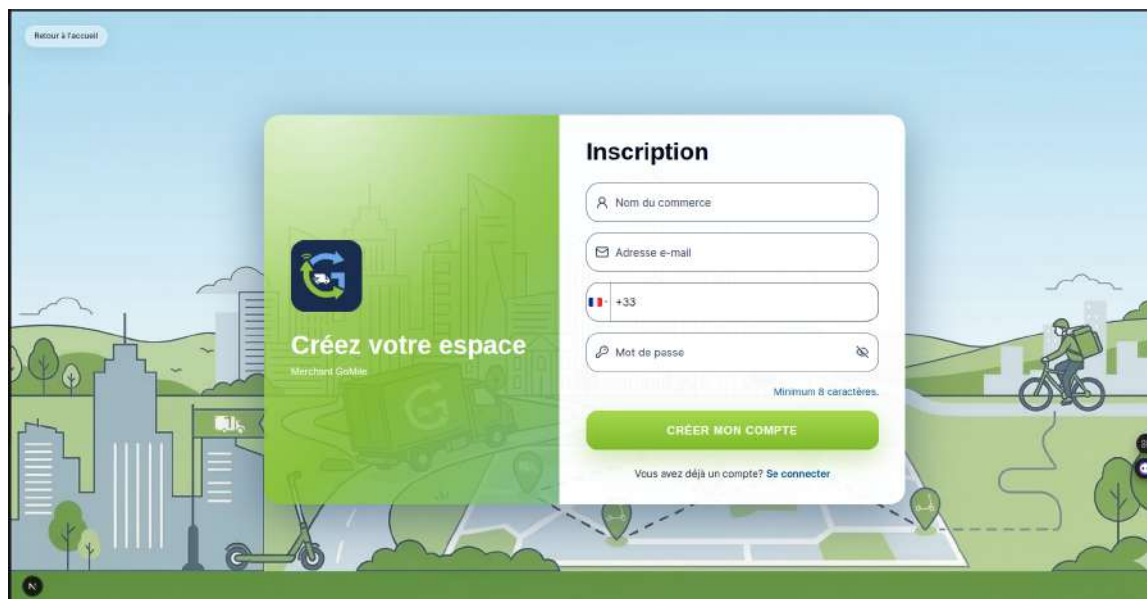
Le commerçant se connecte depuis la page de connexion marchand. Il saisit son adresse e-mail et son mot de passe. Après validation, le front vérifie le compte et redirige l'utilisateur vers son tableau de bord si l'accès est autorisé.



FIGURE 2.13 – Page de connexion marchand

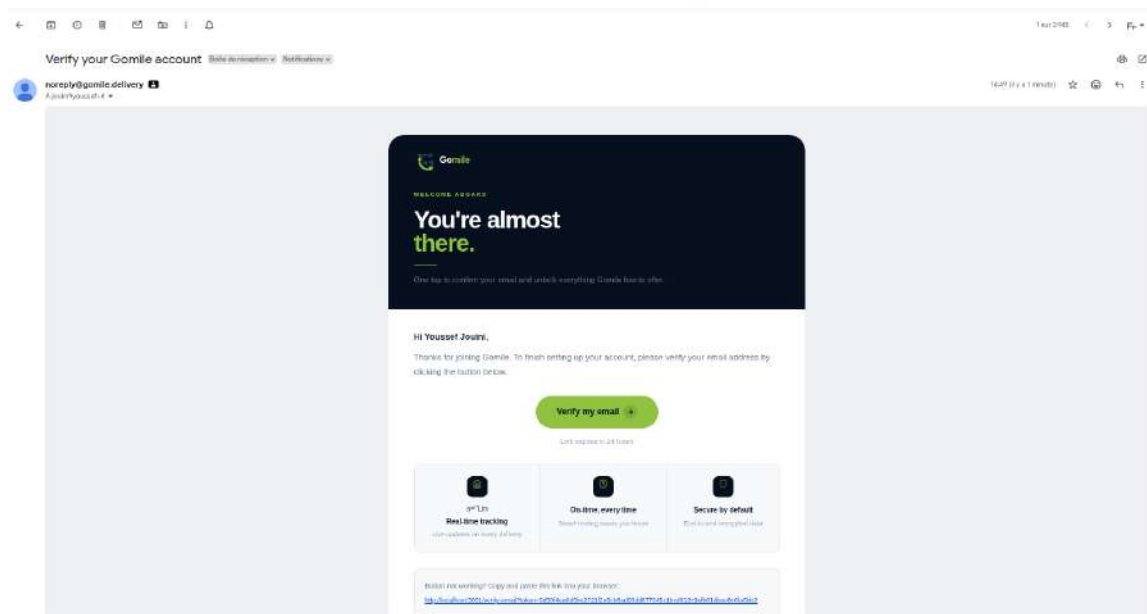
3.5 Inscription commerçant

La page d'inscription marchand permet de créer un compte pour un commerce. L'utilisateur renseigne le nom du commerce, l'adresse e-mail, le numéro de téléphone et le mot de passe. Une fois le formulaire envoyé, le compte est créé mais il doit encore être confirmé par e-mail.



3.6 Vérification de l'adresse e-mail

Après l'inscription, GoMile envoie un e-mail de vérification à l'adresse indiquée. Le commerçant doit cliquer sur le bouton de confirmation pour activer son adresse. Cette étape sécurise le compte et évite de travailler avec une adresse e-mail incorrecte.



3.7 Tableau de bord marchand

Après connexion, le marchand arrive sur le tableau de bord. Il y retrouve une vue globale de son activité : carte de suivi, notifications récentes, commandes créées et changements de statut des livraisons. C'est l'écran principal pour consulter rapidement l'état du service.

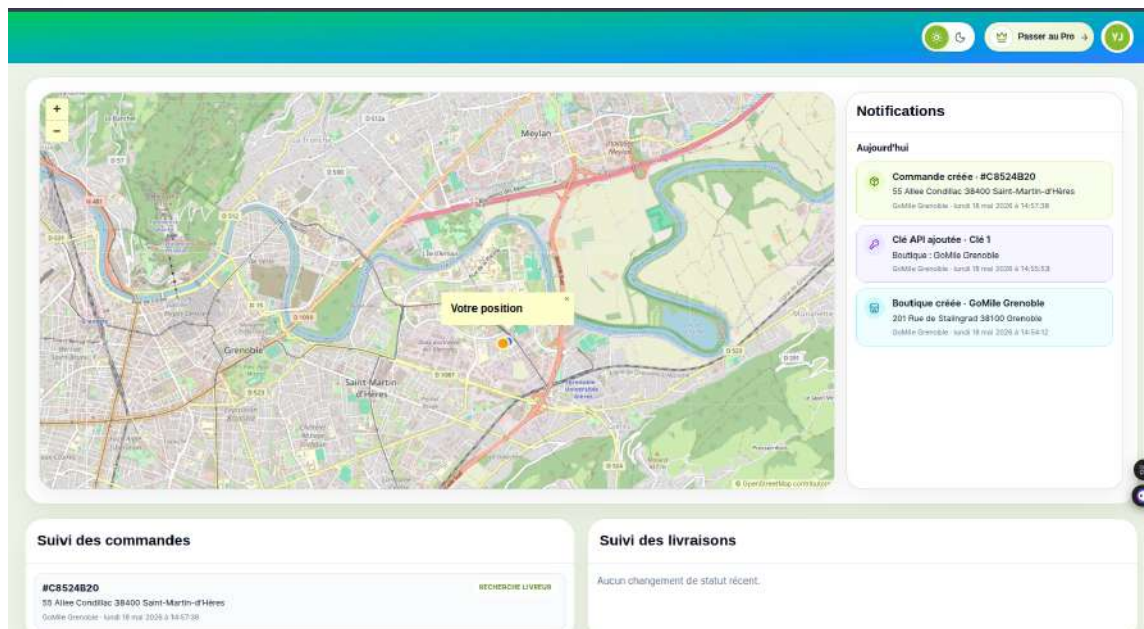


FIGURE 2.16 – Tableau de bord marchand

3.8 Gestion des magasins

L'interface de gestion des magasins liste les boutiques du commerçant. Elle permet de consulter leur statut, leur plateforme, leur nombre de commandes et les actions disponibles. Depuis cette page, le commerçant peut aussi créer une nouvelle boutique.

Détail	Magasin	Plateforme	Statut	Commandes	Création	Actions
	GoMile Grenoble	Non configuré	ACTIF	0	18 mai 2025, 14:54	[Edit] [Delete] [Add]

FIGURE 2.17 – Liste des magasins du commerçant

3.9 Création d'une boutique

Le formulaire de création de boutique demande les informations principales du magasin : nom, adresse, plateforme, domaine et description. Ces données servent ensuite à rattacher les commandes et les clés API à la bonne boutique.

Créer une boutique

Nom du magasin
GoMile Grenoble

Adresse
201 Rue de Stalingrad 38100 Grenoble

Plateforme
☐ Shopify
☐ WooCommerce
☒ Autre

Domaine
myshop.com

Description
Boutique de grenoble de GoMile

Créer **Annuler**

FIGURE 2.18 – Formulaire de création d'une boutique

3.10 Suivi des commandes

L'interface de commandes permet de créer et suivre les livraisons du commerçant. Chaque commande affiche son client, le magasin concerné, l'adresse de livraison, le statut et les actions possibles. Le commerçant peut consulter les détails d'une commande pour vérifier les informations importantes.

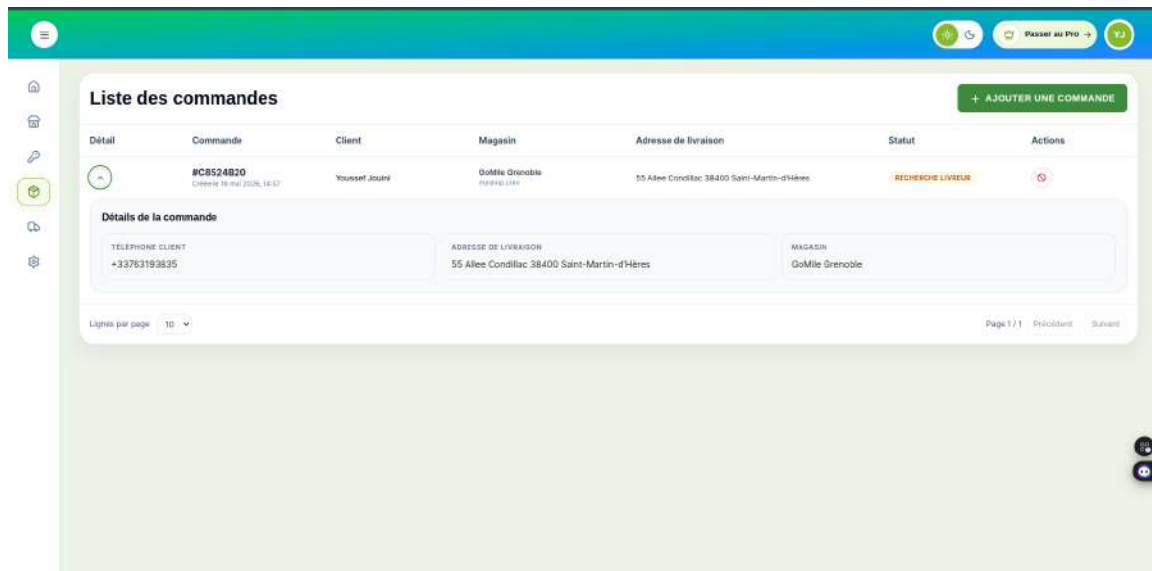


FIGURE 2.19 – Liste et détail des commandes marchand

3.11 Gestion des clés API

L'interface des clés API permet de créer et gérer les clés utilisées par les intégrations externes, par exemple un plugin e-commerce. Chaque clé est liée à une boutique précise, ce qui permet de séparer les accès entre plusieurs magasins.

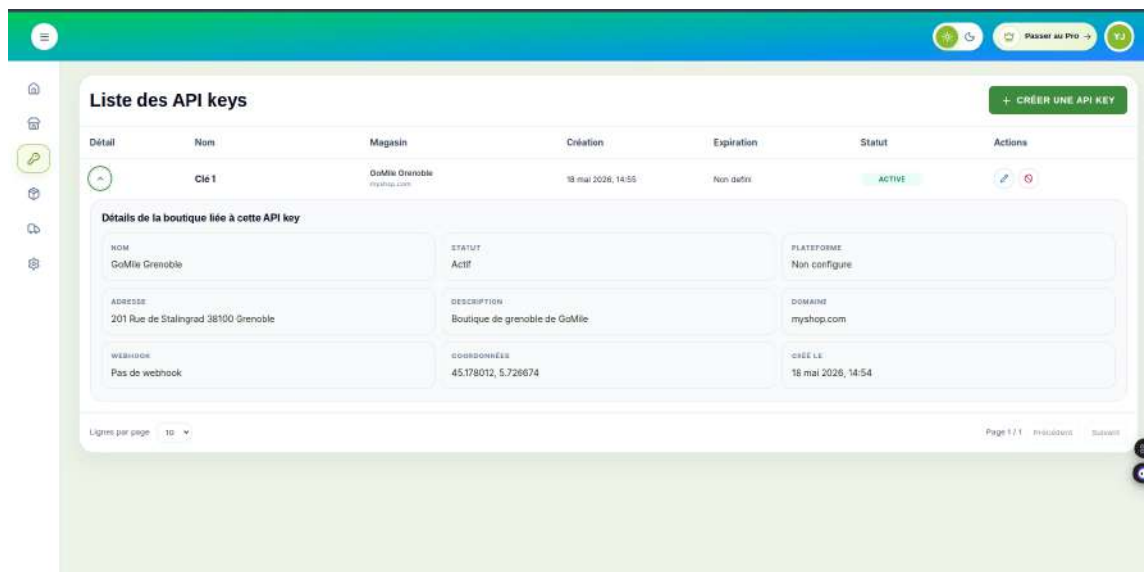


FIGURE 2.20 – Gestion des clés API liées aux boutiques

3.12 Inscription livreur web

La partie livreur web est centrée sur un formulaire en plusieurs étapes. Le découpage rend l'inscription plus lisible : le livreur commence par son identité, complète ensuite son profil, précise sa zone de livraison, puis ajoute les documents nécessaires au contrôle de son dossier.

FIGURE 2.21 – Inscription livreur : identité et contact

La deuxième étape complète les informations personnelles du livreur. Elle regroupe notamment la date de naissance, le genre et l'adresse. Ces informations permettent de

construire un profil complet avant la vérification administrative.

Retour à l'accueil

Inscription livreur

1 Identité 2 **Profil** 3 Livraison 4 Documents

Profil livreur

Date de naissance: 18/05/2026 Genre: Homme

55 Allée Condillac 38400 Sai Saint-Martin-d'Hères

38400 55 Allée Condillac

ÉTAPE PRÉCÉDENTE ÉTAPE SUIVANTE

Vous avez déjà fini votre dossier? [Retour à l'accueil](#)

FIGURE 2.22 – Inscription livreur : profil

La troisième étape décrit les conditions de livraison. Le livreur indique sa ville de livraison, son rayon d'intervention et son moyen de transport. Ces données servent à déterminer dans quelle zone il pourra recevoir des missions.

Retour à l'accueil

Inscription livreur

1 Identité 2 Profil 3 **Livraison** 4 Documents

Livraison

Grenoble 20

Voiture

ÉTAPE PRÉCÉDENTE ÉTAPE SUIVANTE

Vous avez déjà fini votre dossier? [Retour à l'accueil](#)

FIGURE 2.23 – Inscription livreur : zone de livraison et transport

La dernière étape concerne les documents justificatifs. Le livreur ajoute les fichiers demandés et peut renseigner son numéro SIRET si nécessaire. Le dossier est ensuite transmis pour vérification côté administration.

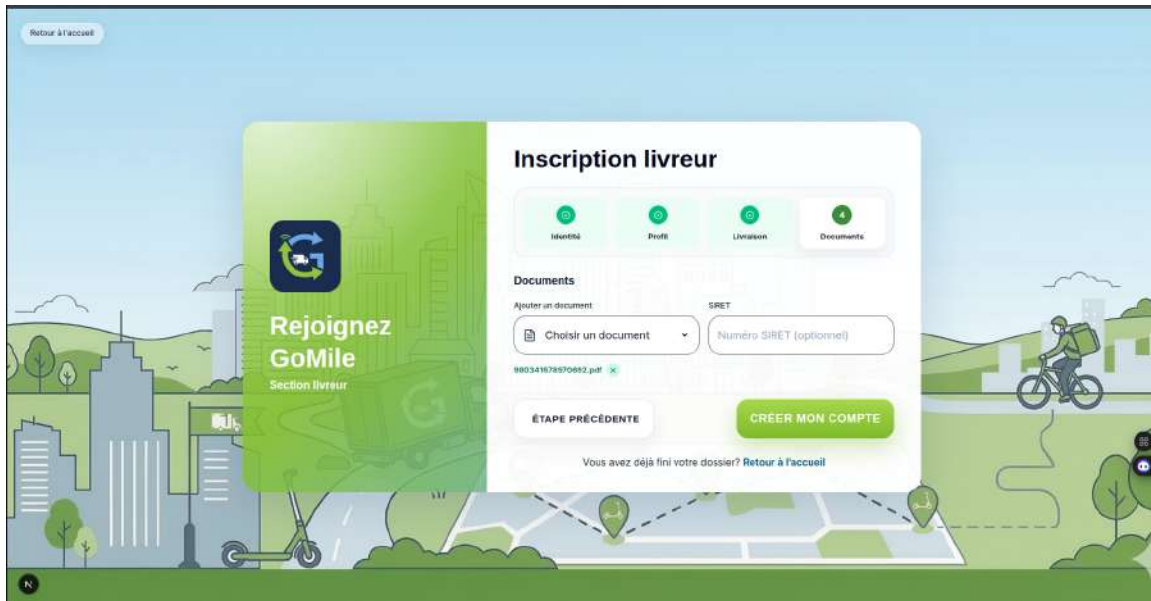


FIGURE 2.24 – Inscription livreur : documents justificatifs

4 Vérifications du front web

Cette partie décrit les commandes utiles pour contrôler que le front fonctionne correctement. Les commandes sont à lancer depuis la racine du dépôt.

4.1 Tests Cypress

Cypress sert à tester les parcours utilisateur dans un navigateur. Avant de lancer les tests, il faut que l'application soit construite et servie, ou que le serveur de développement soit déjà disponible sur le port configuré.

Une séquence simple pour tester le front en conditions proches de la production est :

```
pnpm --filter web build
pnpm --filter web start
pnpm --filter web cypress:run
```

Pour ouvrir l'interface interactive de Cypress :

```
pnpm --filter web cypress:open
```

Pour lancer toute la suite Cypress en mode automatique :

```
pnpm --filter web cypress:run
```

Pour exécuter tous les scénarios front présents dans le dossier Cypress :

```
pnpm --filter web test:front
```

Pour cibler uniquement les scénarios liés à l'espace marchand :

```
pnpm --filter web test:merchant
```

4.2 Typecheck et qualité du code

Le typecheck vérifie que le code TypeScript est cohérent sans générer de fichiers de sortie. La commande globale du front est :

```
pnpm --filter web typecheck
```

Cette commande lance deux contrôles :

- `typecheck:app` vérifie le code de l'application avec `tsc -noEmit`
- `typecheck:cypress` vérifie les tests Cypress avec `tsc -project cypress/tsconfig.json -noEmit`.

Ces deux contrôles peuvent aussi être lancés séparément :

```
pnpm --filter web typecheck:app  
pnpm --filter web typecheck:cypress
```

Les commandes de lint contrôlent la qualité du code TypeScript, React et CSS :

```
pnpm --filter web lint  
pnpm --filter web lint:tsx  
pnpm --filter web lint:css
```

Pour lancer les vérifications principales du front en une seule commande :

```
pnpm --filter web quality
```

Cette commande exécute le lint TypeScript/React, le lint CSS et le typecheck.

4.3 Build de validation

Le build vérifie que l'application Next.js peut être compilée pour la production :

```
pnpm --filter web build
```

Après un build réussi, l'application peut être lancée en mode production avec :

```
pnpm --filter web start
```

4.4 Technologies utilisées

Recharts

Recharts est utilisé pour afficher les graphiques du tableau de bord administrateur. Cette bibliothèque permet de créer des visualisations de données avec React, comme des graphiques en barres ou des graphiques circulaires.

Dans la partie admin, les graphiques rendent certaines données plus rapides à lire pour l'administrateur. Ils servent par exemple à visualiser :

- la répartition des livreurs par type de véhicule ;
- les statuts KYC des livreurs ;
- les livraisons réalisées sur une période donnée ;
- la répartition des commerçants par abonnement ;
- le nombre de commerces ajoutés.

On a choisi Recharts car elle s'intègre naturellement à React et c'est une bibliothèque qui est assez simple à prendre en main et propose pas mal de composants déjà prêt à être utilisés en vue du temps limité du projet malgré qu'elle offre moins de flexibilité.

Lucide React

Lucide React sert à intégrer des icônes dans l'interface. Ces icônes rendent les actions plus faciles à reconnaître, notamment dans les boutons, les menus et les pages de détail. Elle permet de conserver un style graphique cohérent et léger. Les icônes sont importées comme des composants React, ce qui simplifie leur personnalisation avec des tailles, des couleurs ou des classes CSS.

Composants internes du design system

En plus des bibliothèques externes, la partie admin réutilise plusieurs composants déjà présents dans le projet, comme les boutons, les champs de saisie, la navigation, le footer ou la typographie. Cette réutilisation aide à garder une cohérence visuelle avec le reste de l'application GoMile.

Les composants comme `Button`, `Input`, `Typography`, `Navigation` ou `Navbar` évitent de recréer les mêmes éléments plusieurs fois. Ils facilitent aussi les évolutions de l'interface, puisqu'une modification dans un composant partagé peut être répercutée sur plusieurs pages.

5 Plateforme web-Admin

Ce manuel décrit les fonctionnalités accessibles à un administrateur depuis l'interface web de GoMile. Il couvre la connexion, la navigation dans le tableau de bord et l'ensemble des actions disponibles.

5.1 Connexion à l'espace administrateur

L'espace d'administration est accessible depuis la route `/admin/login`. Pour se connecter, l'administrateur doit renseigner :

- son adresse e-mail ;
- son mot de passe.

Si l'un des champs est vide, un message d'erreur s'affiche et aucune requête n'est envoyée. En cas d'identifiants incorrects ou de compte non administrateur, la session est annulée et un message d'erreur est affiché. Après une connexion réussie, l'administrateur est automatiquement redirigé vers le tableau de bord `/admin/dashboard`.



FIGURE 2.25 – Page de connexion administrateur

5.2 Navigation dans le tableau de bord

Après connexion, l'administrateur accède au tableau de bord principal. Un menu latéral permanent permet de naviguer entre les différentes sections :

- **Vue d'ensemble** : synthèse des livreurs et des commerçants ;
- **Livreurs** : gestion et détail des comptes livreurs ;
- **Commerçants** : suivi des comptes commerçants ;

- **Demandes de retrait** : traitement des demandes de démission ;
- **Conflits** : renvoi du handshake pour les commandes en conflit ;
- **Ajouter un membre** : création de nouveaux comptes.

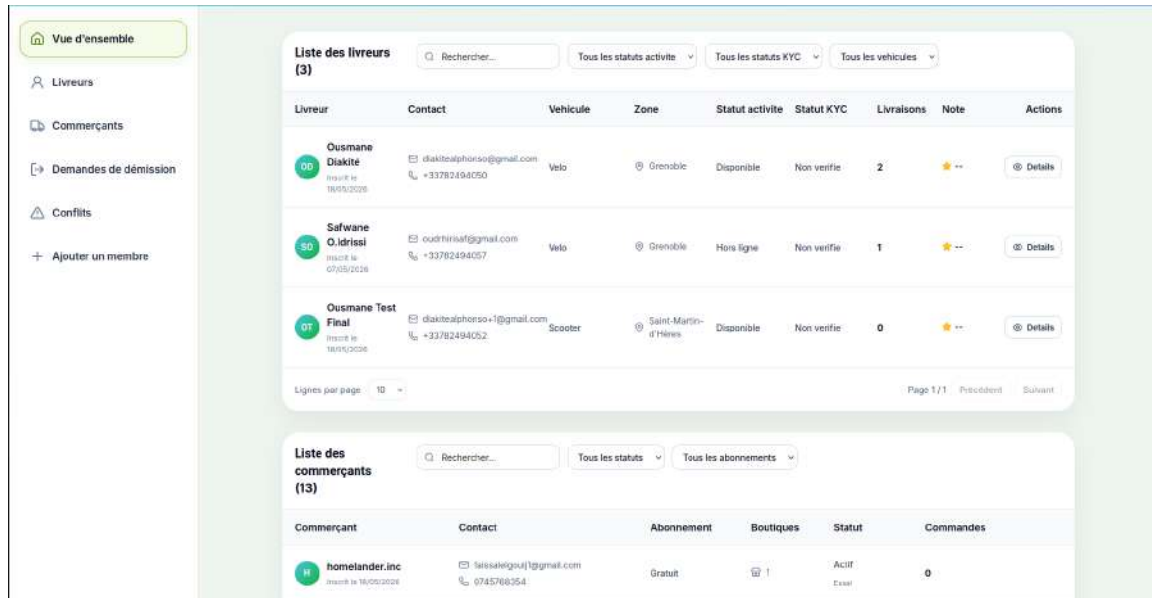


FIGURE 2.26 – Vue d’ensemble du tableau de bord administrateur

5.3 Consultation et gestion des livreurs

La section **Livreurs** affiche la liste complète des livreurs inscrits sur la plateforme. Pour chaque livreur, le tableau présente :

- le nom et les informations de contact ;
- le véhicule utilisé ;
- la zone de livraison ;
- le statut d’activité et le statut KYC ;
- le nombre de livraisons effectuées et la note.

Une barre de recherche et des filtres permettent de retrouver rapidement un livreur précis ou de restreindre la liste selon un statut, un état KYC ou un type de véhicule.

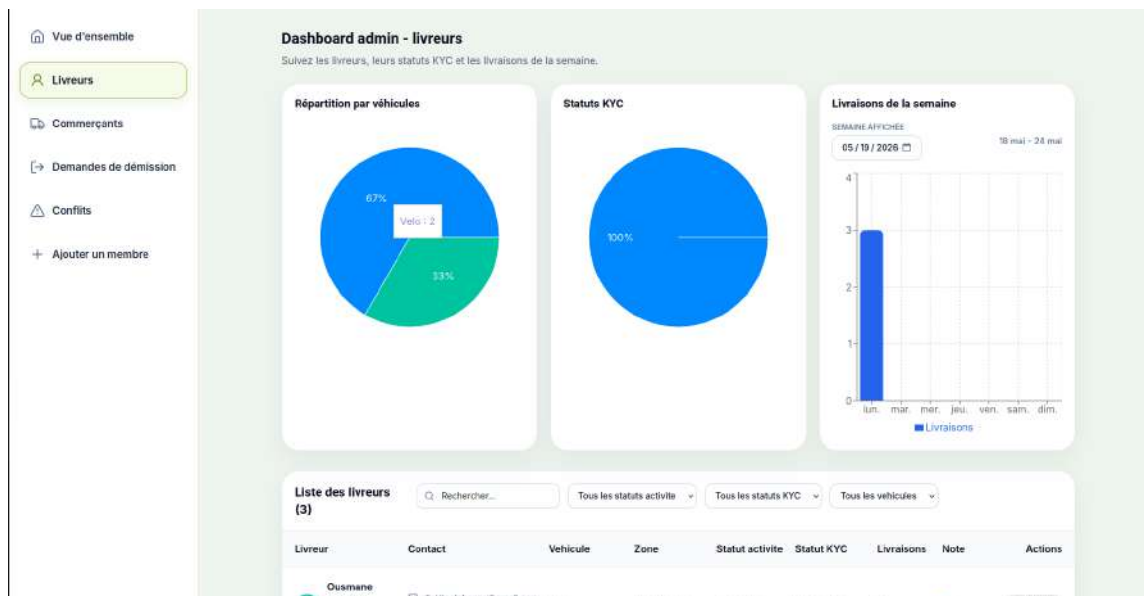


FIGURE 2.27 – Tableau des livreurs

Fiche détaillée d'un livreur

En cliquant sur un livreur, l'administrateur accède à sa fiche complète (/admin/dashboard/drivers/[driverId]). Cette page regroupe :

- l'identité du livreur (nom, photo, coordonnées) ;
- ses statistiques d'activité ;
- son historique de soumissions KYC ;
- ses documents KYC à vérifier.

Validation ou refus des documents KYC

Depuis la fiche d'un livreur, l'administrateur peut traiter les documents KYC transmis. Pour chaque document, il peut :

- ouvrir le document dans un nouvel onglet ;
- le marquer comme **accepté** ;
- le marquer comme **refusé**.

Acceptation du KYC. Lorsque tous les documents sont marqués comme acceptés, le bouton de validation finale devient disponible. L'administrateur confirme la décision : la page se met à jour automatiquement et un message de succès s'affiche.

Refus du KYC. Si au moins un document est refusé, un champ de saisie apparaît pour indiquer le motif de refus. Ce motif est obligatoire avant de pouvoir confirmer la décision. Après validation, la page se recharge et un message de confirmation s’affiche.

The screenshot displays the KYC (Know Your Customer) section for a driver's profile. At the top, personal information is shown: 'Inscription 07/05/2026', 'Adresse 2 Avenue Jules Vallès 38100 Grenoble', 'Zone de livraison Grenoble (25 km)', and 'SIRET Non renseigné'. Below this, a 'Documents KYC' section lists four documents: 'CNI', 'DRIVING_LICENSE', 'REGISTRATION_CARD', and 'RIB', each with an 'Ajouté le 07/05/2026' timestamp. For each document, there are 'Accepter' (green) and 'Refuser' (red) buttons, along with an 'Ouvrir' (blue) button. The 'RIB' document's 'Accepter' button is highlighted with a green border. At the bottom, a 'Motif de refus' section contains a text input field labeled 'Précisez le motif de refus du KYC' and 'ACCEPTER' (green) and 'REJECT' (red) buttons.

FIGURE 2.28 – Section KYC dans la fiche d’un livreur

5.4 Consultation des commerçants

La section **Commerçants** affiche la liste des comptes commerçants avec les informations suivantes :

- le nom du commerçant et ses coordonnées ;
- le type d’abonnement souscrit ;
- le nombre de boutiques associées ;
- le statut du compte et le nombre de commandes.

Comme pour les livreurs, une barre de recherche et des filtres par statut ou abonnement sont disponibles.

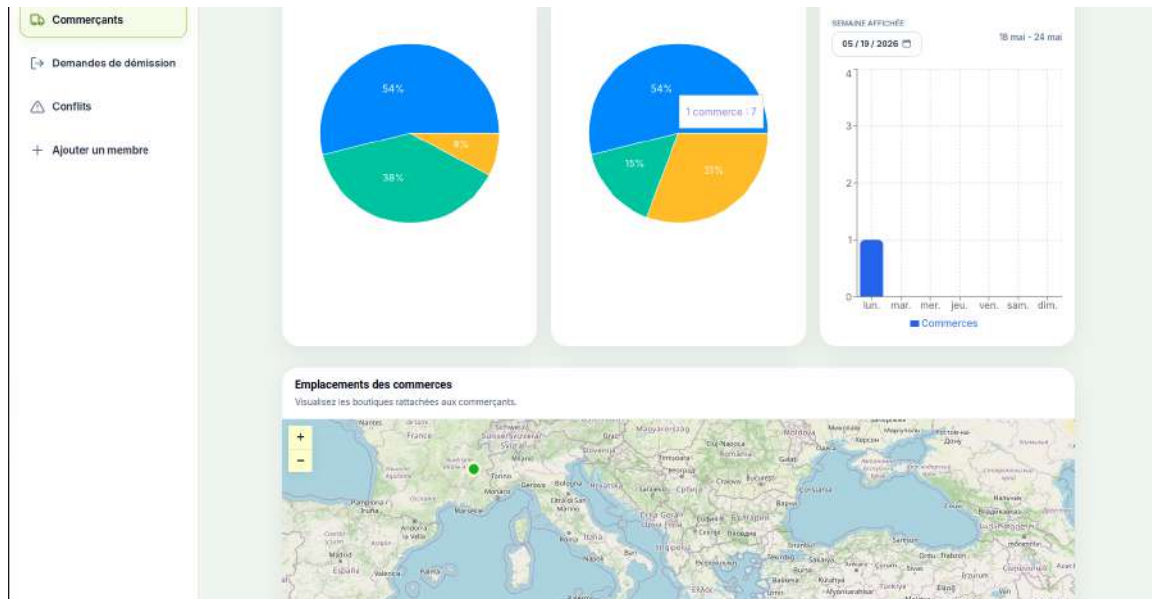


FIGURE 2.29 – Tableau des commerçants

5.5 Traitement des demandes de retrait

La section **Demandes de retrait** (/admin/dashboard/withdrawals) liste les demandes de démission effectuées par les livreurs. Par défaut, le tableau affiche uniquement les demandes en attente. L'administrateur peut filtrer par statut (*En attente*, *Acceptée*, *Refusée*) ou rechercher une demande par nom, e-mail, montant ou statut.

Pour chaque demande en attente, deux actions sont disponibles :

- **Accepter** : valide la demande de démission (statut COMPLETED) ;
- **Refuser** : annule la demande (statut CANCELLED).

Le tableau se met à jour immédiatement après chaque action, sans rechargement complet de la page.

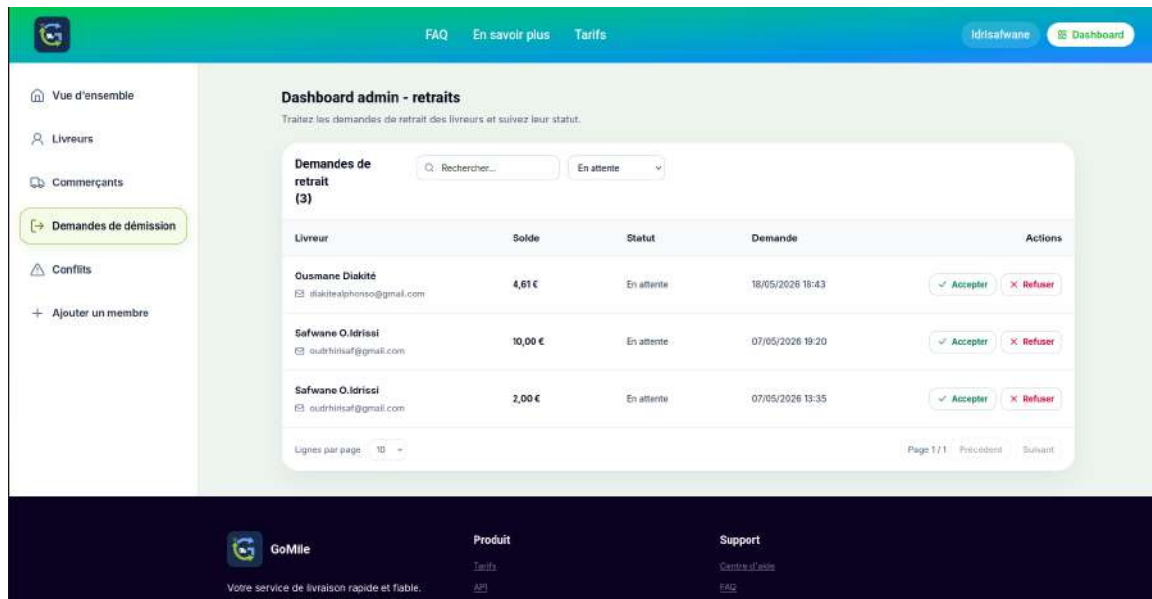


FIGURE 2.30 – Page de gestion des demandes de retrait

5.6 Renvoi du handshake d'une commande

La section **Conflits** (/admin/dashboard/conflicts) permet de relancer le handshake d'une commande bloquée. La procédure est la suivante :

1. Saisir l'identifiant de la commande concernée dans le champ prévu.
2. Cliquer sur le bouton de validation.

Si le champ est vide, un message d'erreur s'affiche. Pendant le traitement, le bouton est désactivé pour éviter les soumissions multiples. Le résultat (succès ou erreur) s'affiche ensuite directement dans la page.

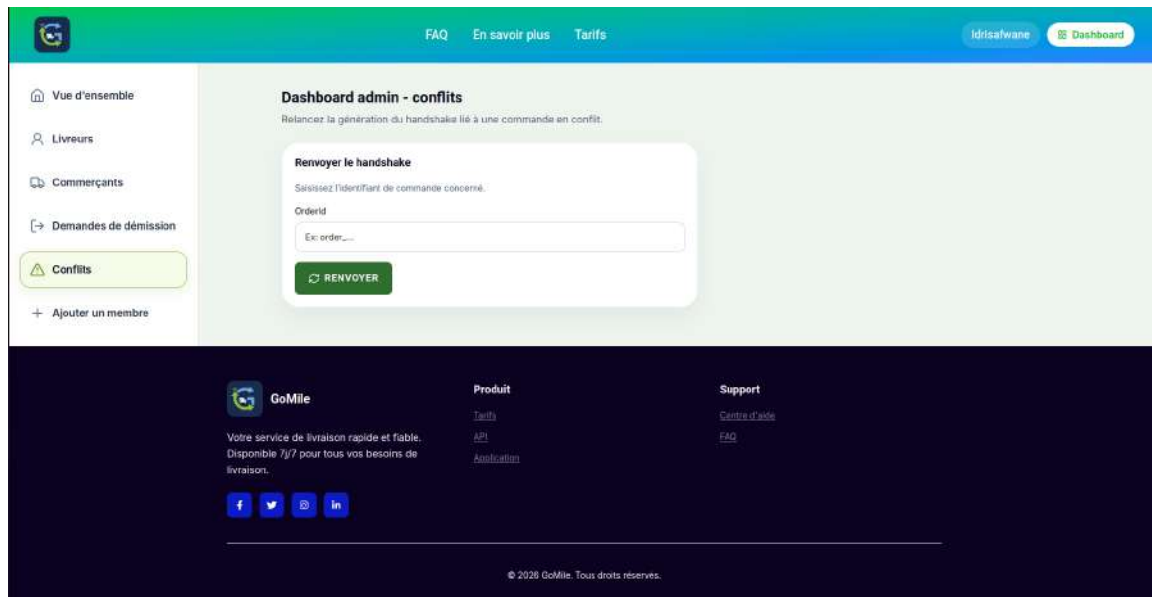


FIGURE 2.31 – Page de renvoi du handshake

5.7 Création d'un nouveau compte

La section **Ajouter un membre** (/admin/dashboard/admins) permet de créer un compte directement depuis l'interface admin. L'administrateur sélectionne d'abord le type de compte à créer :

- **Commerçant** : nom du commerce, e-mail, mot de passe, téléphone (optionnel) ;
- **Livreur** : prénom, nom, e-mail, mot de passe, téléphone, date de naissance, adresse, ville et rayon de livraison, genre, moyen de transport, URL de l'avatar ;
- **Administrateur** : e-mail et mot de passe.

Le formulaire s'adapte dynamiquement au type sélectionné. Avant l'envoi, le frontend vérifie que les champs obligatoires sont renseignés, que le mot de passe comporte au moins huit caractères et, pour un livreur, que le rayon de livraison est un entier supérieur ou égal à 1.

Après une création réussie, le formulaire est réinitialisé et un message de succès s'affiche.

The screenshot displays the GoMile admin dashboard. At the top, a navigation bar includes links for 'FAQ', 'En savoir plus', and 'Tarifs', along with a user profile 'Idrissawane' and a 'Dashboard' button. A left sidebar contains menu items: 'Vue d'ensemble', 'Livres', 'Commerçants', 'Demandes de démission', 'Conflits', and a highlighted 'Ajouter un membre' button. The main content area is titled 'Dashboard admin - ajouter un membre' with a subtitle 'Créez un compte commerçant, livreur ou administrateur depuis un seul formulaire.' Below this is a 'Nouveau membre' form. The form has a 'Type de membre' dropdown set to 'Commerçant'. It includes fields for 'Nom du commerce' (filled with 'Restaurant Soleil'), 'Email' (filled with 'membre@gonile.fr'), 'Mot de passe' (with a note '8 caractères minimum'), and 'Téléphone' (filled with '06 12 34 56 78'). A green button at the bottom of the form reads 'Ajouter un membre'.

GoMile

Votre service de livraison rapide et fiable. Disponible 7j/7 pour tous vos besoins de livraison.

Produit

Tarifs

API

Application

Support

Centre d'aide

FAQ

f t i in

FIGURE 2.32 – Page de création d'un compte depuis l'espace admin

Chapitre 3

Back-end

1 Stack technique et choix de conception

1.1 Routage et géocodage : OpenRouteService

Le routage et le géocodage sont assurés par **OpenRouteService (ORS)**, un moteur open source basé sur les données OpenStreetMap. Son quota gratuit — 2 000 requêtes de géocodage et 2 000 requêtes d’itinéraires par jour — est suffisant pour notre charge, et l’API REST retourne du GeoJSON standard, ce qui facilite une éventuelle migration.

Pour limiter le volume d’appels externes, tous les résultats sont **mis en cache dans Redis** :

- Clé de cache géocodage : `ors:geocode:{adresse_normalisée}`
- Clé de cache itinéraire : `ors:route:{lon1,lat1}:{lon2,lat2}` (6 décimales)
- L’adresse est normalisée avant la recherche en cache : suppression des espaces superflus, passage en minuscules

En pratique, les commandes répétées vers la même destination — cas fréquent pour les pharmacies — ne sollicitent l’API ORS qu’une seule fois.

1.2 Stockage des fichiers : Amazon S3

Les documents des livreurs (CNI, permis de conduire, RIB) sont stockés sur **Amazon S3**. La base de données ne conserve que l’URL du fichier, pas les octets. Cette séparation offre plusieurs avantages :

- **Pas de gonflement de la base** : chaque document fait 1–5 Mo ; S3 absorbe ce volume à \$0,023/Go/mois sans affecter les performances PostgreSQL.
- **Déchargement réseau** : les fichiers sont servis directement depuis S3, sans passer par le pool de connexions de l’API.

- **Extensibilité** : S3 peut être mis derrière CloudFront si besoin de CDN.

Le pattern implémenté est celui des **URL pré-signées PUT** : l'API génère une URL signée temporaire (validité 5 minutes), la renvoie au client, et le client envoie les octets directement sur S3. Le serveur API ne voit jamais les octets du fichier.

Les fichiers sont déposés dans le préfixe de staging `uploads/`. Une règle de cycle de vie S3 les supprime automatiquement après 2 jours. Lors de l'enregistrement officiel d'un document, le service appelle `commitFile()`, qui copie l'objet de `uploads/` vers le préfixe permanent `documents/` via une copie côté serveur S3, puis supprime l'objet de staging.

1.3 SMS : Twilio

L'envoi de SMS est géré par **Twilio**. Trois critères ont guidé ce choix :

- Opérationnel immédiatement à la création du compte, sans période de validation sandbox
- Tarif compétitif (\$0,0079/SMS sortant vers la France)
- SDK simple, sans dépendance au support d'un fournisseur cloud pour débloquer l'envoi en production

Détail comportemental : les numéros français commençant par 0 sont normalisés au format international +33 avant l'envoi (*ex.* 0612345678 → +33612345678).

1.4 Hébergement : Raspberry Pi 4 avec Cloudflare Tunnel

L'application est hébergée sur un **Raspberry Pi 4** sous Raspberry Pi OS. Cette infrastructure à coût nul présente la contrainte habituelle : les FAI grand public attribuent des adresses IP dynamiques et bloquent les connexions entrantes sur les ports 80/443.

Cloudflare Tunnel (`cloudflared`) résout ce problème en créant un tunnel chiffré sortant du Pi vers le réseau de bordure de Cloudflare. Cloudflare expose ensuite publiquement les domaines avec HTTPS/TLS, protection DDoS et gestion DNS, sans redirection de port ni IP statique.



FIGURE 3.1 – Nos "Serveur" fait maison

GoMile

1.5 Autres choix techniques

Technologie	Rôle
NestJS 11	Système de modules imposant la séparation des responsabilités. Conteneur d'injection de dépendances intégré, définition déclarative des routes et guards, TypeScript natif.
Prisma ORM	Schema-first : <code>schema.prisma</code> est la source de vérité unique pour la base. Génère automatiquement les types TypeScript, gère les migrations. SQL brut supporté pour les requêtes PostGIS.
PostgreSQL + PostGIS	PostGIS ajoute les types spatiaux (<code>geometry(Point)</code>) et les fonctions (<code>ST_DWithin</code>) nécessaires aux requêtes de proximité géographique.
Redis 7	Cache en mémoire pour les résultats ORS. Lectures sous la milliseconde, évitant de dépasser le quota gratuit sur les requêtes répétées.
Socket.io 4	WebSocket avec fallback automatique en long-polling. Mises à jour temps réel du statut des commandes.
Stripe	Processeur de paiement. Pages Checkout et Customer Portal pré-construites.
Expo Push	Notifications push multiplateformes (iOS + Android) via un seul appel API.
React Email	Templates email en JSX compilés côté serveur en HTML.
Resend	Fournisseur d'envoi d'emails avec bonne délivrabilité et API simple.

2 Architecture abstraite

2.1 Vue d'ensemble des couches applicatives

Le système est organisé en quatre couches. Le tableau de bord web passe obligatoirement par le BFF, tandis que l'application mobile et les intégrations tierces appellent l'API directement.

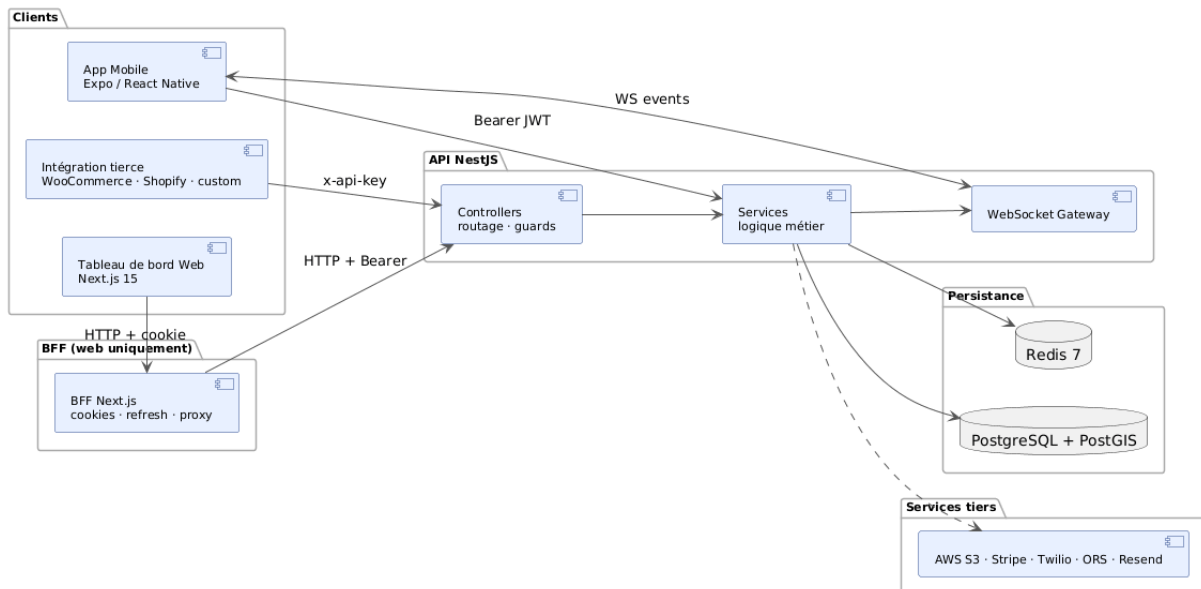


FIGURE 3.2 – Architecture abstraite — couches applicatives et flux principaux

3 Architecture de déploiement

3.1 Vue d'ensemble de l'infrastructure

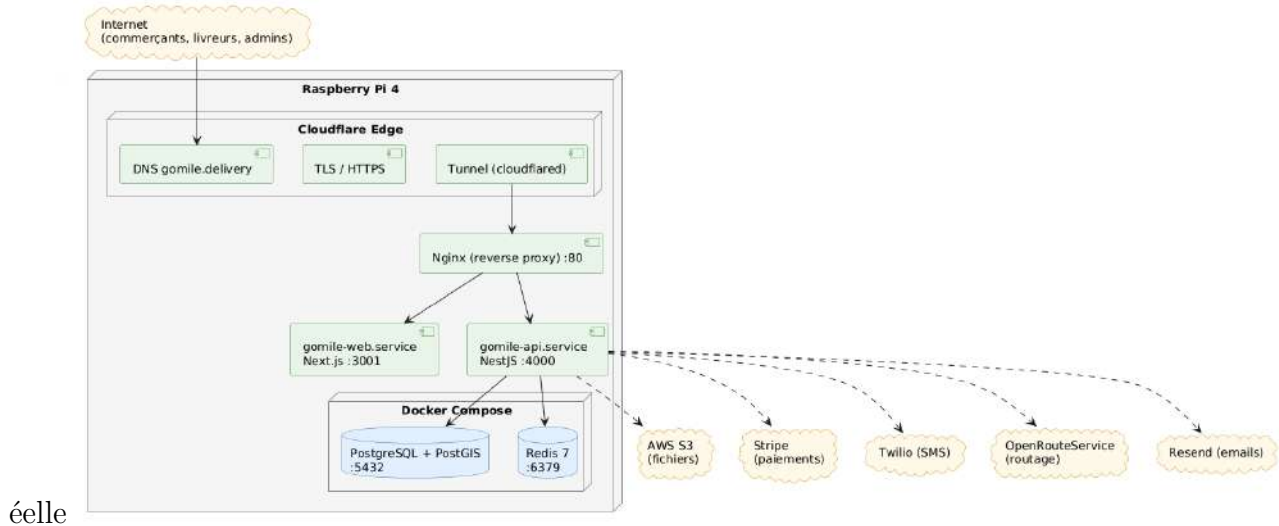


FIGURE 3.3 – Infrastructure complète de déploiement GoMile

3.2 Configuration Nginx

Nginx écoute sur le port 80 (le TLS est terminé chez Cloudflare). Deux hôtes virtuels sont définis :

Domaine	Upstream	Notes
gomile.delivery	127.0.0.1:3001	Assets statiques <code>/_next/static/</code> mis en cache 1 an (immutable). <code>www.</code> redirigé vers le domaine racine.
api.gomile.delivery	127.0.0.1:4000	<code>client_max_body_size</code> 20M. En-tête <code>WebSocket Upgrade</code> transmis.

Les deux hôtes transmettent `X-Real-IP`, `X-Forwarded-For` et `X-Forwarded-Proto` pour préserver l'adresse IP réelle du client.

3.3 Gestion des processus (systemd)

Les deux services applicatifs sont gérés par `systemd` :

```
# gomile-api.service
ExecStart=/usr/bin/node dist/main
WorkingDirectory=/home/dangddk/Ensimag/gomile/apps/api
EnvironmentFile=.env
Restart=always RestartSec=5s
After=network.target postgresql.service redis.service

# gomile-web.service
ExecStart=pnpm --filter web start
WorkingDirectory=/home/dangddk/Ensimag/gomile
Environment=NODE_ENV=production PORT=3001
Restart=always RestartSec=5s
After=network.target gomile-api.service
```

La contrainte `After=gomile-api.service` garantit que l'API est démarrée avant le frontend web.

3.4 Déploiement continu (webhook GitLab)

Un webhook GitLab se déclenche à chaque push sur la branche `dev`. Un contrôleur NestJS dédié valide l'en-tête `X-Gitlab-Token` et exécute `infra/deploy.sh` :

```
git pull && pnpm install
pnpm exec prisma generate && pnpm exec prisma migrate deploy
pnpm --filter api build && pnpm --filter web build
sudo systemctl restart gomile-api gomile-web
```

Un push sur `dev` provoque la mise à jour complète du Pi et le redémarrage des services en ≈ 60 secondes.

4 Pipeline CI/CD

Le pipeline GitLab CI (`.gitlab-ci.yml`) s'exécute en trois étapes : **validate**, **build** et **security**. Le runner utilise Node 22 avec pnpm 10.30.3 (Corepack). Le dossier `.pnpm-store` et `node_modules` sont mis en cache entre les exécutions.

Job	Étape	Action
lint	validate	<code>pnpm -r lint</code> — ESLint sur tous les packages
typecheck	validate	<code>pnpm -r typecheck</code> — <code>tsc --noEmit</code>
test_api	validate	<code>pnpm --filter api test:ci</code> — suite Jest
build_api	build	<code>pnpm --filter api build</code> — produit <code>dist/</code>
build_web	build	<code>pnpm --filter web build</code> — produit <code>.next/</code>
mobile_check	build	Expo lint + typecheck + validation build EAS
audit	security	<code>pnpm audit --prod</code> — non bloquant

Les jobs `validate` s'exécutent en parallèle. Les jobs `build` dépendent du succès complet de `validate`. L'audit est `allow_failure` pour ne pas bloquer une release sur un avis de dépendance transitive.

5 Schéma de base de données

La base de données est PostgreSQL 16 avec l'extension PostGIS 3.4. Prisma est utilisé comme ORM. Le schéma est organisé en quatre groupes fonctionnels.

5.1 Groupe 1 : Authentification et utilisateurs

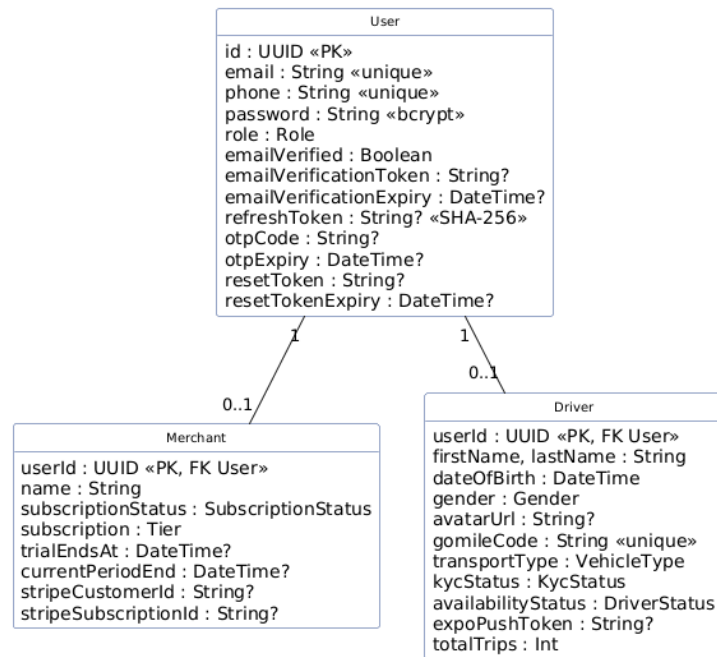


FIGURE 3.4 – Entités d’authentification et de profil utilisateur

5.2 Groupe 2 : Commandes et livraison

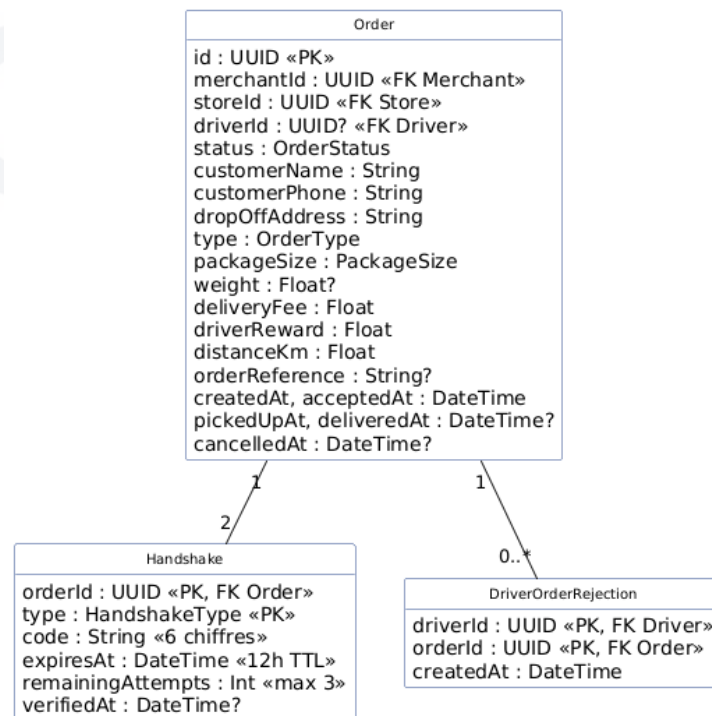


FIGURE 3.5 – Entités de gestion des commandes

5.3 Groupe 3 : Magasins et intégrations API



FIGURE 3.6 – Entités de gestion des magasins et des clés API

5.4 Groupe 4 : Finance et KYC

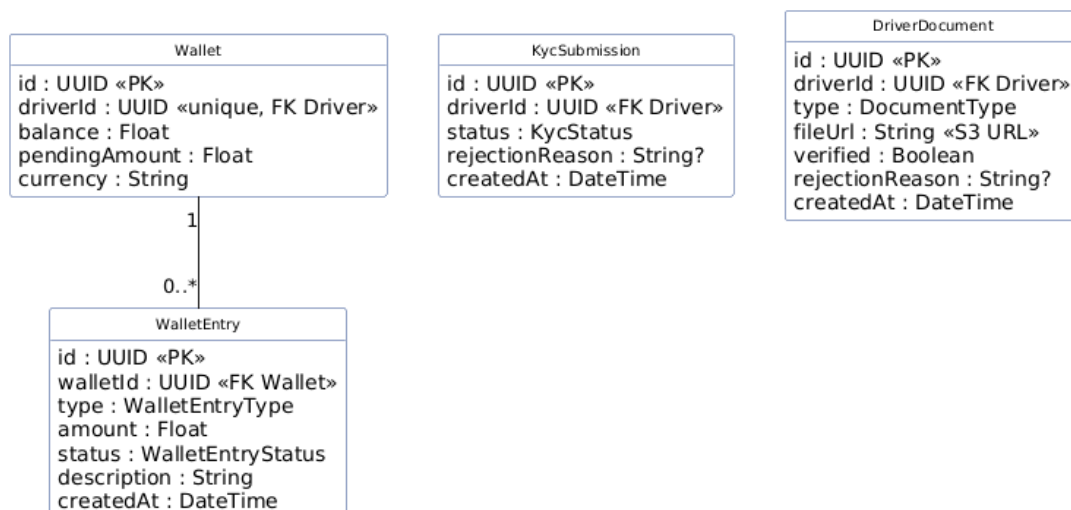


FIGURE 3.7 – Entités finance (portefeuille) et KYC

5.5 Référence des énumérations

Énumération	Valeurs
Role	ADMIN DRIVER MERCHANT CUSTOMER
OrderStatus	SEARCHING_DRIVER DRIVER_ACCEPTED PICKED_UP DELIVERED CANCELLED
SubscriptionStatus	TRIAL ACTIVE PAST_DUE LOCKED
Tier	FREE PRO BUSINESS
KycStatus	NOT_SUBMITTED PENDING ACCEPTED REJECTED
DriverStatus	AVAILABLE BUSY OFFLINE
HandshakeType	A (retrait) B (livraison)
PackageSize	SMALL MEDIUM LARGE EXTRA_LARGE
OrderType	FOOD PHARMACY GROCERY CLOTHING ELECTRONICS FURNITURE DOCUMENTS OTHER
VehicleType	CAR BIKE SCOOTER TRUCK
DocumentType	DRIVING_LICENSE REGISTRATION_CARD RIB CNI PASSPORT OTHER
WalletEntryType	CREDIT DEBIT
WalletEntryStatus	PENDING COMPLETED CANCELLED

6 Modèle du domaine

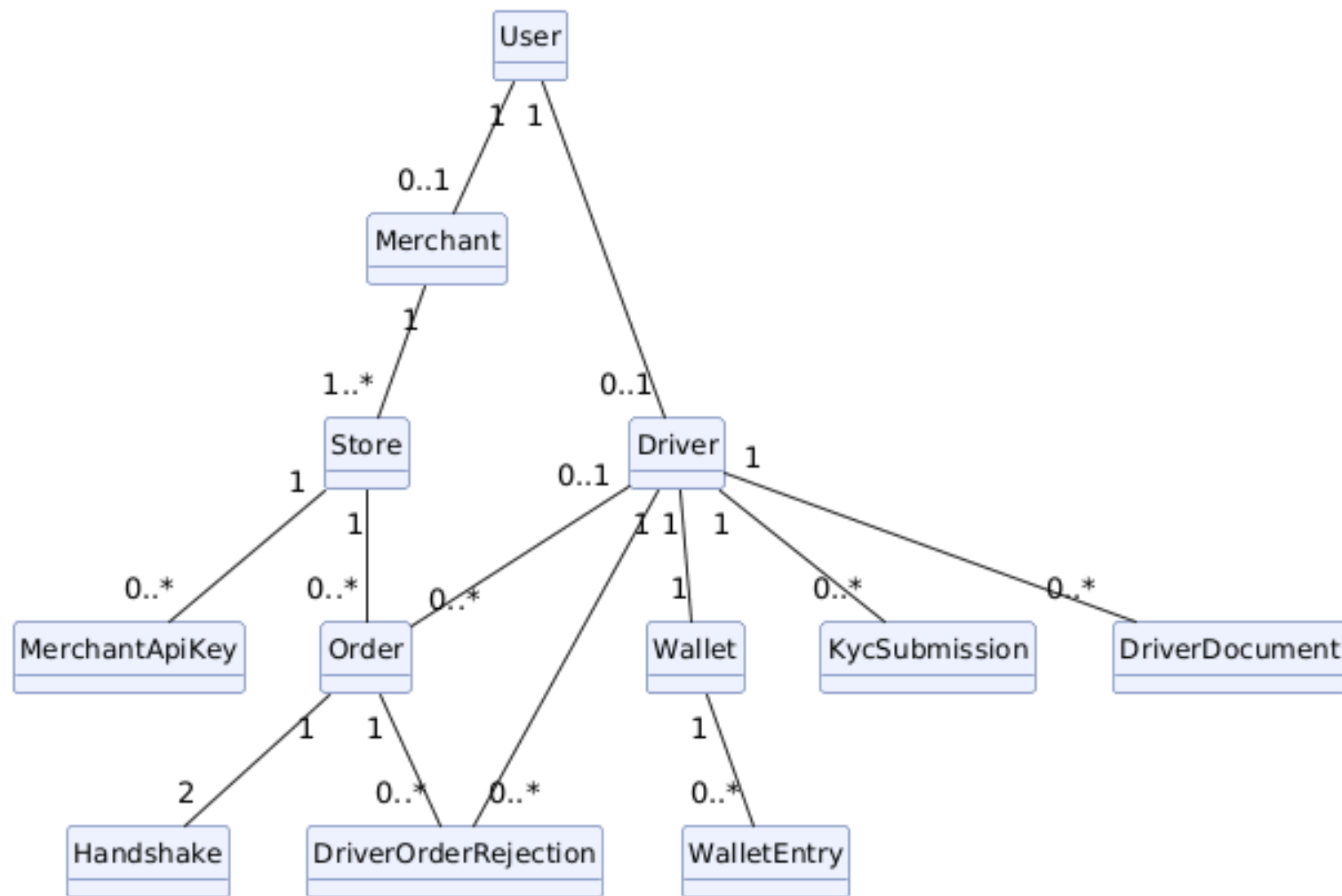


FIGURE 3.8 – Modèle du domaine — diagramme de classes UML (multiplicités aux extrémités)

7 Couche BFF (Backend-for-Frontend)

7.1 Rôle et motivations

L'application web Next.js ne communique **jamais directement** avec l'API NestJS depuis le navigateur. Toutes les requêtes authentifiées transitent par des routes API Next.js qui jouent le rôle de **serveur intermédiaire (BFF)**.

- Les tokens JWT sont stockés dans des **cookies HttpOnly**, inaccessibles au JavaScript côté client, ce qui élimine le risque de vol par injection XSS.
- La protection CSRF est assurée par **SameSite=Lax** : les cookies ne sont pas envoyés dans les requêtes cross-origin initiées par des tiers.
- L'API NestJS n'est exposée directement qu'aux applications mobiles (Expo), qui gèrent leurs propres tokens en mémoire.

7.2 Architecture des cookies d'authentification

Cookie	Durée de vie	HttpOnly	Notes
gomile_access_token	15 minutes	Oui	SameSite=Lax, Secure en production
gomile_refresh_token	7 jours	Oui	SameSite=Lax, Secure en production

7.3 Flux de requête authentifiée (proxyApiRequest)

La fonction principale du BFF est `proxyApiRequest` (`lib/api-proxy.ts`). Elle implémente un mécanisme de rafraîchissement de token transparent pour le frontend :

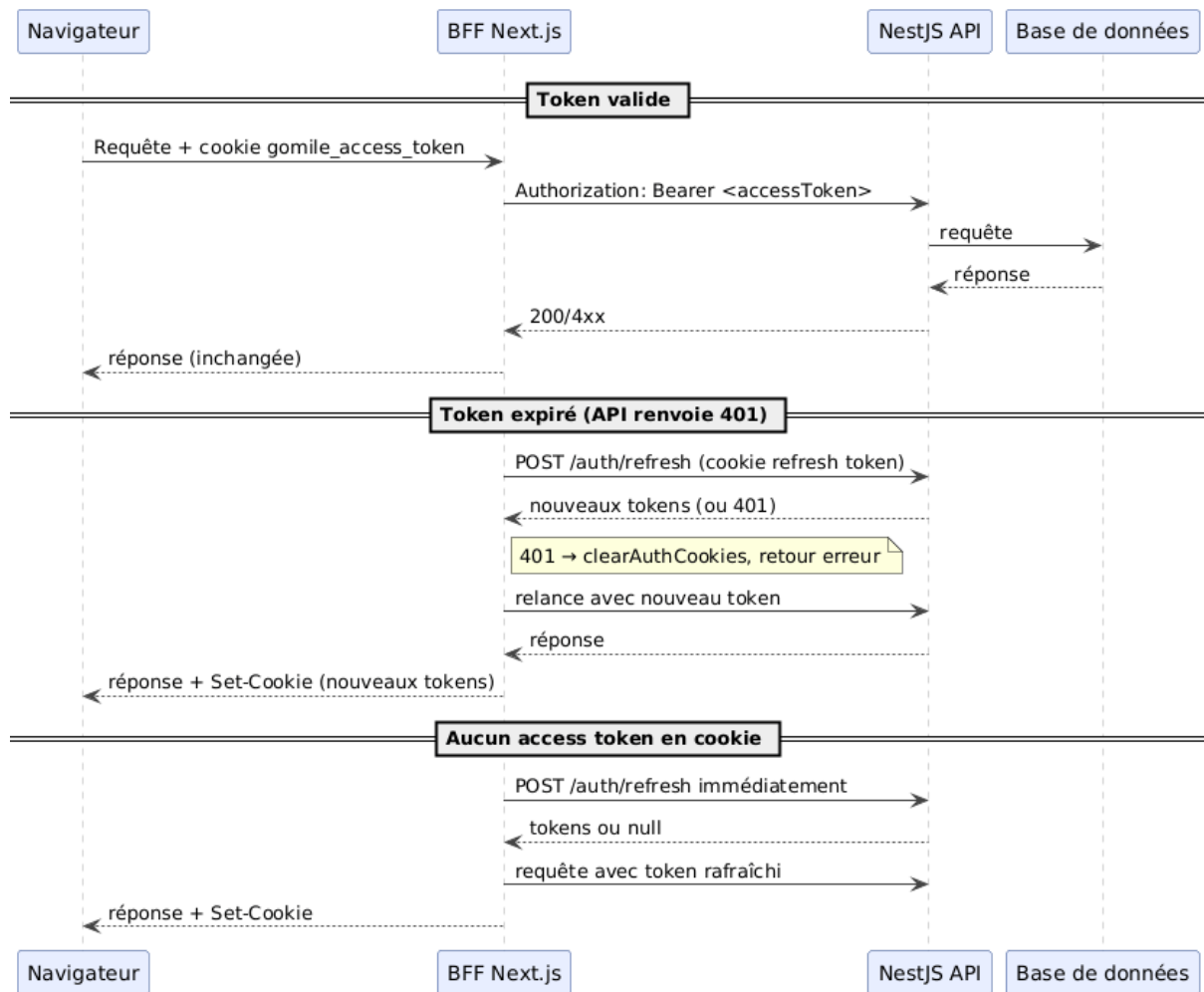


FIGURE 3.9 – Flux BFF — gestion transparente du rafraîchissement de token

7.4 Déduplication du rafraîchissement (token-refresh.ts)

Si plusieurs requêtes concurrentes reçoivent toutes un 401 simultanément, un mécanisme de singleton de promesse évite d'appeler `/auth/refresh` plusieurs fois en parallèle :

```

let inFlightRefresh: Promise<AuthSession | null> | null = null;

export async function ensureTokenRefreshed() {
  if (!inFlightRefresh) {
    inFlightRefresh = refreshSession()
      .finally(() => { inFlightRefresh = null; });
  }
  return inFlightRefresh; // toutes les requetes attendent la meme
    promesse
}
  
```

7.5 Factories de routes (route-factories.ts)

Factory	Comportement
<code>makePublicRoute(path)</code>	Transfère la requête sans authentification (<code>forwardRequest</code>)
<code>makeProtectedRoute(path)</code>	Utilise <code>proxyApiRequest</code> : lit le cookie, gère le refresh
<code>makeProtectedParamRoute(fn)</code>	Identique, avec paramètres de chemin dynamiques
<code>makeSessionCreationRoute(path)</code>	Appelle <code>proxySessionCreation</code> : intercepte les tokens, pose les cookies
<code>makeSessionRefreshRoute()</code>	Appelle <code>proxySessionRefresh</code> : renouvelle les cookies
<code>makeSessionLogoutRoute()</code>	Appelle <code>proxySessionLogout</code> : efface les cookies

7.6 Flux de connexion et déconnexion via BFF

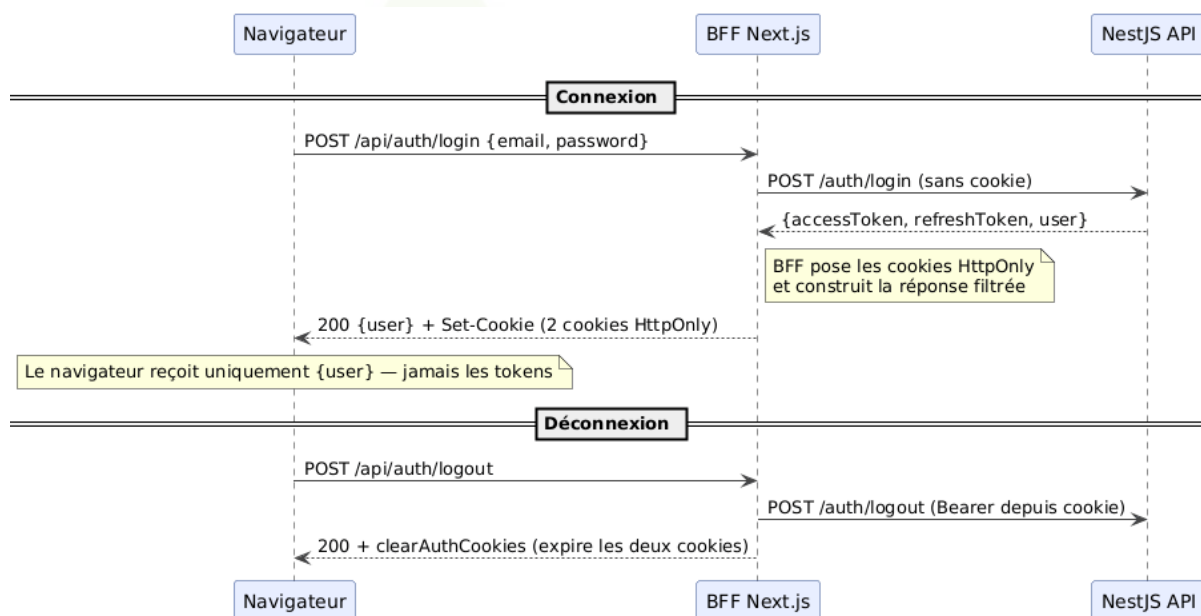


FIGURE 3.10 – Connexion et déconnexion via BFF — les tokens restent côté serveur

7.7 Contrats partagés (shared/)

Le répertoire `shared/` contient les types TypeScript partagés entre le frontend et le backend :

Fichier	Contenu
auth-contracts.ts	AuthTokensResponse, AuthSession, types de connexion/inscription
order-contracts.ts	CreateOrderInput, OrderResponse, statuts de commande
store-contracts.ts	StoreResponse, CreateStoreInput
driver-contracts.ts	DriverProfile, DriverDashboard, types mission
kyc-contracts.ts	KycStatus, DriverDocument
api-key-contracts.ts	ApiKeyResponse, types de création/révocation
subscription-contracts.ts	SubscriptionPlan, CheckoutResponse
delivery-contracts.ts	DeliveryEstimate, AddressInput
events-contracts.ts	Types des événements WebSocket
api-errors.ts	createApiError, codes d'erreur typés
auth-messages.ts	Messages de réponse auth partagés

8 Authentification et autorisation

8.1 Inscription commerçant

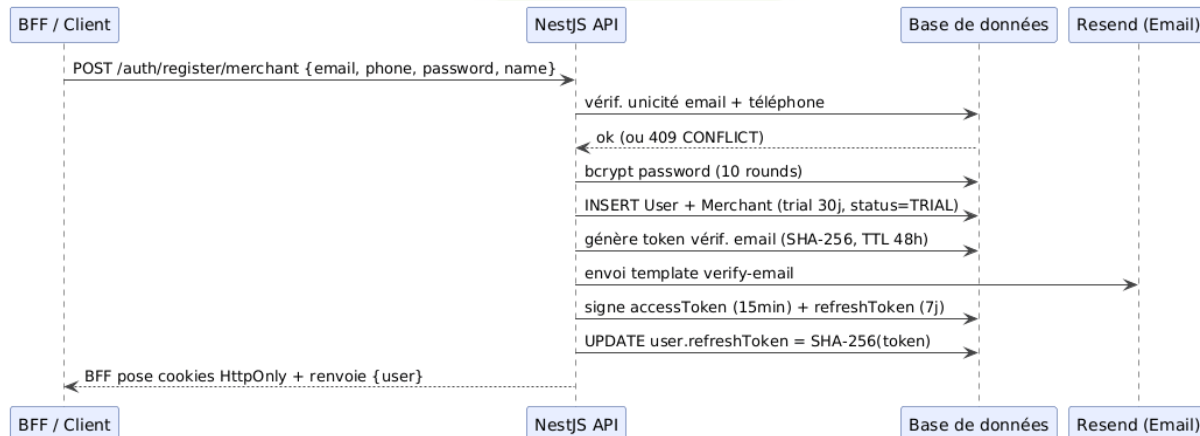


FIGURE 3.11 – Flux d'inscription commerçant

8.2 Inscription livreur — spécificités

L'inscription livreur comprend deux étapes supplémentaires par rapport à l'inscription commerçant :

1. **Commit S3 préalable** : les URLs de documents fournis (`documentsUrls[]`) sont copiées de `uploads/` vers `documents/` avant l'écriture en base. Si la transaction DB échoue, les fichiers committés sont supprimés de S3 (rollback manuel).

2. **Génération du gomileCode** : code unique au format GM-{6 hex aléatoires}-{année}, stocké sur le profil livreur.
3. La Wallet (solde initial 0) est créée dans la même transaction que User et Driver.

8.3 Connexion et rotation des tokens

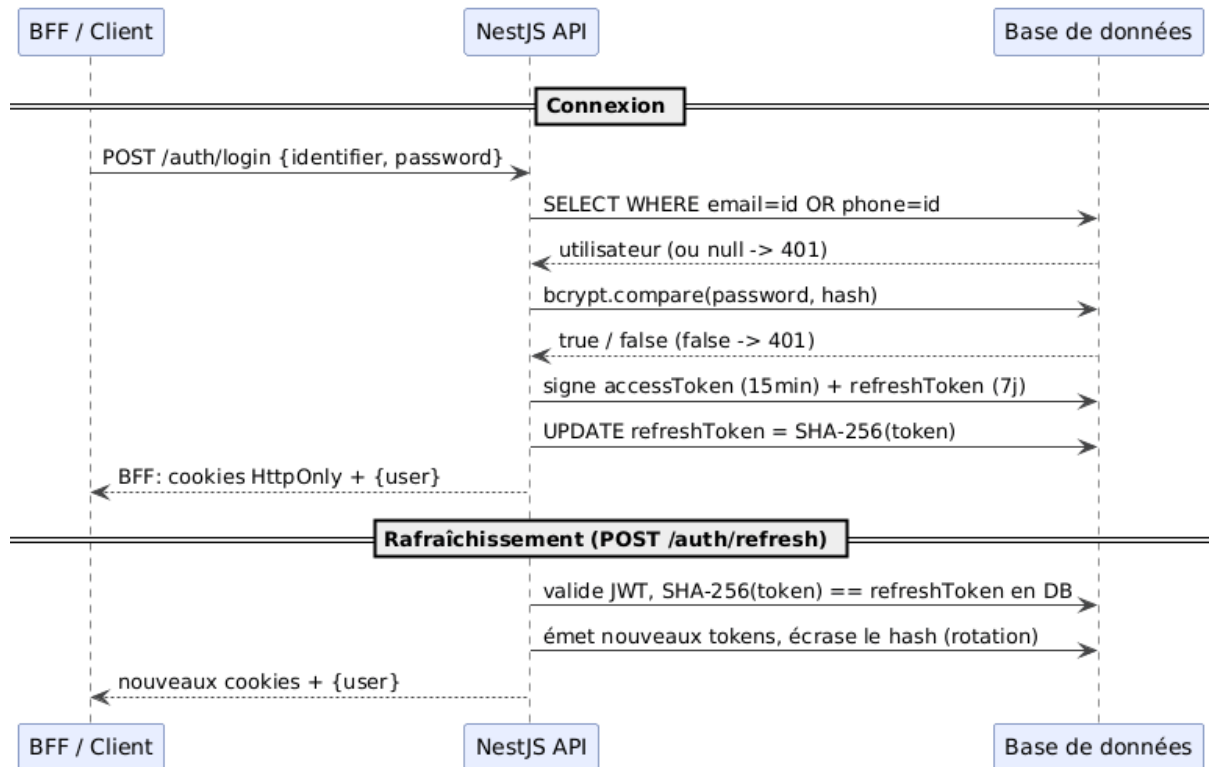


FIGURE 3.12 – Connexion et rotation du refresh token

Rotation des refresh tokens : chaque appel à **/auth/refresh** invalide l'ancien refresh token en écrasant son hash en base. Un token volé n'est donc utilisable qu'une seule fois avant que le refresh légitime du vrai utilisateur le révoque automatiquement.

8.4 Réinitialisation du mot de passe (3 étapes)

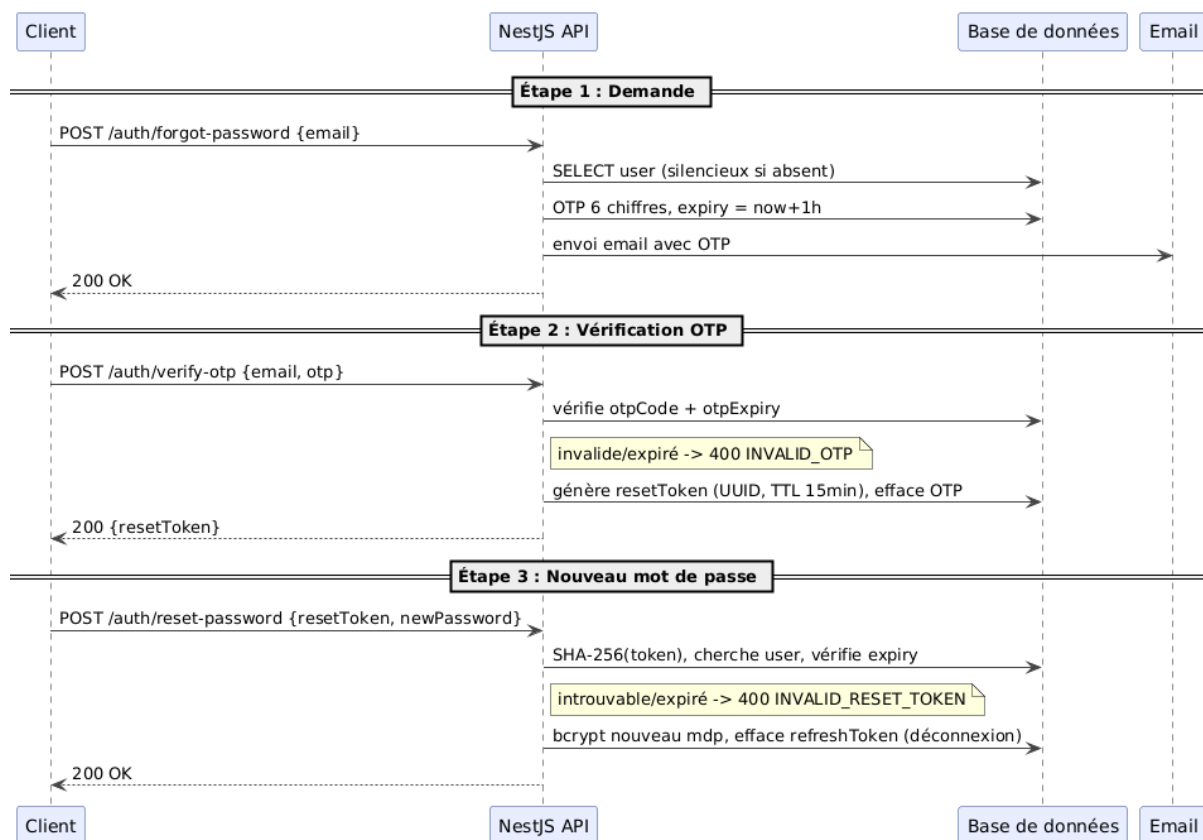


FIGURE 3.13 – Réinitialisation de mot de passe en 3 étapes

8.5 Guards et décorateurs

Guard / Stratégie	Comportement
JwtAuthGuard	Valide Authorization: Bearer avec JWT_ACCESS_SECRET. Injecte l'utilisateur complet dans <code>request.user</code> .
JwtRefreshGuard	Identique mais utilise JWT_REFRESH_SECRET et vérifie le hash stocké en base.
EmailVerifiedGuard	Rejette avec 403 si <code>user.emailVerified === false</code> .
RolesGuard	Lit les métadonnées <code>@Roles(...)</code> , rejette si <code>user.role</code> ne correspond pas.
ApiKeyGuard	Lit l'en-tête <code>x-api-key</code> , le hache en SHA-256, cherche dans <code>MerchantApiKey</code> , vérifie non révoqué/expiré, injecte <code>MerchantApiPrincipal</code> .
JwtOrApiKeyGuard	Accepte soit un JWT valide, soit une clé API valide.

9 Cycle de vie d'un token JWT

Un token JWT passe par quatre phases depuis sa création jusqu'à sa destruction. Le diagramme suivant illustre les trois scénarios de terminaison possibles.

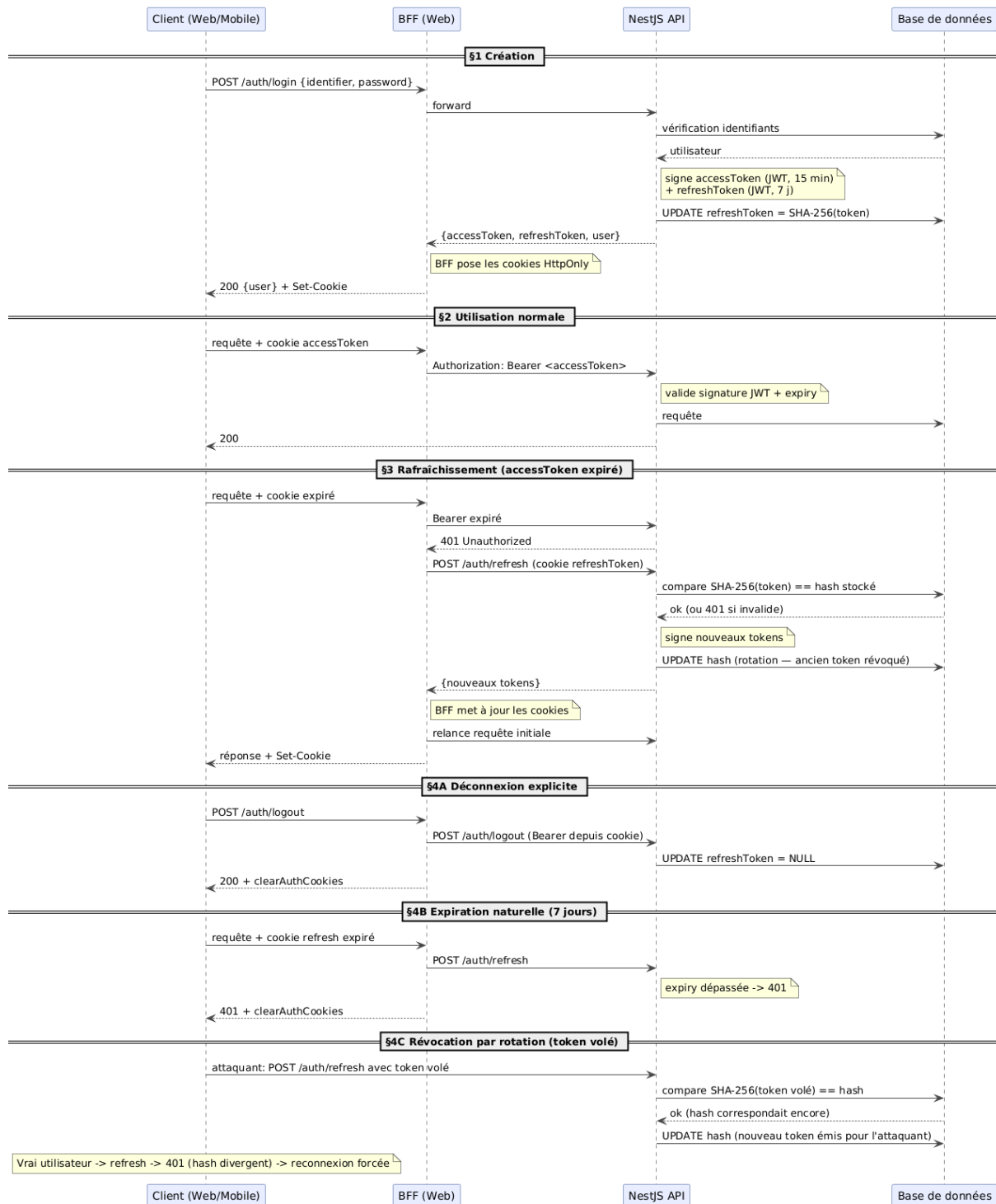


FIGURE 3.14 – Cycle de vie d'un token JWT — création, utilisation, rafraîchissement, destruction

Propriété de rotation : chaque refresh invalide le token précédent. Si un token est volé et utilisé par un attaquant *avant* le légitime, le vrai utilisateur sera bloqué lors de son prochain refresh — signal d’alerte détectable.



10 Cycle de vie d'une commande

10.1 Flux métier complet

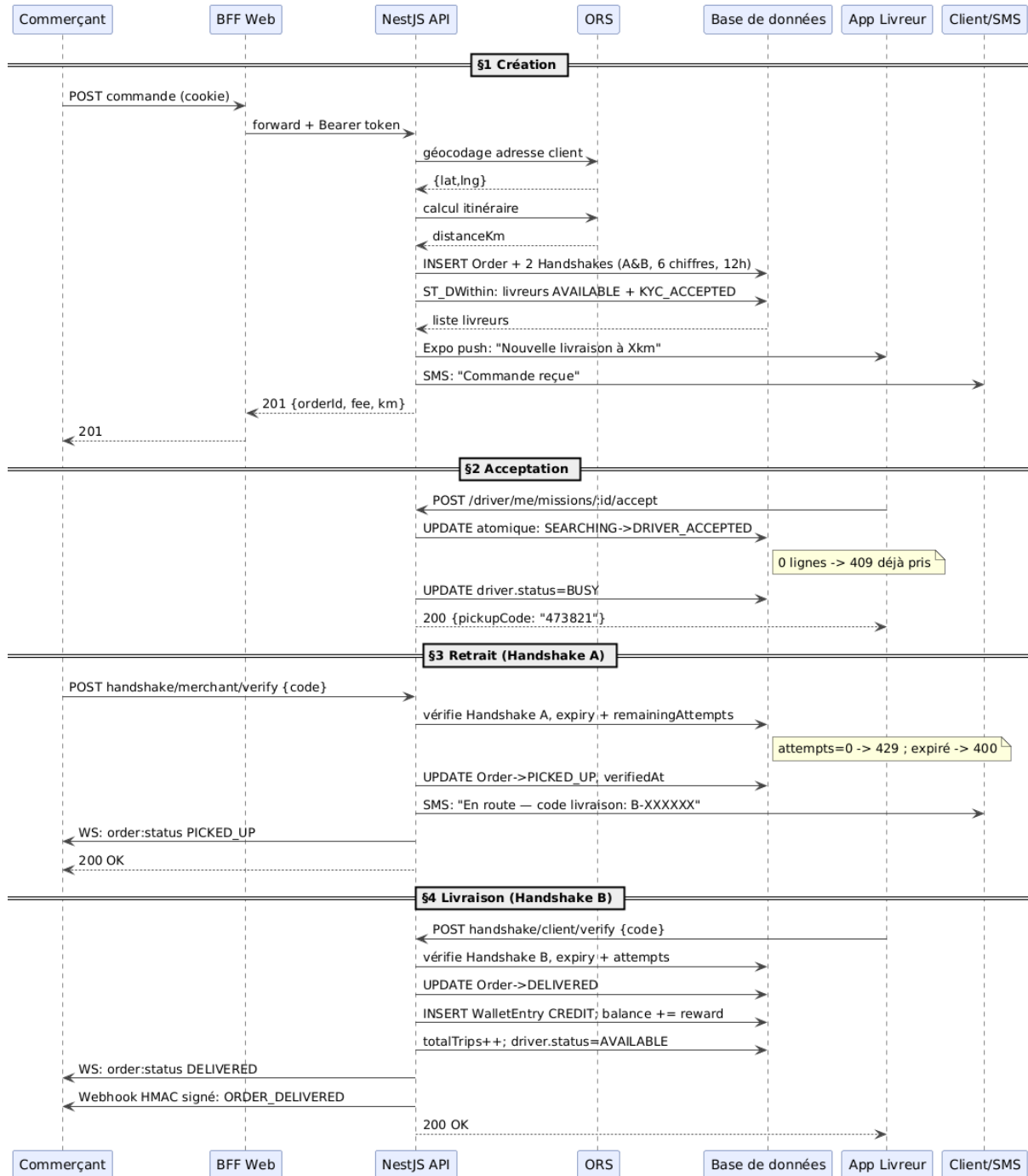


FIGURE 3.15 – Flux métier complet d'une commande — de la création à la livraison

10.2 Annulation de commande

POST /merchants/:merchantId/orders/:orderId/cancel

L'annulation n'est possible que depuis les statuts `SEARCHING_DRIVER` ou `DRIVER_ACCEPTED`. Une fois la commande en `PICKED_UP`, le livreur est déjà en transit et l'annulation n'est plus disponible. En cas de succès :

1. Statut $\rightarrow$ `CANCELLED`, `cancelledAt` = `now`
2. Si un livreur était assigné : `driver.availabilityStatus` = `AVAILABLE`
3. Émission d'un événement WebSocket `order:status`
4. Déclenchement du webhook sortant (`ORDER_CANCELLED`)



11 Machines à états

11.1 Statut de commande

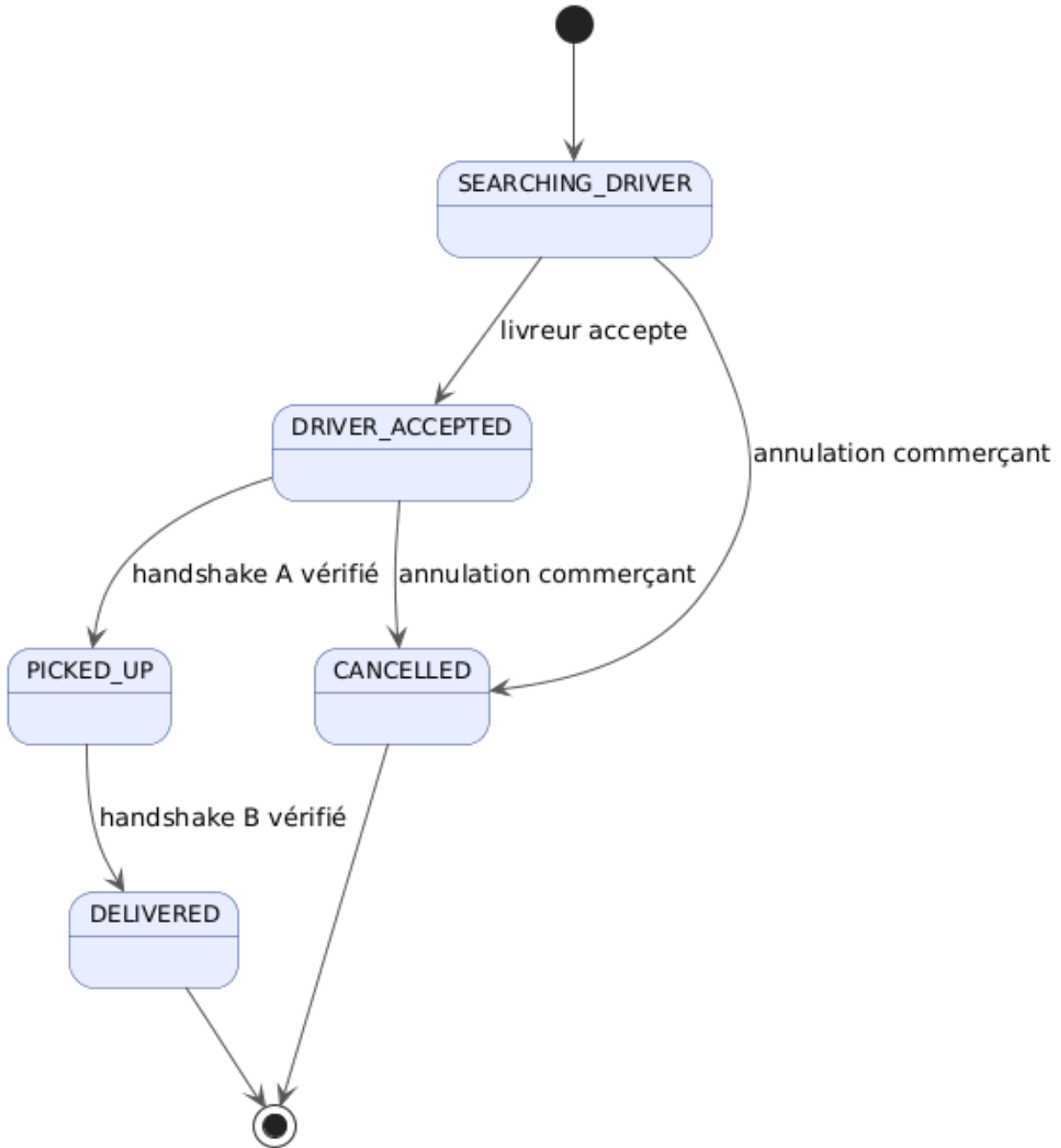


FIGURE 3.16 – Machine à états — statut de commande

11.2 Disponibilité du livreur

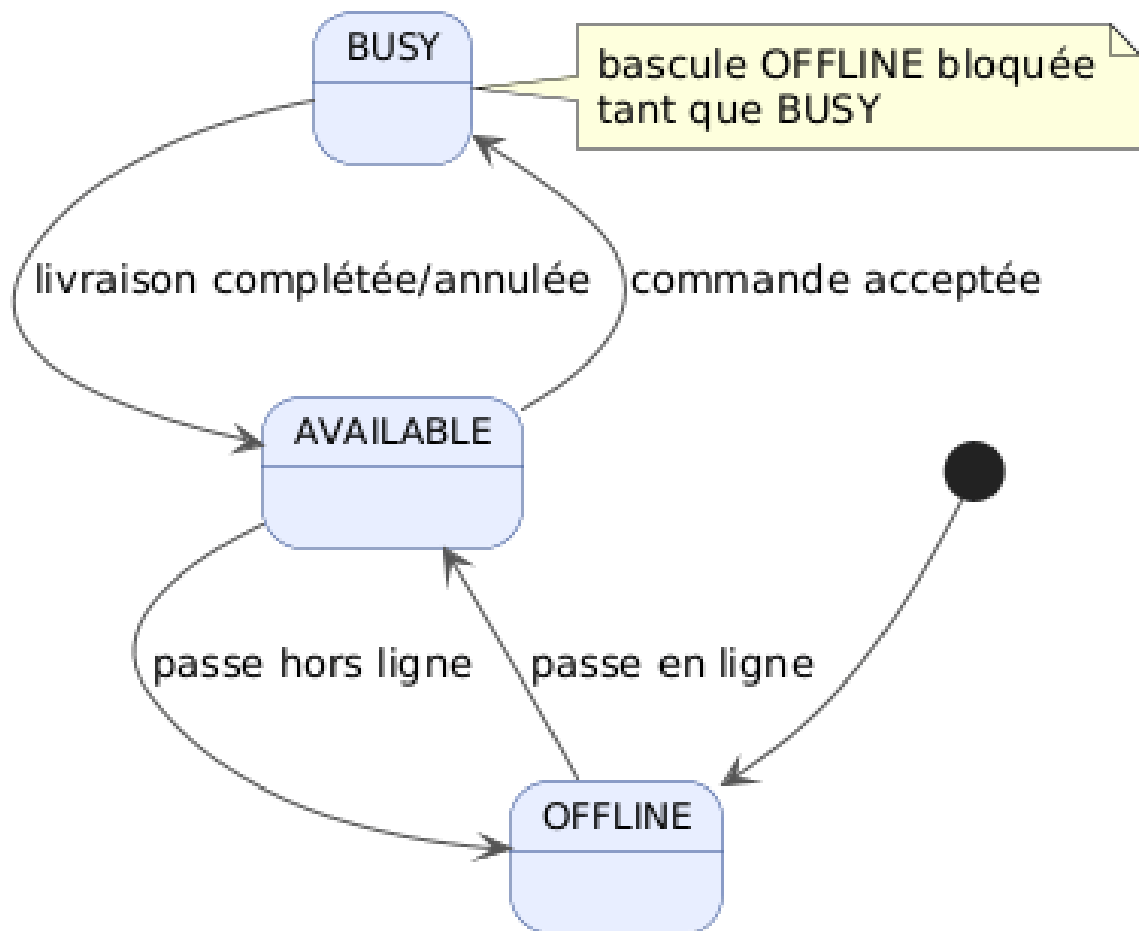


FIGURE 3.17 – Machine à états — disponibilité du livreur

11.3 Statut d'abonnement

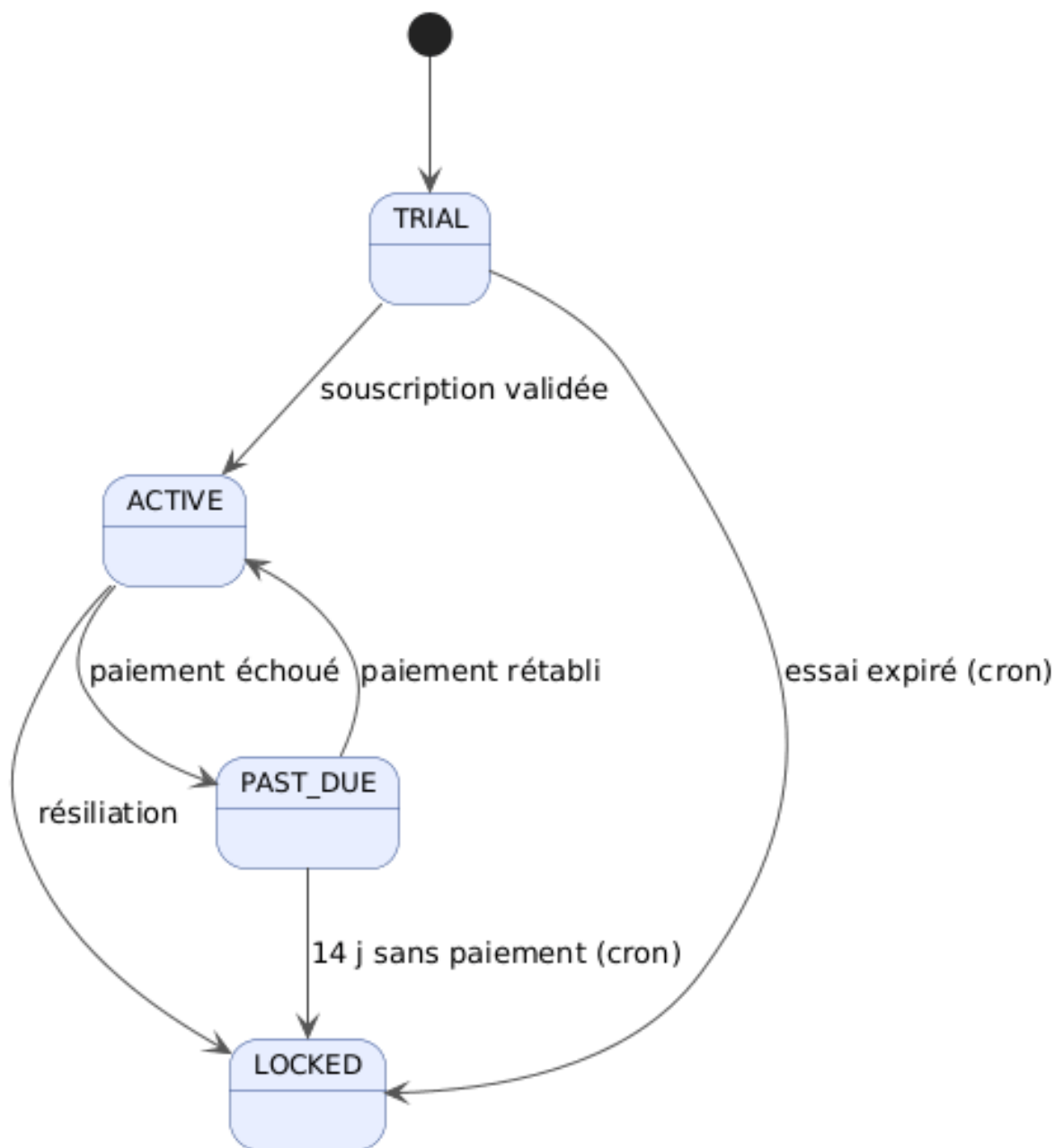


FIGURE 3.18 – Machine à états — statut d'abonnement marchand

12 Système de handshake

12.1 Principe

Le système de handshake garantit l'authenticité des transferts de colis. Deux codes à 6 chiffres sont générés à la création de chaque commande ; chacun doit être présenté au bon

moment par la bonne personne.

Type	Détenteur	Vérifié par	Effet
A (retrait)	Livreur	Commerçant (saisit le code)	Commande → PICKED_UP
B (livraison)	Client	Livreur (saisit le code)	Commande → DELIVERED

12.2 Flux de vérification et limitation des tentatives

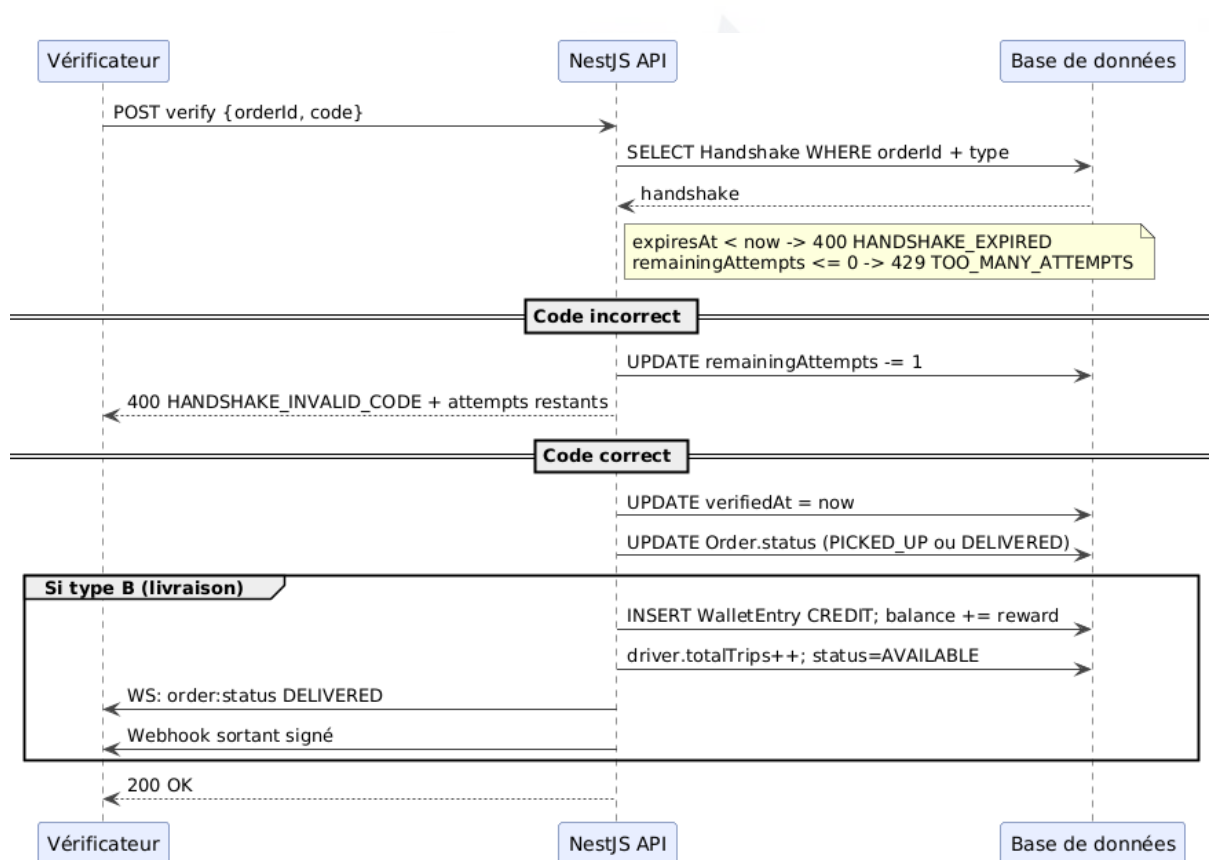


FIGURE 3.19 – Flux de vérification handshake avec gestion des tentatives

12.3 Verrouillage définitif

Lorsque `remainingAttempts` atteint 0, chaque appel ultérieur retourne 429 Too Many Requests, quel que soit le code soumis. Un endpoint d'administration permet de réinitialiser les tentatives (`POST /admin/orders/:orderId/reset`).

Avec 3 tentatives sur 10^6 combinaisons possibles, la probabilité de deviner aléatoirement le bon code est de $3/10^6 = 0,0003\%$.

12.4 Expiration et récupération du code

Les deux codes expirent 12 heures après la création de la commande. Un handshake expiré retourne 400 HANDSHAKE_EXPIRED indépendamment du code soumis ou du nombre de tentatives restantes.

Si le livreur ferme son application après acceptation, il récupère le code de retrait (type A) via :

```
GET /driver/me/missions/:missionId/handshake/pickup-code
```

L'endpoint vérifie que la commande est en DRIVER_ACCEPTED et que le handshake n'est pas expiré, puis retourne le code existant — il n'y a pas de régénération.

13 Module livreur

Base path : /driver/me Auth : JWT + rôle DRIVER + email vérifié

13.1 Mise à jour de position

```
PATCH /driver/me/location Corps : { "latitude": 48.85, "longitude": 2.35 }
```

Met à jour la géométrie PostGIS via SQL brut (Prisma ne supporte pas nativement ST_MakePoint) :

```
UPDATE "Driver"  
SET "lastKnownLocation" = ST_MakePoint(:lon, :lat)  
WHERE "userId" = :driverId
```

13.2 Basculement de disponibilité

PATCH /driver/me/availability bascule entre AVAILABLE et OFFLINE. Le basculement est **bloqué** si availabilityStatus = BUSY (commande en cours). Le passage en OFFLINE émet un événement WebSocket driver:status.

13.3 Missions disponibles

GET /driver/me/missions/available retourne les commandes en SEARCHING_DRIVER dont le magasin se trouve dans le rayon de livraison du livreur, en excluant les commandes déjà refusées :

```
SELECT o.* FROM "Order" o  
JOIN "Store" s ON s.id = o."storeId"  
WHERE o.status = 'SEARCHING_DRIVER'
```

```

AND ST_DWithin(s.location::geography,
    ST_MakePoint(:driverLon,:driverLat)::geography,
    :deliveryRadius * 1000)
AND o.id NOT IN (SELECT "orderId" FROM "DriverOrderRejection"
    WHERE "driverId" = :driverId)

```

13.4 Refus de mission

POST /driver/me/missions/:id/reject effectue un upsert dans DriverOrderRejection (idempotent). La commande reste en `SEARCHING_DRIVER` pour les autres livreurs. Les commandes refusées sont exclues des résultats futurs de /missions/available.

13.5 Portefeuille et retraits

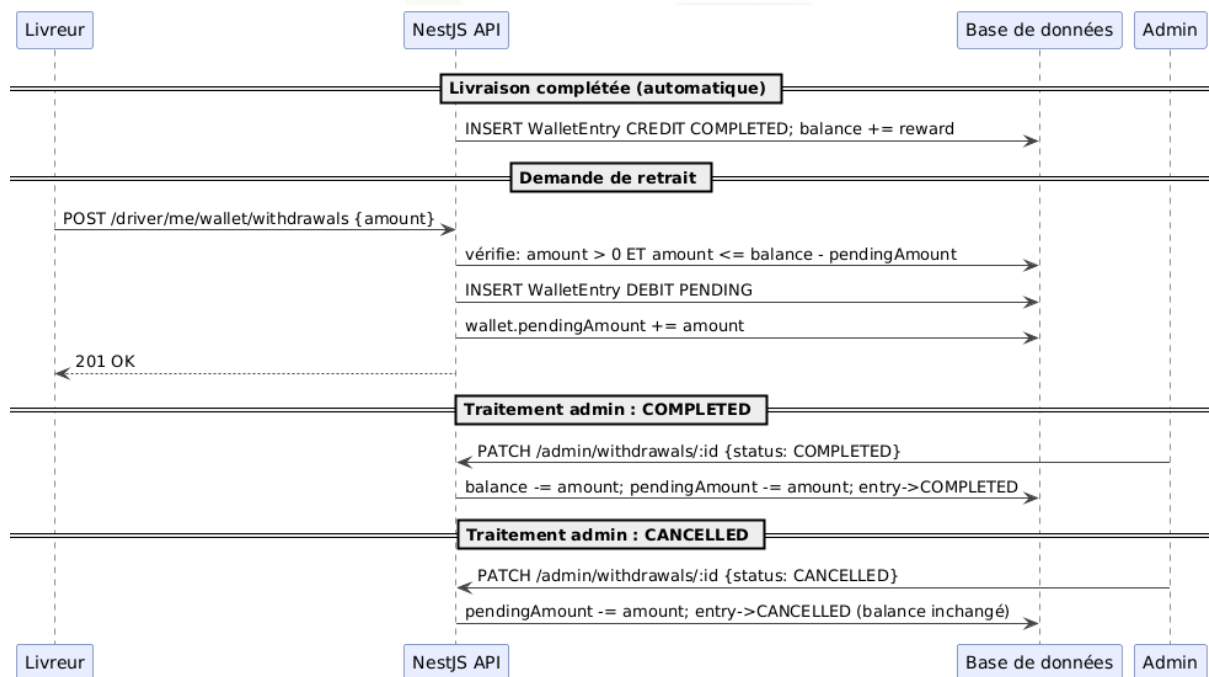


FIGURE 3.20 – Flux portefeuille livreur — crédit automatique et cycle de retrait

14 KYC et vérification des documents

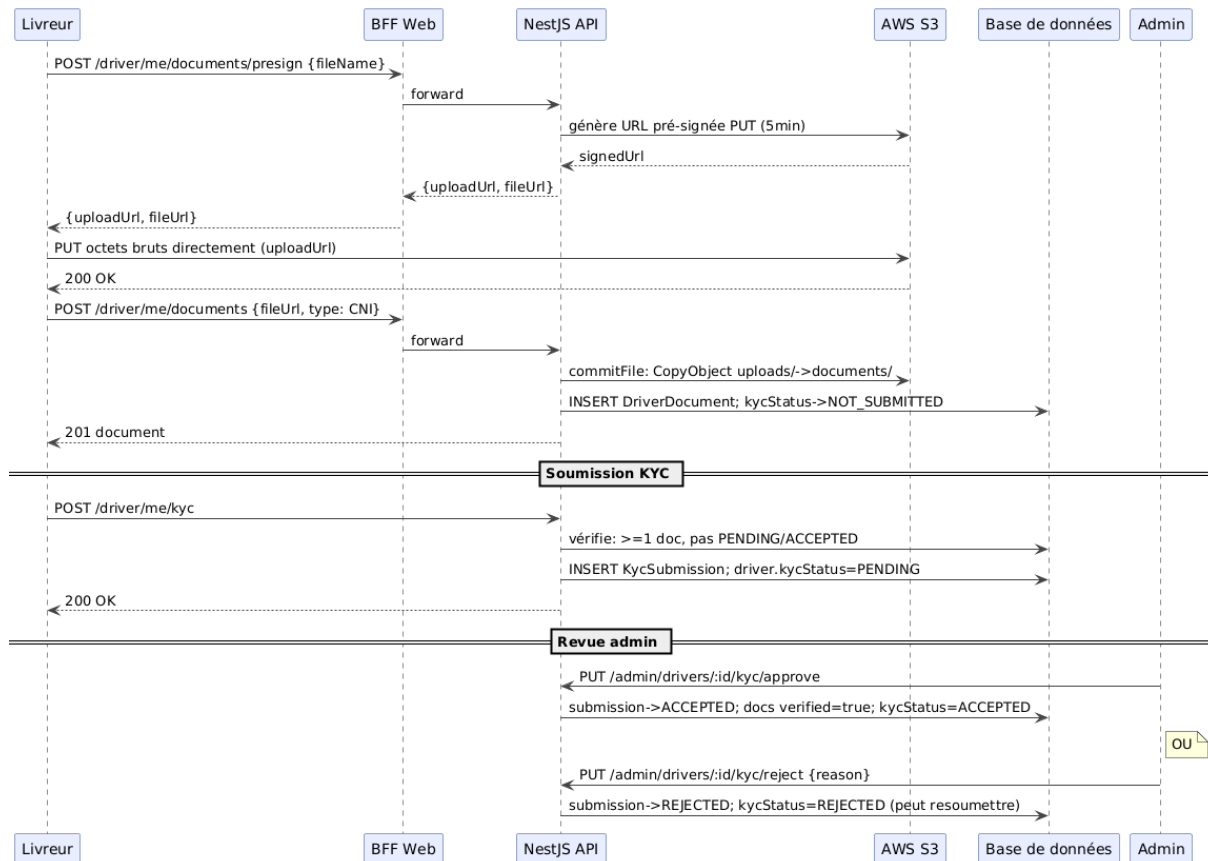


FIGURE 3.21 – Flux KYC complet — upload S3, soumission et revue admin

Règles métier :

- Seuls les livreurs avec `kycStatus = ACCEPTED` apparaissent dans les requêtes de proximité.
- La modification des champs d'adresse du profil remet `kycStatus` à `NOT_SUBMITTED`.
- L'upload d'un nouveau document remet `kycStatus` à `NOT_SUBMITTED` s'il était `ACCEPTED` ou `REJECTED`.
- Les documents vérifiés dans le cadre d'un KYC accepté ne peuvent pas être supprimés.

15 Module administrateur

Auth : JWT + rôle ADMIN sur toutes les routes.

Méthode	Chemin	Description
POST	/admin/accounts/admin	Créer un compte administrateur
POST	/admin/accounts/merchant	Créer un compte commerçant
POST	/admin/accounts/driver	Créer un compte livreur
GET	/admin/merchants	Tous les commerçants avec statut d'abonnement
GET	/admin/merchants/:merchantId	Profil complet, magasins et clés API
GET	/admin/drivers?status=&kycStatus=	Tous les livreurs (filtres optionnels)
GET	/admin/drivers/:driverId	Profil complet, documents, portefeuille
PUT	/admin/drivers/:driverId/kyc/approve	Approuver le KYC
PUT	/admin/drivers/:driverId/kyc/reject	Rejeter le KYC avec motif
GET	/admin/withdrawals?status=...	Filtrer les retraits
PATCH	/admin/withdrawals/:withdrawalId	Traiter un retrait (COMPLETED ou CANCELLED)
POST	/admin/orders/:orderId/reset	Réinitialiser les tentatives handshake

16 Abonnements et facturation (Stripe)

Plan	Magasins	Tarif
FREE	1	0 € (essai 30j, puis LOCKED)
PRO	5	14,90 €/mois
BUSINESS	Illimité	59,90 €/mois

Méthode	Chemin	Description
POST	/subscriptions/checkout?plan=PRO BUSINESS	Crée une session Stripe Checkout, retourne <code>checkoutUrl</code>
POST	/subscriptions/portal	Crée une session Stripe Customer Portal

Flux Stripe : création session → paiement sur page Stripe → web-hook `customer.subscription.updated` → `subscriptionStatus=ACTIVE` et `currentPeriodEnd` mis à jour. En cas d'échec de paiement : `invoice.payment_failed` → email d'avertissement. En cas d'annulation : `customer.subscription.deleted` → `subscriptionStatus=LOCKED`, magasins verrouillés.

17 Tarification et calcul d'itinéraire

17.1 Formule de calcul

$$\text{fraisLivraison} = \left(2,00 + d_{\text{km}} \times 0,50\right) \times M_{\text{taille}} + S_{\text{lourd}} \quad \text{rémunérationLivreur} = \text{fraisLivraison} \times 0,70$$

Taille colis	Poids	Multiplicateur	Majoration poids
SMALL	<1 kg	1,0×	—
MEDIUM	1–5 kg	1,2×	—
LARGE	5–15 kg	1,5×	—
EXTRA_LARGE	>15 kg	2,0×	+1,50 €

La taille est inférée automatiquement du poids si elle n'est pas fournie explicitement dans la requête.

18 Upload de fichiers (S3)

Voir §7 pour le flux complet depuis le BFF. Points clés :

- URL pré-signée PUT : validité 5 minutes, générée par le SDK AWS v3
- Les octets ne transitent jamais par le serveur API
- Staging uploads/ → permanent documents/ via copie côté serveur S3
- Suppression automatique des fichiers orphelins dans uploads/ après 2 jours (cycle de vie S3)

19 Événements temps réel (WebSocket)

Transport : Socket.io 4. **Auth** : token JWT courte durée (2 min) obtenu via GET /auth/ws-token, passé comme `socket.auth.token`.

Le token WS est distinct du token d'accès (15 min) pour limiter la fenêtre d'utilisation d'un token volé sur une connexion socket persistante.

Direction	Événement	Salle cible	Déclencheur
C → S	join-order	—	Rejoindre le suivi d'une commande
C → S	join-merchant	—	Rejoindre les events commerçant
C → S	join-driver	—	Rejoindre les events livreur
S → C	order:status	order:{id}	Tout changement de statut de commande
S → C	order:new	driver:{id}	Nouvelle commande à proximité
S → C	driver:status	merchant:{id}	Changement de disponibilité livreur

20 Notifications

20.1 Push (Expo)

Enregistrement du token : `PUT /driver/me/push-token { "token": "ExponentPushToken[...]" }`

Message envoyé à chaque nouvelle commande dans le rayon du livreur :

```
"Nouvelle livraison : Alice pres de 12 Rue de la Paix - 3,4 km - 4,73 EUR"
```

20.2 SMS (Twilio)

Déclencheur	Dest.	Message
Commande créée	Client	Votre commande a bien été reçue, nous vous informerons quand elle sera prise en charge.
Retrait vérifié	Client	Votre commande est en route. Montrez le code B-XXXXXX au livreur.
Commande livrée	Client	Votre commande a été livrée. Merci d'avoir utilisé GoMile!

21 Système d'emails

Fournisseur : Resend. **Rendu** : React Email (JSX compilé côté serveur en HTML).

Template	Déclencheur
verify-email.tsx	Inscription (tous rôles)
reset-password.tsx	Mot de passe oublié (OTP)
account-locked.tsx	Essai expiré ou abonnement résilié
payment-failed.tsx	Stripe invoice.payment_failed
subscription-renewing.tsx	Renouvellement dans 9–10 jours
trial-expiring.tsx	Fin d'essai dans 9–10 jours

22 Webhooks sortants

Quand un magasin a une `webhookUrl` configurée, un POST HTTP signé est envoyé à chaque événement de commande, permettant l'intégration avec WooCommerce, Shopify ou tout système custom.

Événements : `ORDER_ACCEPTED`, `ORDER_PICKED_UP`, `ORDER_DELIVERED`, `ORDER_CANCELLED`.

Signature : En-tête `X-GoMile-Signature: sha256={hmac}`. Le HMAC est calculé avec le `webhookSecret` propre à chaque magasin, ce qui permet au destinataire de vérifier l'authenticité du message.

```
{ "event": "ORDER_DELIVERED", "orderId": "uuid",
  "orderReference": "WC-1042", "timestamp": "2026-05-09T14:23:00Z" }
```

23 Tâches planifiées (cron jobs)

Module : `src/tasks/` via `@nestjs/schedule`.

Planification	Job	Logique
Quotidien 02h00	Supprimer comptes non vérifiés	DELETE users WHERE emailVerified=false AND createdAt < now-48h
Quotidien 02h00	Verrouiller essais expirés	Marchands TRIAL avec trialEndsAt < now : verrouiller magasins, LOCKED, email.
Quotidien 02h00	Révoquer clés API expirées	UPDATE clés où revokedAt IS NULL AND expiresAt < now : revokedAt=now.
Quotidien 02h00	Verrouiller PAST_DUE	Marchands PAST_DUE avec currentPeriodEnd < now-14j : verrouiller, LOCKED, email. Filet de sécurité si webhook Stripe manqué.
Quotidien 07h00	Avertir fins d'essai	Marchands TRIAL avec trialEndsAt dans 9-10 jours : email trial-expiring.
Quotidien 08h00	Avertir renouvellements	Marchands ACTIVE avec currentPeriodEnd dans 9-10 jours : email subscription-renewing.
Hebdo (dim. 03h)	Purger rejets anciens	DELETE DriverOrderRejection où commande n'est plus en SEARCHING_DRIVER.
Hebdo (dim. 03h)	Purger handshakes fermés	DELETE Handshake où commande est DELIVERED ou CANCELLED.

24 Gestion des erreurs

Toutes les erreurs sont produites par une factory typée (`src/common/api-error.ts`), garantissant un format de réponse uniforme sur toute l'API :

```
{ "statusCode": 429, "code": "HANDSHAKE_TOO_MANY_ATTEMPTS",
  "message": "Trop de tentatives incorrectes. Contacter le support.",
  "error": "Too Many Requests" }
```

Domaine	Codes principaux
AUTH_ERRORS	INVALID_CREDENTIALS, EMAIL_ALREADY_USED, OTP_EXPIRED, INVALID_RESET_TOKEN, AUTH_TOKEN_MISSING
ORDER_ERRORS	ORDER_NOT_FOUND, ORDER_BAD_STATUS, ORDER_ALREADY_CLAIMED, HANDSHAKE_EXPIRED, HANDSHAKE_INVALID_CODE, HANDSHAKE_TOO_MANY_ATTEMPTS
STORE_ERRORS	STORE_NOT_FOUND, NOT_OWNER, STORE_HAS_ACTIVE_ORDERS, STORE_LOCKED
SUBSCRIPTION_ERRORS	SUBSCRIPTION_NOT_ACTIVE, QUOTA_EXCEEDED
KYC_ERRORS	KYC_NO_DOCUMENTS, KYC_ALREADY_PENDING, KYC_ALREADY_ACCEPTED
DRIVER_ERROR	DRIVER_NOT_FOUND, DRIVER_BUSY, DRIVER_KYC_NOT_ACCEPTED
BACKEND_UNREACHABLE	Ajouté par la couche BFF quand l'API NestJS est inaccessible

25 Variables d'environnement

```
# Serveur
PORT=4000
API_BASE_URL=https://api.gomile.delivery
APP_URL=https://gomile.delivery

# Base de donnees
DATABASE_URL=postgres://gomile:gomile@localhost:5432/gomile

# JWT
JWT_ACCESS_SECRET=<64+ caracteres aleatoires>
JWT_REFRESH_SECRET=<64+ caracteres differents>

# Redis
REDIS_URL=redis://localhost:6379

# AWS S3
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_REGION=eu-west-3
S3_BUCKET_NAME=gomile-uploads

# Stripe
STRIPE_SECRET_KEY=sk_live...
STRIPE_WEBHOOK_SECRET=whsec...
```

```
# Twilio (SMS)
TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_PHONE_NUMBER=+33...

# Resend (Email)
RESEND_API_KEY=re...

# OpenRouteService
ORS_API_KEY=

# Auto-déploiement (webhook GitLab)
GITLAB_WEBHOOK_SECRET=
```

26 Couverture de tests

26.1 Stratégie générale

La stratégie de tests de l'API backend repose sur deux niveaux complémentaires :

- **Tests unitaires** (`test/unit/`) : chaque service et contrôleur est testé de manière isolée, toutes les dépendances étant remplacées par des *mocks* Jest. Cette approche garantit une exécution rapide et une localisation précise des régressions.
- **Tests end-to-end** (`test/*.e2e-spec.ts`) : l'intégralité du module NestJS est démarré via `TestingModule`, connecté à une vraie base de données PostgreSQL. Seuls les services externes réellement distants (notifications push Expo, SMS Twilio, géocodage OpenRouteService, envoi d'e-mails, stockage S3, Stripe) sont remplacés par des *mocks*, afin d'éviter toute dépendance réseau tout en conservant la logique métier réelle.

Ce choix de stratification répond à la nature de l'application : une API REST dont la valeur réside dans l'enchaînement correct des états (cycle de vie d'une commande, rotation des jetons JWT, validation KYC, gestion du portefeuille). Les tests unitaires vérifient les comportements isolés, les tests e2e valident les flux complets de bout en bout.

26.2 Périmètre des tests end-to-end

Sept suites e2e couvrent les fonctionnalités principales :

auth.e2e-spec.ts Enregistrement marchand et livreur, rotation du *refresh token* (l'ancien doit être rejeté après rafraîchissement), connexion par e-mail ou numéro de

téléphone, rejet des doublons d'e-mail, rejet de mot de passe invalide, invalidation après déconnexion.

store.e2e-spec.ts CRUD complet de boutique : création, listage avec filtre `isActive`, récupération, mise à jour, désactivation/réactivation, suppression. Vérification de l'isolation entre marchands (tentative de création pour un autre marchand → 403).

order.e2e-spec.ts Cycle de vie complet d'une livraison : création → acceptation → poignée de main A (marchand vérifie le code de retrait) → poignée de main B (livreur vérifie le code de livraison client) → crédit du portefeuille. Cas d'erreur couverts : annulation avant enlèvement, impossibilité d'annuler après enlèvement (409), concurrence entre deux livreurs (409), code de poignée de main incorrect (401 et décrémentation des tentatives), épuisement des tentatives (429), listage des commandes, garde d'authentification.

driver-me.e2e-spec.ts Tableau de bord livreur, mise à jour GPS, bascule de disponibilité (OFFLINE AVAILABLE, 409 si BUSY), profil, mise à jour de profil (réinitialisation du statut KYC si l'adresse change), véhicule de session, portefeuille (solde, entrées, retrait insuffisant, retrait avec débit), documents (création, suppression, isolation entre livreurs), soumission KYC (prérequis document, doublon 409).

kyc.e2e-spec.ts Lecture du statut KYC par le livreur, approbation et rejet par un administrateur, rejet d'un marchand tentant d'approuver (403).

admin.e2e-spec.ts Contrôle d'accès (401/403 pour marchands et livreurs), création de comptes admin/marchand/livreur par un administrateur, supervision des marchands et livreurs (listage, filtrage par statut KYC, profil complet), approbation/rejet KYC, gestion des retraits (liste, approbation, annulation avec remboursement, doublon 409), réinitialisation des tentatives de poignée de main.

delivery-pricing.e2e-spec.ts Authentification par clé API : estimation de livraison avec clé valide, rejet de clé invalide ou manquante (401), rejet de clé révoquée (403), révocation à la volée.

26.3 Périmètre des tests unitaires

Les tests unitaires (`test/unit/`) couvrent les services et contrôleurs suivants : `AuthService`, `ApiKeyService`, `AdminService`, `DeliveryPricingService`, `OpenRouteService`, `DriverMeService`, `KycService`, `MerchantService`, `NotificationService`, `OrderService` (marchands et livreurs), `SmsService`, `StoreService`, `SubscriptionService`, `TasksService`, `OutboundWebhookService`,

EventsGateway, ainsi que les contrôleurs associés. Cette couverture garantit le comportement des couches service indépendamment de l'infrastructure.

26.4 Résultats de couverture

Le tableau 3.1 présente la couverture mesurée par Jest (`jest -coverage`) après exécution des tests unitaires.

TABLE 3.1 – Couverture de code par module (tests unitaires)

Module	Instructions	Fonctions	Branches
admin	92 %	86 %	76 %
auth	86 %	78 %	74 %
delivery	100 %	100 %	90 %
driver	61 %	64 %	39 %
events	100 %	100 %	77 %
kyc	97 %	100 %	91 %
merchant	100 %	100 %	92 %
notification	100 %	100 %	85 %
order	76 %	76 %	59 %
sms	94 %	100 %	83 %
store	95 %	100 %	78 %
subscription	82 %	93 %	54 %
tasks	100 %	100 %	75 %
upload	42 %	28 %	42 %
webhook	100 %	100 %	82 %
Global	83 %	79 %	67 %

26.5 Analyse et justification

Modules à couverture maximale. Les modules `delivery`, `events`, `merchant`, `notification`, `tasks` et `webhook` atteignent 100 % d'instructions et de fonctions. Ces modules exposent soit des contrats d'interface critiques (`webhook`, `events`), soit une logique métier dense qui justifie une couverture exhaustive (`delivery`, `merchant`).

Module `driver` (61 % instructions, 39 % branches). Le module livreur présente la couverture unitaire la plus faible. Il agrège une logique complexe de machine à états (disponibilité, missions, poignées de main) dont de nombreuses branches conditionnelles sont exercées uniquement par les tests e2e (`driver-me.e2e-spec.ts` et `order.e2e-spec.ts`). La couverture globale réelle de ce domaine est donc supérieure à celle indiquée dans le tableau des tests unitaires.

Module upload (42 % instructions, 28 % fonctions). Ce module encapsule des appels directs au SDK AWS S3. En test unitaire, il est systématiquement remplacé par un *mock* dans toutes les autres suites, ce qui explique son faible taux de couverture directe. Les chemins d'erreur propres au SDK (URL expirée, permission refusée) ne sont pas couverts car ils nécessiteraient des identifiants AWS actifs.

Module order (76 % instructions, 59 % branches). Le service de commande implémente une machine à états à plusieurs transitions ; certaines branches d'erreur (transitions invalides, cas de concurrence extrêmes) ne sont couvertes que partiellement en unitaire. Les flux principaux (création, acceptation, livraison, annulation, concurrence entre livreurs) sont intégralement exercés par les tests e2e.

Module subscription (82 % instructions, 54 % branches). Les branches non couvertes correspondent aux gestionnaires de *webhooks* Stripe (événements de paiement échoué, renouvellement d'abonnement). Ces chemins nécessitent une infrastructure de simulation Stripe difficilement reproductible en environnement d'intégration continue.

Choix des mocks dans les tests e2e. Dans les suites e2e impliquant des flux de commande ou de livreur, les services `OpenRouteService`, `NotificationService`, `SmsService`, `EmailService`, `UploadService` et `SubscriptionService` sont systématiquement remplacés par des *mocks* Jest. Ce choix est délibéré : ces services sont des points d'intégration externe (réseau, facturation) dont le comportement est vérifié séparément en unitaire. Les remplacer permet d'exécuter les tests e2e de manière déterministe, sans dépendance réseau, dans un environnement d'intégration continue.

Chapitre 4

Application Web

1 Implémentation de la partie frontend administrateur

Cette section décrit les choix techniques retenus pour la réalisation de l'interface d'administration de GoMile : organisation des fichiers, composants réutilisables, couche de services et typage des données.

1.1 Structure générale du code

Le code de la partie admin est réparti dans trois emplacements principaux :

- `src/app/admin` : pages de l'espace administrateur, organisées selon le système de routage de Next.js;
- `src/components/admin/dashboard` : composants réutilisables propres au dashboard admin;
- `src/lib/api-admin.ts` : couche centralisée de consommation des services backend.

```
src/  
|-- app/  
|   |-- admin/  
|   |   |-- login/  
|   |   |   |-- page.tsx  
|   |   |   |-- use-login.ts  
|   |   |   |-- styles.ts  
|   |   |  
|   |   |-- dashboard/  
|   |       |-- page.tsx
```

```

|   |   |   |-- admin.ts
|   |   |   |-- admin-dashboard-menu.ts
|   |   |   |-- drivers/
|   |   |   |-- merchants/
|   |   |   |-- withdrawals/
|   |   |   |-- conflicts/
|   |   |   |-- admins/
|
|-- components/
|   |-- admin/
|       |-- dashboard/
|           |-- admin-dashboard-shell.tsx
|           |-- admin-entity-table.tsx
|           |-- TableCard.tsx
|           |-- ChartContainer.tsx
|           |-- WeekleyBarChart.tsx
|           |-- DateSelector.tsx
|           |-- DateFunctions.ts
|
|-- lib/
    |-- api-admin.ts

```

1.2 Routage et organisation des pages

Le dossier `src/app/admin` suit le système de routage fichier de Next.js. Chaque sous-dossier correspond à une route exposée dans l'application.

Route	Rôle
<code>/admin/login</code>	Page de connexion administrateur
<code>/admin/dashboard</code>	Vue d'ensemble du tableau de bord
<code>/admin/dashboard/drivers</code>	Liste et gestion des livreurs
<code>/admin/dashboard/drivers/[driverId]</code>	Fiche détaillée d'un livreur
<code>/admin/dashboard/merchants</code>	Liste et gestion des commerçants
<code>/admin/dashboard/withdrawals</code>	Suivi des demandes de retrait
<code>/admin/dashboard/conflicts</code>	Renvoi du handshake de commande
<code>/admin/dashboard/admins</code>	Création de nouveaux comptes

TABLE 4.1 – Routes de l'espace administrateur

1.3 Composants réutilisables

AdminDashboardShell

Ce composant définit la structure commune à la quasi-totalité des pages admin : barre de navigation, menu latéral, zone de contenu principale et footer. Il reçoit la propriété `activeMenuLabel` pour mettre en évidence la section active dans le menu.

```
<AdminDashboardShell activeMenuLabel="Livreurs">
  {/* contenu de la page */}
</AdminDashboardShell>
```

AdminEntityTable

Composant générique typé avec un paramètre de type `T`. Il prend en charge les éléments communs à tous les tableaux de l'espace admin :

- titre et compteur de lignes ;
- barre de recherche et filtres ;
- états de chargement et d'erreur ;
- message de liste vide ;
- pagination via `DynamicTable`.

AdminDataTableCard

Défini dans `TableCard.tsx`, ce composant sert de conteneur aux tableaux. Il normalise l'affichage du titre, des contrôles, du compteur et des messages d'état pour l'ensemble des listes du dashboard.

Composants de graphiques et de dates

Les fichiers `DateSelector.tsx`, `DateFunctions.ts` et `WeekleyBarChart.tsx` gèrent les graphiques dépendant d'une plage hebdomadaire. `DateFunctions.ts` expose notamment des fonctions pour :

- formater une date pour un champ HTML de type `date` ;
- récupérer les jours de la semaine courante ;
- afficher le libellé d'une plage hebdomadaire.

`AdminWeeklyBarChart` exploite ces fonctions pour construire les données d'un graphique en barres à partir de la date sélectionnée par l'administrateur.

1.4 Composants métier spécialisés

En complément des composants génériques, chaque domaine fonctionnel dispose de ses propres composants.

Livreurs.

- `DriversTable` : tableau avec recherche, filtres et lien vers la fiche détaillée ;
- `VehiclesChart`, `KycChart`, `DeliveriesChart` : graphiques de répartition ;
- `DriverProfileSummary`, `DriverStatsGrid` : résumé et statistiques du profil ;
- `DriverKycDocuments` : affichage et traitement des documents KYC ;
- `DriverKycHistory` : historique des soumissions KYC.

Commerçants.

- `MerchantsTable` : tableau des commerçants ;
- `SubscriptionChart`, `StoresCountChart`, `StoresAddedChart` : graphiques d'activité ;
- `MerchantStoresMap` : carte géographique des boutiques.

Retraits. `WithdrawalsTable` gère le chargement, le filtrage (`useMemo`) et l'affichage des demandes, ainsi que les actions d'acceptation et de refus avec mise à jour locale de l'état React.

1.5 Couche de services frontend : `api-admin.ts`

Le fichier `src/lib/api-admin.ts` centralise toutes les fonctions frontend utilisées pour consommer les services backend. Les pages et composants n'accèdent jamais directement aux routes API sous forme de chaînes dispersées : ils appellent des fonctions métier explicites.

Fonction	Rôle
<code>getDriversList()</code>	Récupère la liste des livreurs
<code>getDriver(driverId)</code>	Récupère le détail d'un livreur
<code>getDriverOrdersList(driverId)</code>	Récupère les commandes d'un livreur
<code>approveDriverKyc(driverId)</code>	Valide le KYC d'un livreur
<code>rejectDriverKyc(driverId, input)</code>	Refuse le KYC avec motif
<code>getMerchantsList()</code>	Récupère la liste des commerçants
<code>getMerchant(merchantId)</code>	Récupère le détail d'un commerçant
<code>getWithdrawals()</code>	Récupère les demandes de retrait
<code>updateWithdrawal(id, input)</code>	Met à jour le statut d'un retrait
<code>createAdminAccount(input)</code>	Crée un compte administrateur
<code>createMerchantAccount(input)</code>	Crée un compte commerçant
<code>createDriverAccount(input)</code>	Crée un compte livreur
<code>resetHandshake(orderId)</code>	Relance le handshake d'une commande

TABLE 4.2 – Fonctions exposées par `api-admin.ts`

Fichier façade `admin.ts`

Le fichier `src/app/admin/dashboard/admin.ts` réexporte les fonctions et types de `api-admin.ts` sous un chemin local plus court, utilisable directement par les pages du dashboard. Il expose également la fonction `getAllDriverOrders`, qui agrège les commandes de tous les livreurs pour alimenter certains graphiques.

1.6 Typage TypeScript

Plusieurs types ont été définis dans `api-admin.ts` pour sécuriser la manipulation des données et rendre le code plus lisible.

Type	Rôle
<code>DriverStatus</code>	État d'activité d'un livreur
<code>KycStatus</code>	État de vérification KYC
<code>WithdrawalStatus</code>	État d'une demande de retrait
<code>AdminDriverListItem</code>	Livreur dans une liste
<code>AdminDriverDetail</code>	Données détaillées d'un livreur
<code>AdminMerchantListItem</code>	Commerçant dans une liste
<code>AdminMerchantDetail</code>	Données détaillées d'un commerçant
<code>AdminWithdrawalItem</code>	Demande de retrait
<code>CreateAdminAccountInput</code>	Données de création d'un compte admin
<code>UpdateWithdrawalInput</code>	Données de mise à jour d'un retrait

TABLE 4.3 – Types TypeScript définis dans `api-admin.ts`

Des types supplémentaires sont définis dans les fichiers utilitaires propres à chaque domaine. `drivers/AdminFunctions.ts` expose par exemple `DriverWithOptionalListFields`, `StatusOption`, `KycOption` et `VehicleOption` pour gérer les filtres et les données optionnelles du tableau des livreurs. `merchants/AdminFunctions.ts` définit de même `MerchantWithOptionalListFields`, `MerchantStore`, `MerchantStatusOption` et `MerchantSubscriptionOption`.

1.7 Fonctions utilitaires

Pour alléger les composants React, la logique de transformation et de filtrage est isolée dans des fichiers `AdminFunctions.ts` propres à chaque domaine.

Pour les livreurs, ces fonctions permettent de :

- construire le nom complet et récupérer les coordonnées ;
- formater la date d'inscription ;
- traduire les statuts techniques en libellés lisibles ;
- filtrer les livreurs selon la recherche, le statut, l'état KYC ou le véhicule ;
- calculer le nombre de livraisons par jour ou par mois ;
- préparer les données des graphiques.

Pour les commerçants, les fonctions utilitaires permettent de :

- récupérer l'identifiant, le nom, l'e-mail et le téléphone ;
- calculer le nombre de boutiques et de commandes ;

- déterminer le statut et traduire les abonnements ;
- filtrer les commerçants et préparer les données des graphiques.

1.8 Authentification et gestion de session

La logique de connexion est encapsulée dans le hook `useLogin`, défini dans `src/app/admin/login/use-login.ts`. Ce hook centralise :

- la gestion des champs du formulaire ;
- la validation frontend (champs non vides) ;
- l'appel à la fonction `login` du service d'authentification ;
- la vérification du rôle dans la session retournée ;
- la déconnexion immédiate via `logout` si le rôle n'est pas `ADMIN` ;
- la redirection vers `/admin/dashboard` après succès.

Ce contrôle frontend ne remplace pas la sécurisation des endpoints backend. Les routes API doivent vérifier elles-mêmes la session, le rôle et les droits de l'utilisateur avant toute réponse ou exécution d'action.

1.9 Gestion des états d'interface

Les pages et composants de l'espace admin gèrent systématiquement trois états d'affichage :

1. **Chargement** : indicateur visuel pendant la récupération des données.
2. **Erreur** : message transmis au composant de tableau ou affiché via `ErrorMessage` si un appel échoue.
3. **Succès** : message affiché via `SuccessMessage` après une action réussie.

Pour les actions susceptibles d'être déclenchées plusieurs fois (mise à jour d'un retrait, renvoi du handshake), un état booléen ou un identifiant en cours de traitement (`updatingWithdrawalId`, `isSubmitting`) désactive temporairement les boutons concernés afin d'éviter les soumissions simultanées.

1.10 Diagramme de cas d'utilisation

Le diagramme suivant présente l'ensemble des fonctionnalités accessibles à un administrateur depuis l'interface d'administration. Les relations *extend* indiquent les cas d'utilisation qui étendent un cas principal.

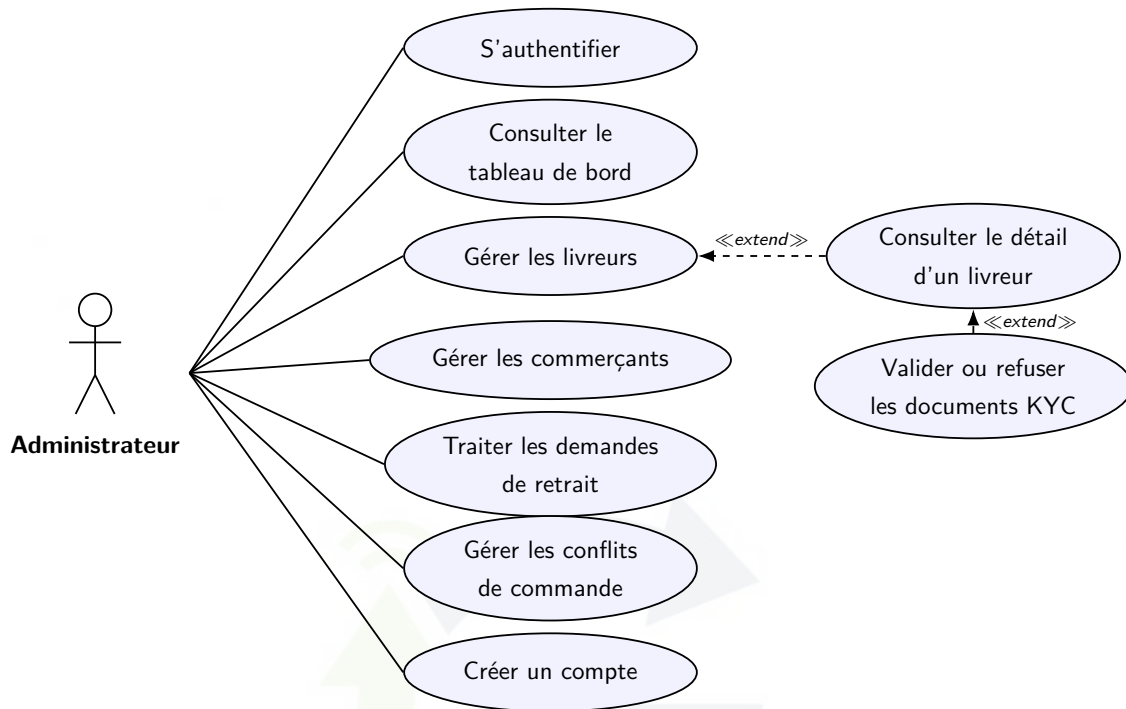


FIGURE 4.1 – Diagramme de cas d'utilisation de l'administrateur

1.11 Diagrammes de séquence système

Les diagrammes de séquence ci-dessous décrivent, pour chaque fonctionnalité principale, le déroulement des interactions entre l'administrateur, les composants frontend et les services backend. Le backend y est représenté comme un service externe consommé par le frontend, sans détail de son implémentation interne.

Authentification administrateur

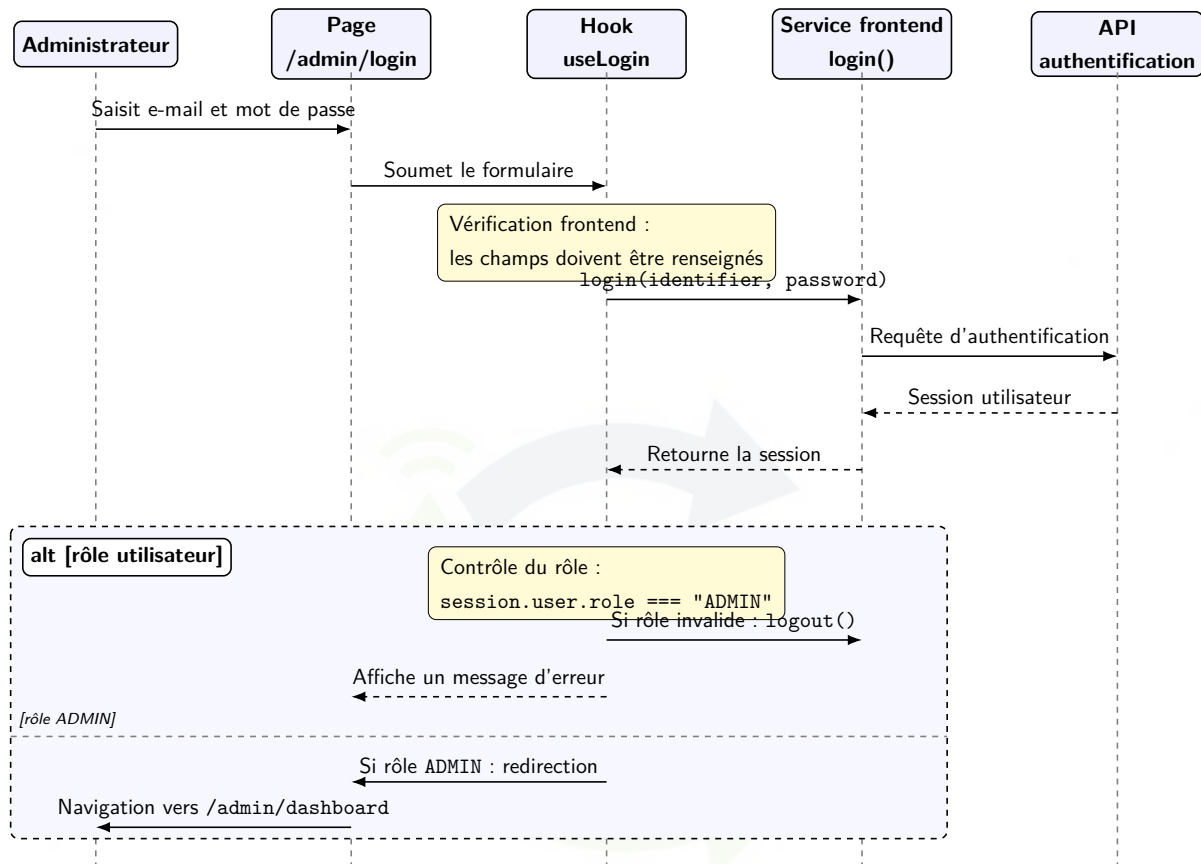


FIGURE 4.2 – Diagramme de séquence système — Authentification administrateur

L'administrateur saisit ses identifiants dans le formulaire. Le hook `useLogin` vérifie que les champs sont renseignés, puis appelle la fonction `login`. Une fois la session reçue, le frontend contrôle le rôle de l'utilisateur. Si le rôle est `ADMIN`, l'administrateur est redirigé vers le tableau de bord ; sinon, la session est annulée et un message d'erreur est affiché.

Gestion des documents KYC d'un livreur

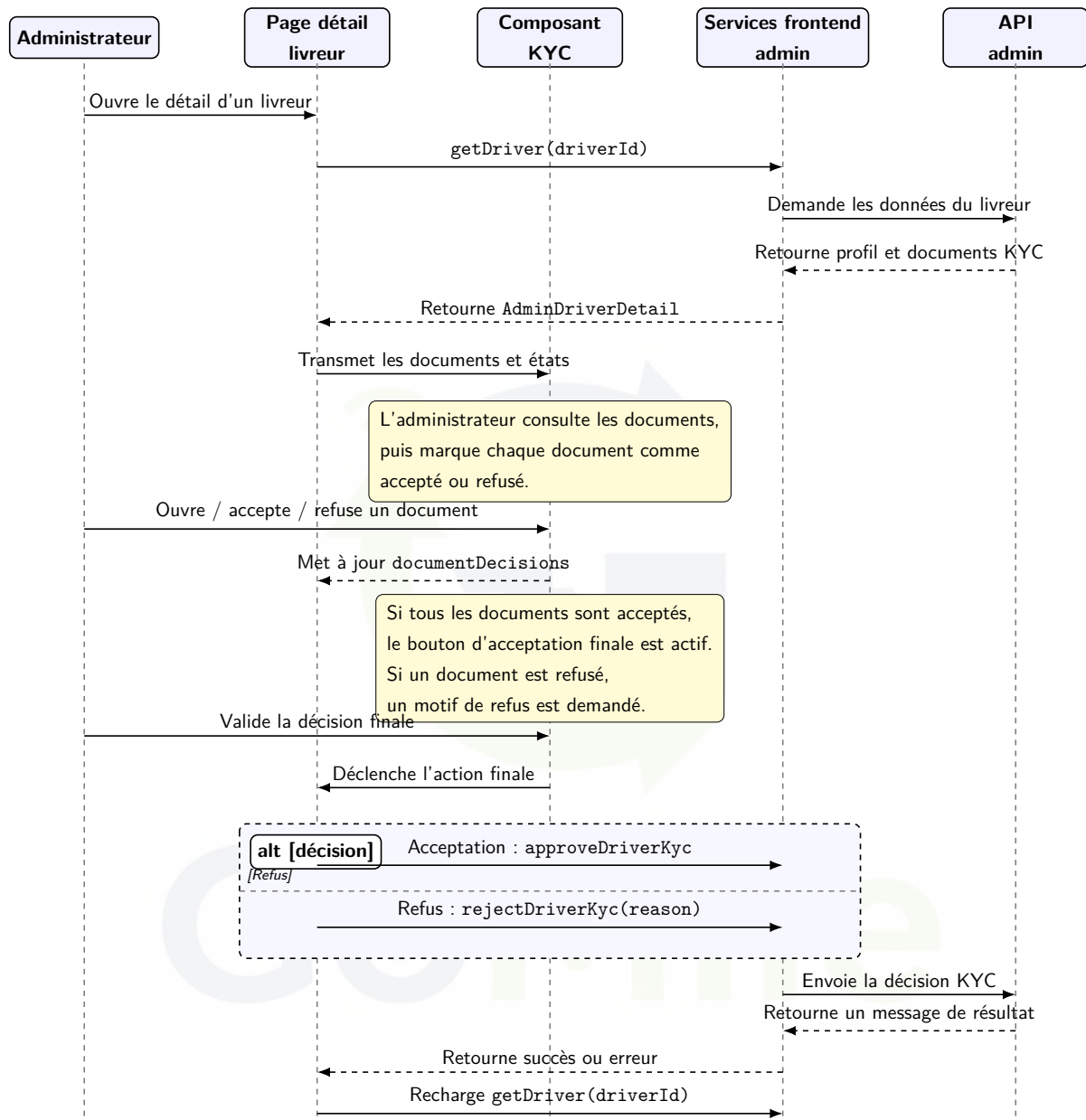


FIGURE 4.3 – Diagramme de séquence système — Gestion KYC d'un livreur

L'administrateur ouvre la fiche d'un livreur. Le frontend récupère les données du livreur et affiche les documents KYC via le composant dédié. Les décisions intermédiaires par document sont conservées localement dans `documentDecisions`. La décision finale est envoyée au service admin sous forme d'acceptation ou de refus motivé. Après la réponse, le frontend recharge les données du livreur et affiche une notification visuelle.

Création d'un nouveau compte

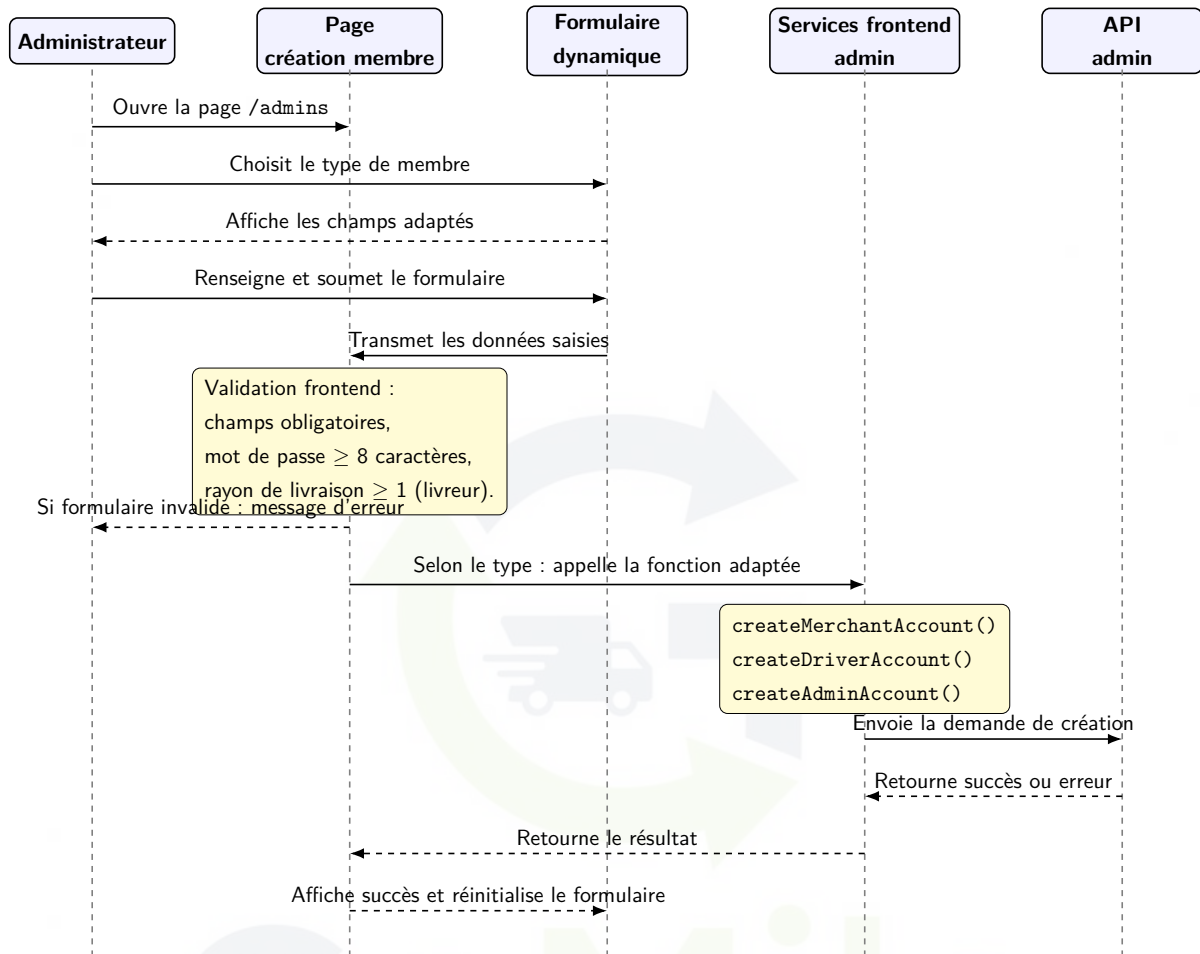


FIGURE 4.4 – Diagramme de séquence système — Création d'un compte

L'administrateur sélectionne le type de compte à créer, ce qui adapte dynamiquement les champs du formulaire. Après saisie, le frontend valide les données localement avant d'appeler la fonction de création correspondante. Le résultat est affiché sous forme de message de succès ou d'erreur, et le formulaire est réinitialisé en cas de succès.

Traitement des demandes de retrait

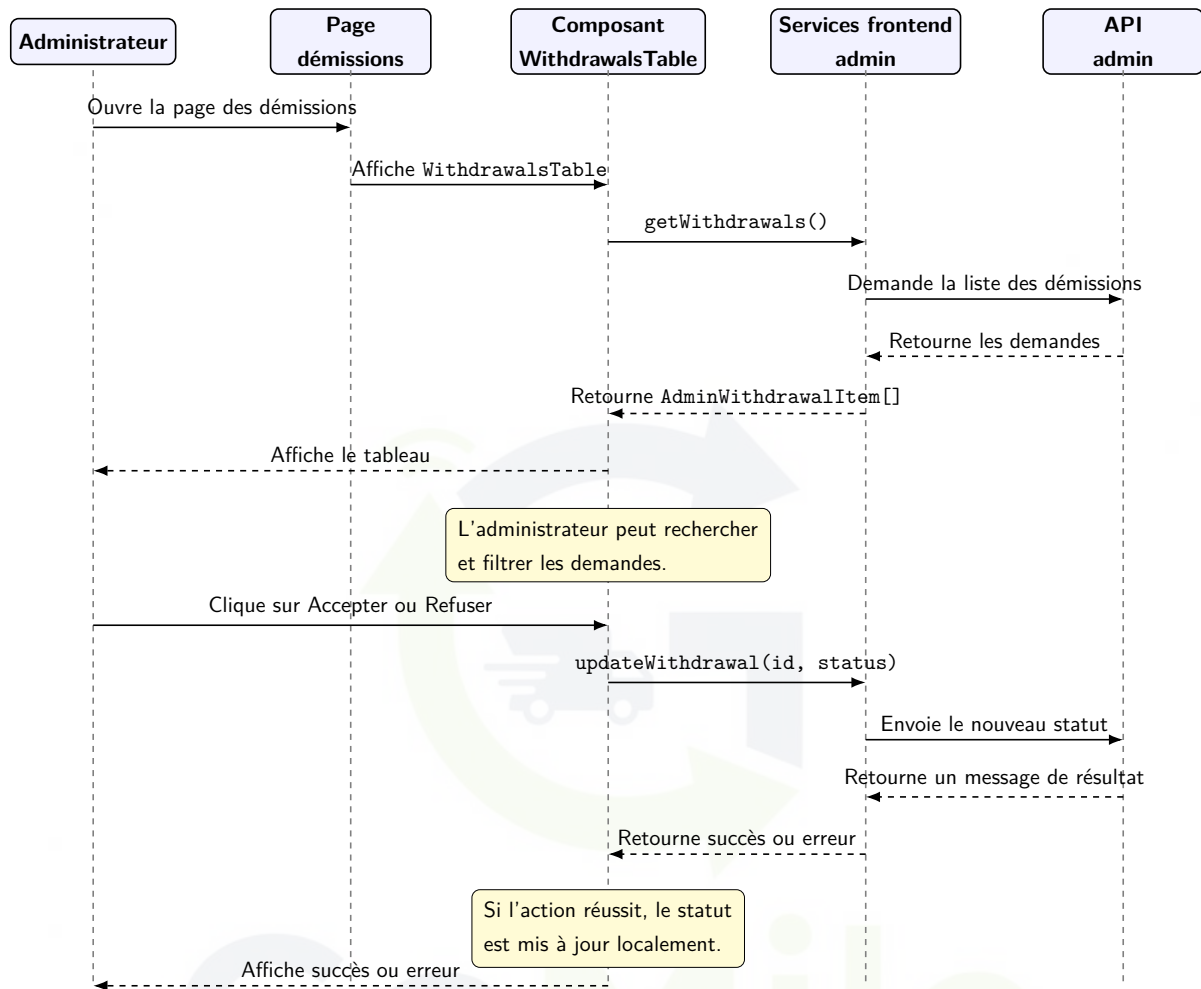


FIGURE 4.5 – Diagramme de séquence système — Gestion des demandes de retrait

À l'ouverture de la page, `WithdrawalsTable` récupère les demandes via le service frontend. L'administrateur peut filtrer les demandes par statut et traiter celles qui sont en attente. Lorsqu'il accepte ou refuse une demande, le frontend envoie le nouveau statut, met à jour l'affichage localement via l'état React et affiche un message indiquant le résultat de l'action.

Renvoi du handshake d'une commande

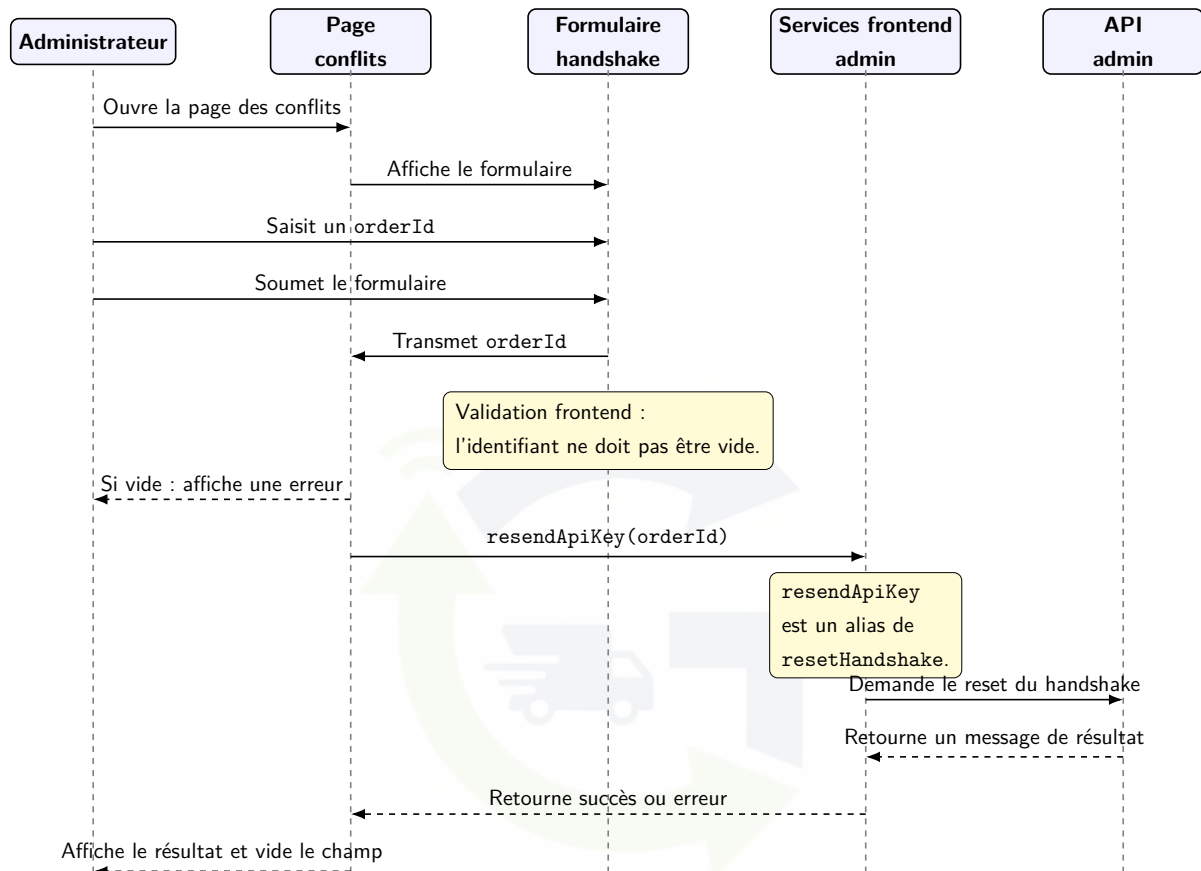


FIGURE 4.6 – Diagramme de séquence système — Renvoi du handshake

L'administrateur saisit l'identifiant de la commande concernée puis soumet le formulaire. Le frontend vérifie que le champ n'est pas vide avant d'appeler `resendApiKey`, alias de `resetHandshake`. Le résultat de l'action est affiché directement dans la page et le champ est vidé en cas de succès.

2 Partie commercial

Le front web de GoMile est développé avec Next.js. Il regroupe les portails : l'espace administrateur, l'espace commerçant et le parcours d'inscription livreur. Les pages sont organisées dans `apps/web/src/app`, tandis que les échanges avec le backend passent par une passerelle web sécurisée côté Next.js. Cette passerelle centralise la gestion des cookies, des sessions, des appels serveur et des erreurs.

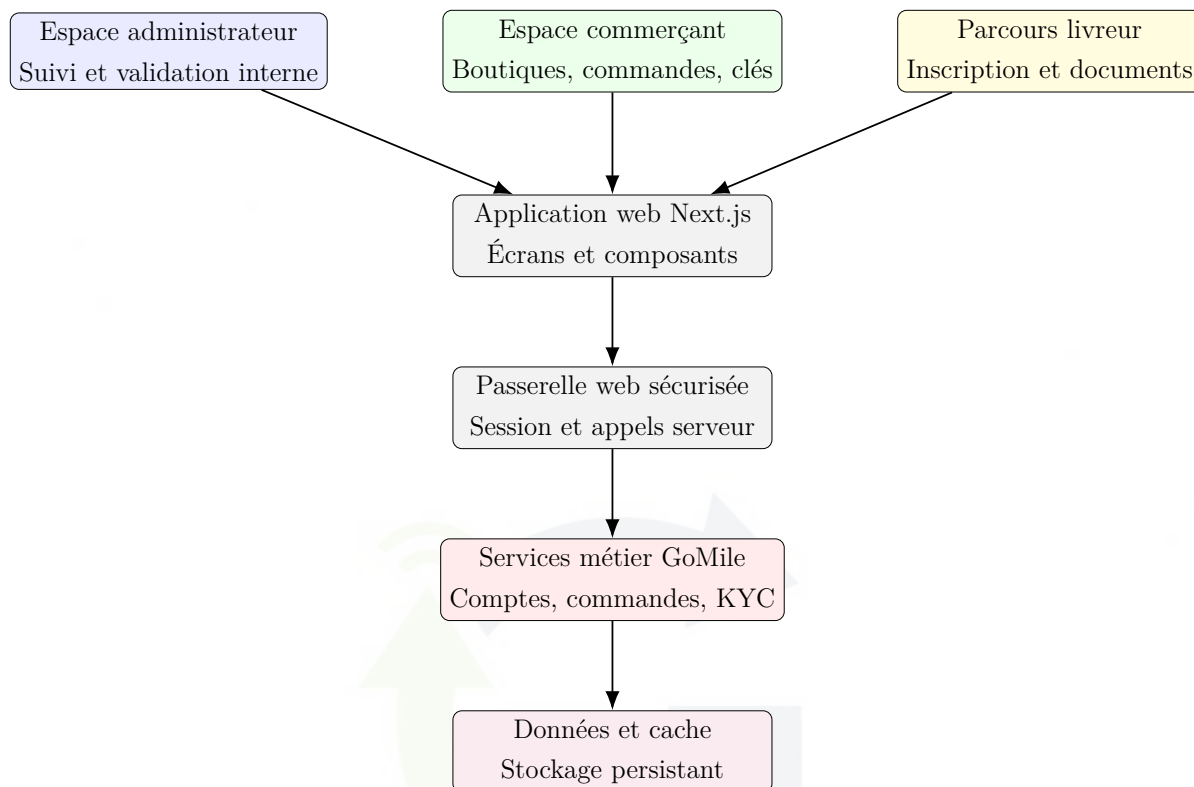


FIGURE 4.7 – Architecture générale du front web GoMile

2.1 Objectif et périmètre

Cette section décrit la partie commerciale du front web. Elle couvre l'espace commerçant, utilisé par le marchand, et le portail d'inscription livreur. L'espace commerçant permet de gérer les magasins, les commandes, les clés API et les informations du compte. Le portail livreur sert à créer un dossier d'inscription et à transmettre les documents nécessaires à la vérification KYC.

2.2 Diagramme de cas d'utilisation

Le diagramme suivant présente les principaux usages de la partie commerciale. Le commerçant peut créer son compte, vérifier son adresse e-mail, se connecter, consulter son tableau de bord, gérer ses boutiques, créer des commandes et administrer ses clés API. Le livreur candidat utilise un parcours séparé pour créer son dossier et envoyer ses documents KYC.

2.4 Parcours d'inscription marchand et vérification e-mail

Point d'entrée et rôle du flux

Le commerçant commence depuis l'écran d'inscription de l'espace commercial. Ce parcours sert à créer un marchand dans le système GoMile, mais aussi à vérifier que l'adresse e-mail saisie appartient réellement à l'utilisateur. Le compte existe après l'inscription, mais il reste incomplet tant que le lien de vérification n'a pas été ouvert.

Données saisies et contrôles

- **Identité du marchand** : nom, e-mail et mot de passe.
- **Validation front** : contrôle des champs obligatoires avant l'envoi.
- **Validation métier** : vérification de l'e-mail, création du marchand et préparation du message de confirmation.

Logique technique

L'écran d'inscription transmet la demande à la passerelle web sécurisée. Cette passerelle relaie ensuite l'appel au service d'authentification. Le service crée le marchand, puis demande au service e-mail externe d'envoyer un lien de vérification. Lorsque le commerçant clique sur ce lien, le jeton de vérification est transmis au backend et l'adresse e-mail du marchand passe à l'état validé.

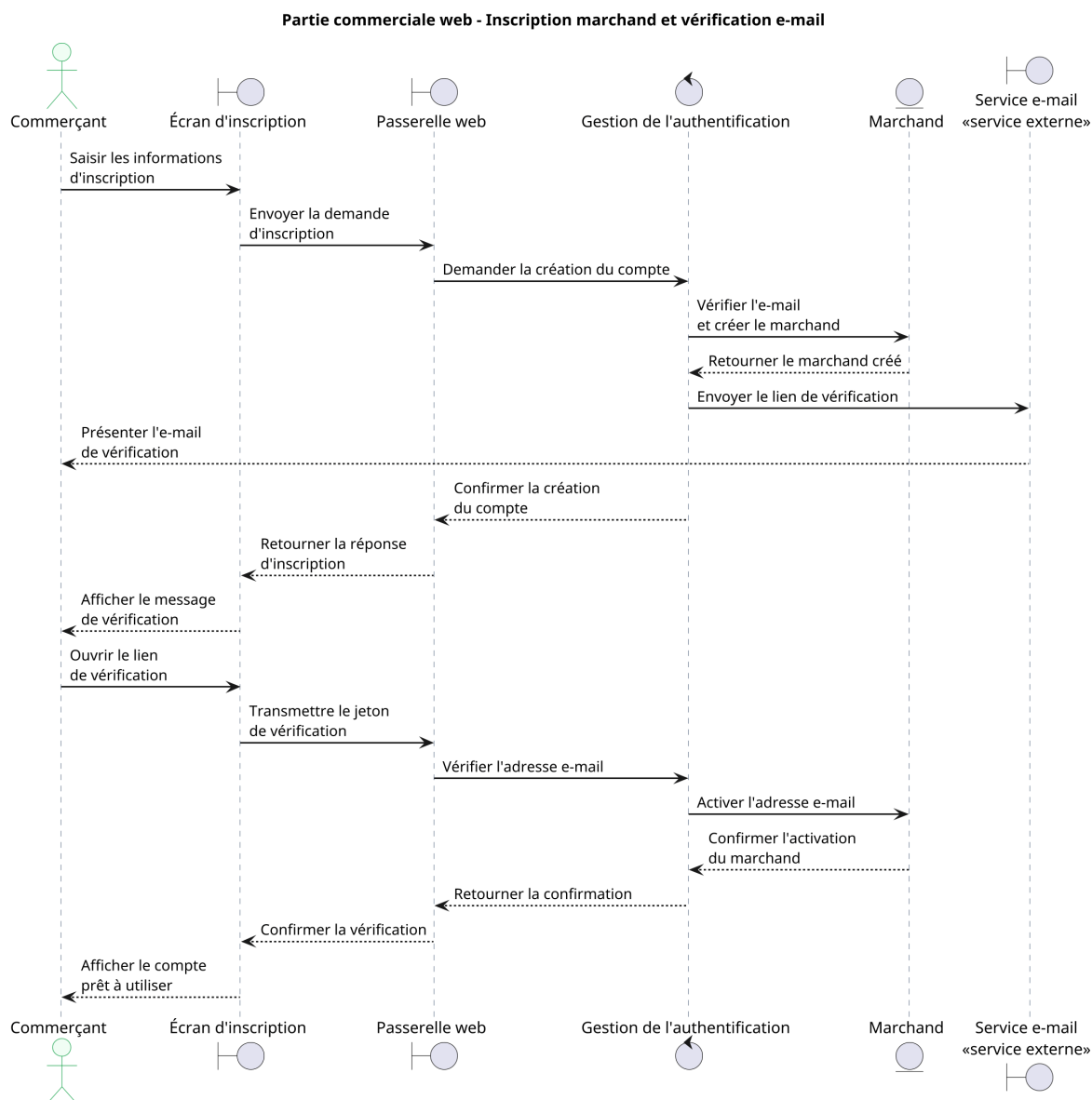


FIGURE 4.9 – Séquence technique d’inscription marchand avec vérification e-mail

2.5 Connexion marchand

Point d’entrée et rôle du flux

Après vérification de son adresse e-mail, le commerçant peut se connecter depuis l’écran de connexion. Ce flux est volontairement séparé de l’inscription : l’inscription crée et active le compte, tandis que la connexion ouvre une session web utilisable dans l’espace commerçant.

Étapes principales

- **Saisie** : le commerçant renseigne son e-mail et son mot de passe.
- **Authentification** : la passerelle web transmet les identifiants au service métier.

- **Session** : les cookies de session sont installés côté navigateur.
- **Redirection** : le commerçant accède au tableau de bord.

Logique technique

La réponse de connexion ne se limite pas à valider le mot de passe. Elle récupère aussi le profil marchand et crée une session web afin que les pages suivantes puissent charger les boutiques, les commandes et les clés API du marchand connecté sans redemander ses identifiants.

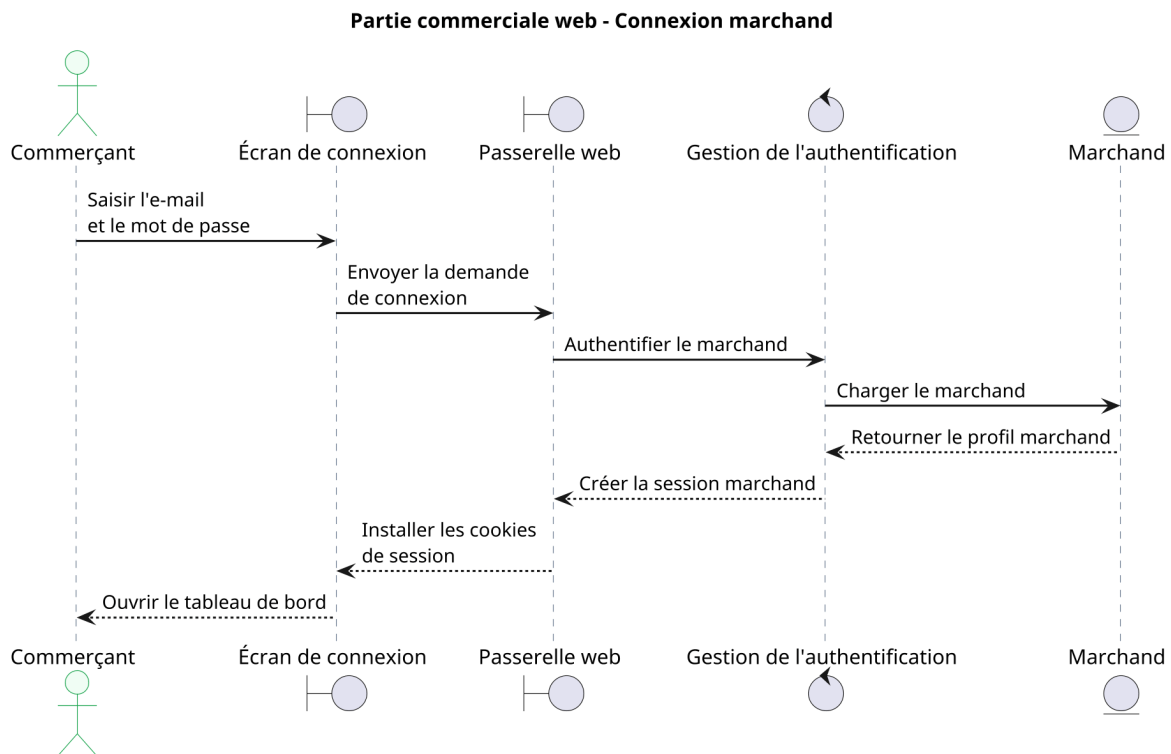


FIGURE 4.10 – Séquence technique de connexion marchand

2.6 Tableau de bord commerçant

Le tableau de bord synthétise l'activité du commerçant. Il affiche une carte des livraisons, les commandes créées récemment, les changements de statut et les notifications d'activité. La logique de chargement récupère les commandes, les magasins et les clés API du marchand courant afin de fournir une vue rapide de l'activité.

Ce tableau de bord sert de point d'entrée après connexion. Il évite au commerçant d'ouvrir directement chaque module pour savoir si une commande vient d'être créée, si une boutique a été ajoutée ou si une livraison a changé d'état.

2.7 Magasins

La page des magasins permet au commerçant de gérer ses boutiques. Un magasin représente un point de vente ou une boutique e-commerce connectée à GoMile. Les actions principales sont la création, la modification, l'activation, la désactivation, la suppression et la configuration du webhook.

La création d'une boutique demande les informations utiles à l'exploitation : nom, adresse, plateforme, domaine, description et coordonnées. Ces données servent ensuite à rattacher correctement les commandes et les clés API.

2.8 Flux de création d'une boutique

Point d'entrée et rôle du flux

Depuis la page des magasins, le commerçant peut déclarer une nouvelle boutique. Cette boutique devient ensuite le point de rattachement des commandes et des clés API. Le flux ne crée donc pas seulement une fiche d'affichage : il prépare aussi les futures intégrations e-commerce.

Données saisies et contrôles

- **Informations boutique** : nom, adresse, plateforme, domaine et description.
- **Adresse** : normalisation des coordonnées via l'API de géocode.
- **Propriétaire** : association de la boutique au marchand connecté.

Logique technique

Le front valide les champs obligatoires, puis transmet la demande à la passerelle web sécurisée. Le service de gestion des boutiques vérifie l'adresse, crée l'entité boutique et renvoie le résultat à l'écran des magasins. La nouvelle boutique peut alors apparaître dans la liste et être utilisée dans les autres modules.

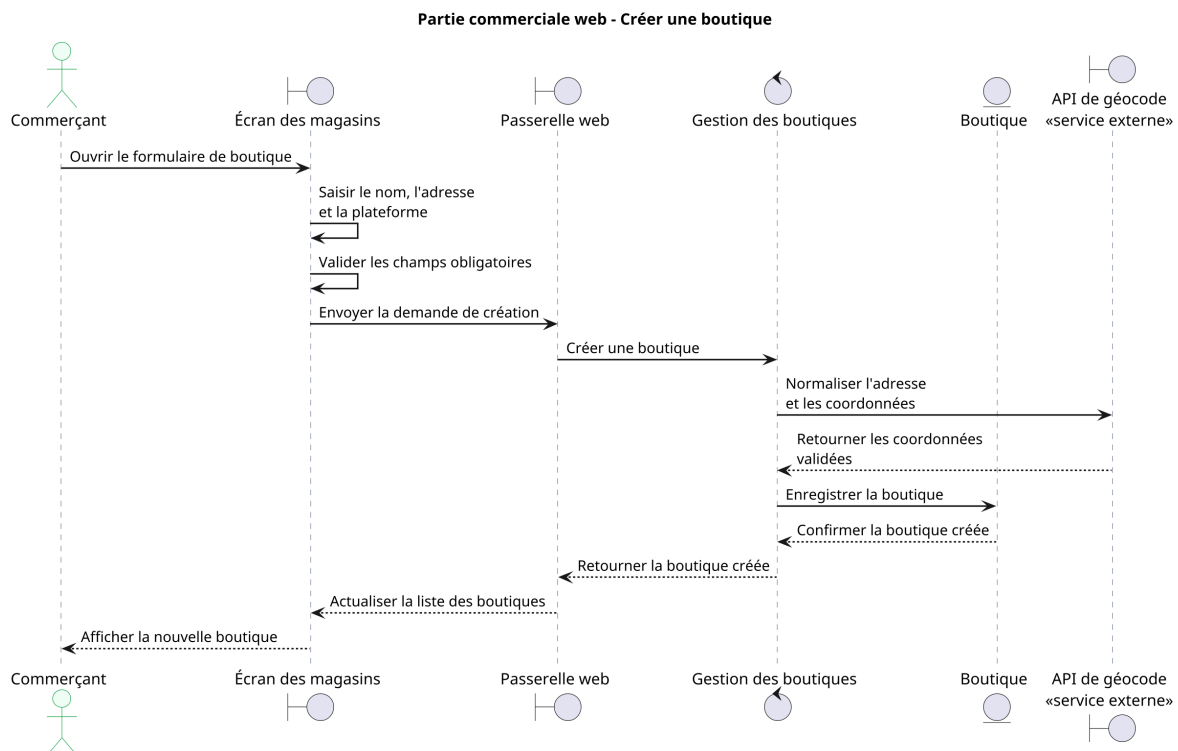


FIGURE 4.11 – Séquence de création d’une boutique

2.9 Commandes

La page des commandes sert à créer et suivre les livraisons. Une commande contient le client, l’adresse de livraison, la référence externe, le type de colis, le magasin concerné et le statut. Le commerçant peut annuler une commande seulement lorsque son statut le permet.

Point d’entrée et rôle du flux

La création d’une commande commence depuis l’écran des commandes. Le commerçant choisit le magasin concerné, renseigne les informations du client final et décrit la livraison. Ce flux transforme une demande commerciale en commande GoMile exploitable par le système de livraison.

Données manipulées

- **Magasin** : boutique du marchand à l’origine de la commande.
- **Client final** : nom, contact et adresse de livraison.
- **Colis** : type de colis, référence externe et informations utiles au suivi.

Logique technique

Au chargement de l’écran, le front récupère les magasins disponibles et les commandes existantes. Lors de la validation, la passerelle web envoie la demande au service de

commandes. Le service crée la commande avec son état initial, puis renvoie le résultat pour que l'écran affiche immédiatement la commande créée.

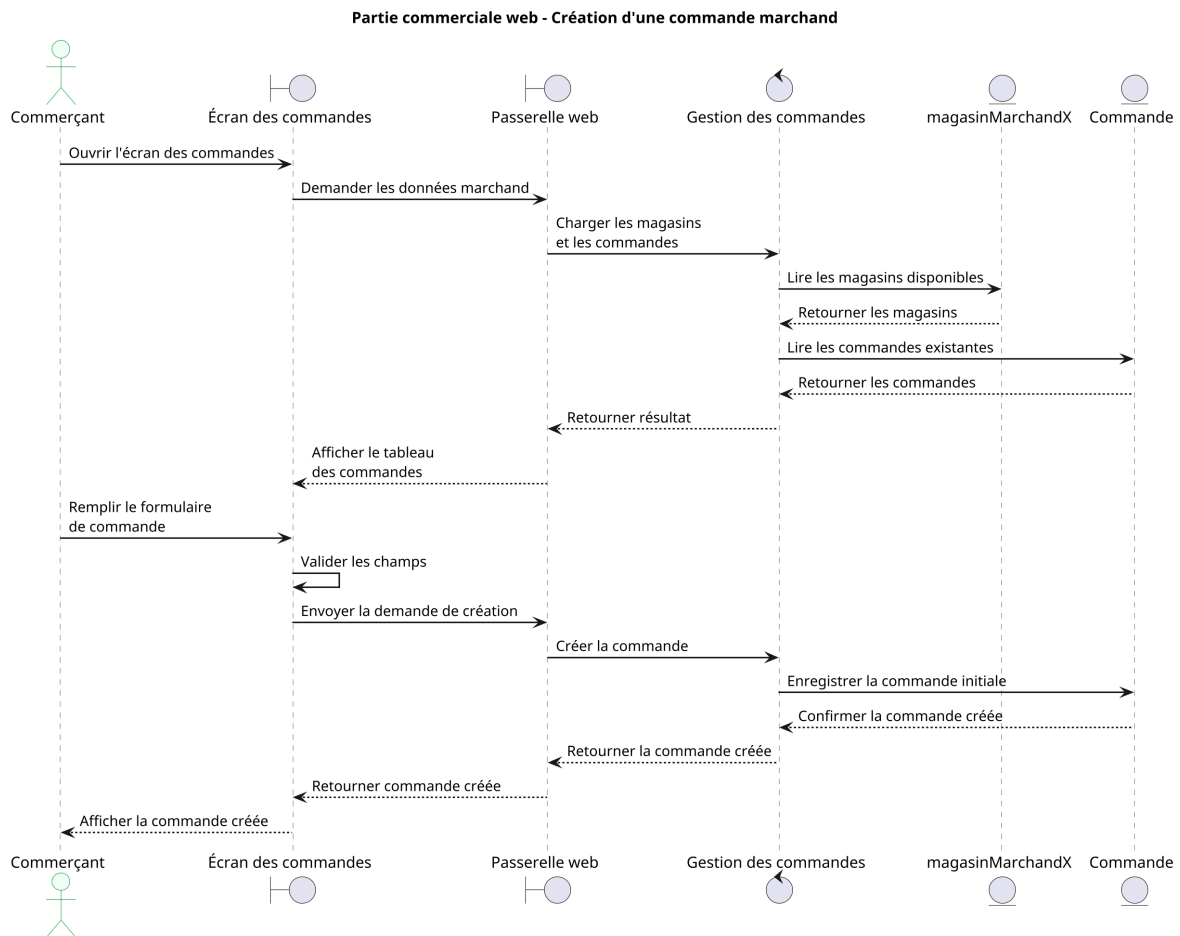


FIGURE 4.12 – Séquence de création d'une commande marchand

2.10 Flux de suivi et d'annulation d'une commande

Point d'entrée et rôle du flux

La liste des commandes permet au commerçant de suivre les livraisons déjà créées. Elle sert aussi de point de contrôle lorsqu'une commande doit être annulée. L'annulation n'est pas une simple action visuelle : elle dépend de l'état réel de la commande côté backend.

Règles appliquées

- **Consultation** : charger les commandes du marchand connecté.
- **Contrôle du statut** : vérifier si la commande peut encore être annulée.
- **Mise à jour** : marquer la commande comme annulée si la règle métier l'autorise.

Logique technique

Le front demande la liste des commandes, puis affiche les statuts retournés. Quand le commerçant déclenche une annulation, la demande repasse par le service de commandes. Le statut courant est relu avant modification afin d'éviter d'annuler une commande déjà récupérée, livrée ou déjà annulée.

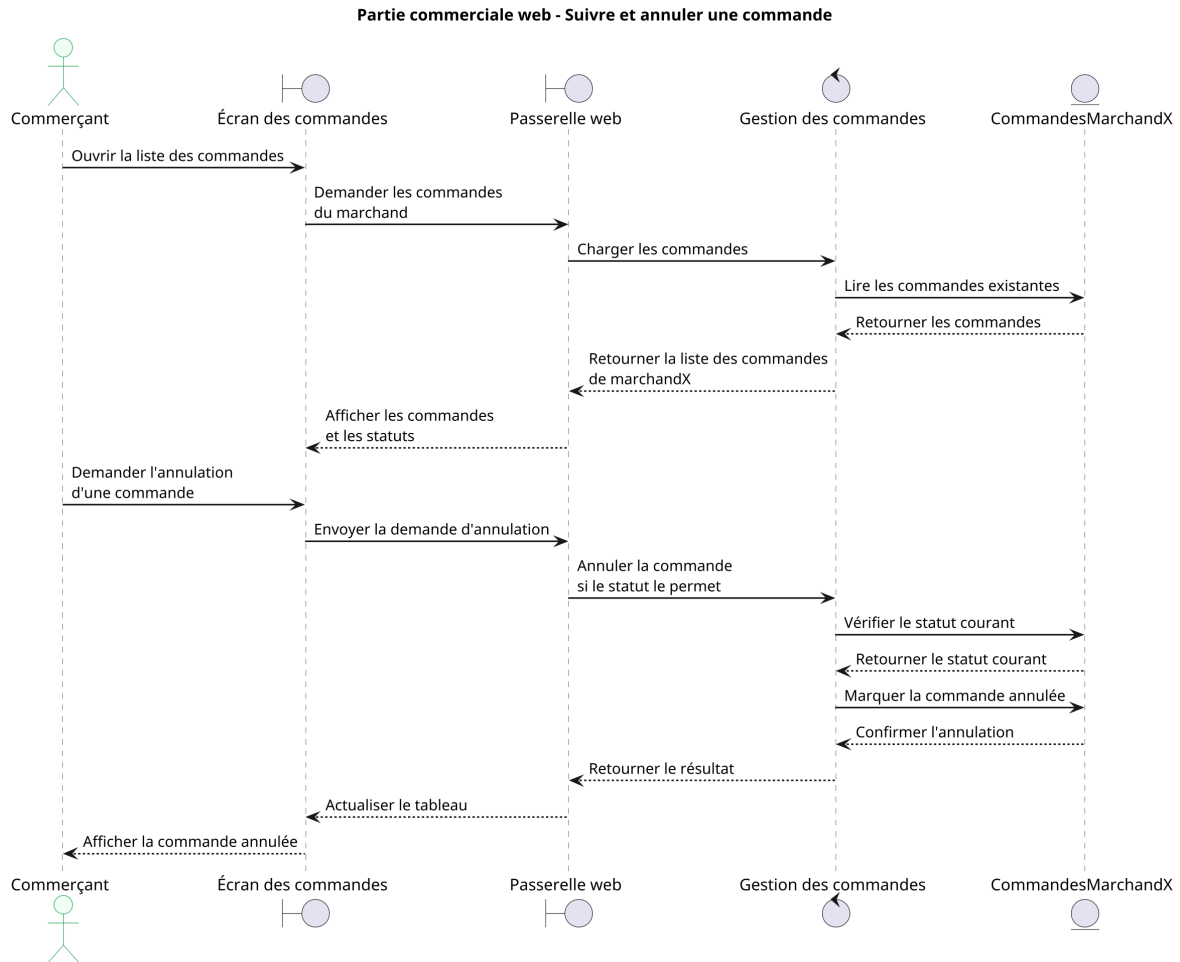


FIGURE 4.13 – Séquence de suivi et d'annulation d'une commande

2.11 Clés API

La page des clés API permet de créer, modifier ou révoquer les clés utilisées par les intégrations e-commerce. Chaque clé est rattachée à un magasin. Cette séparation permet à un commerçant de connecter plusieurs boutiques tout en gardant une clé différente pour chaque intégration.

Cette conception évite qu'une seule clé donne accès à toutes les boutiques d'un commerçant. Elle rend aussi plus simple la révocation d'une intégration précise sans bloquer les autres magasins.

2.12 Flux de création d'une clé API de boutique

Point d'entrée et rôle du flux

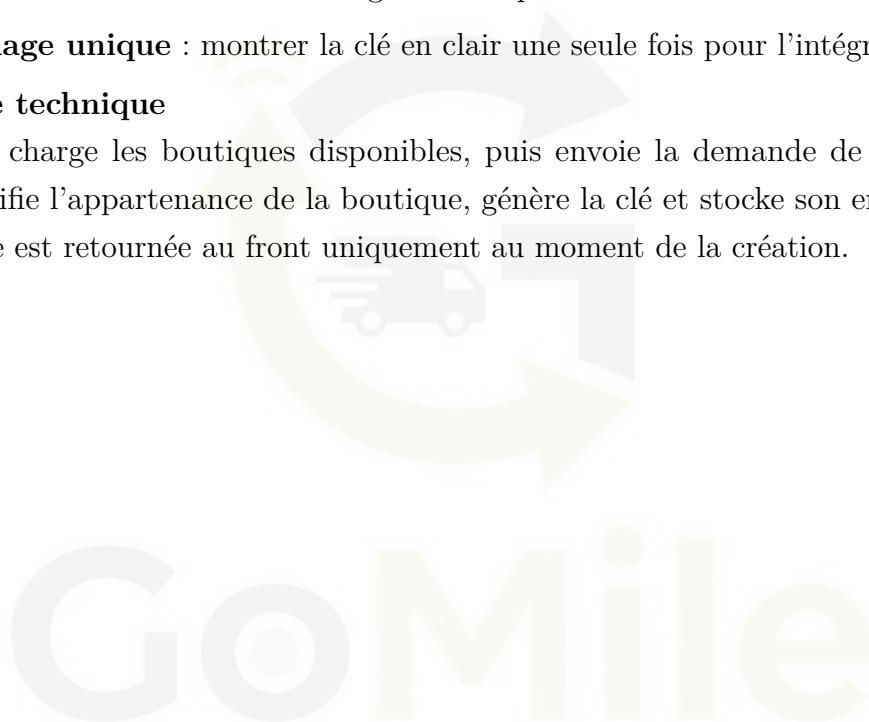
La création d'une clé API permet au commerçant de connecter une boutique à une plateforme e-commerce ou à un outil externe. La clé est rattachée à une seule boutique afin de limiter son périmètre d'utilisation.

Sécurité et données

- **Choix de boutique** : sélectionner le magasin qui sera connecté.
- **Vérification** : contrôler que la boutique appartient au marchand connecté.
- **Génération** : créer la clé et enregistrer uniquement sa version sécurisée.
- **Affichage unique** : montrer la clé en clair une seule fois pour l'intégration.

Logique technique

Le front charge les boutiques disponibles, puis envoie la demande de création. Le backend vérifie l'appartenance de la boutique, génère la clé et stocke son empreinte. La valeur lisible est retournée au front uniquement au moment de la création.



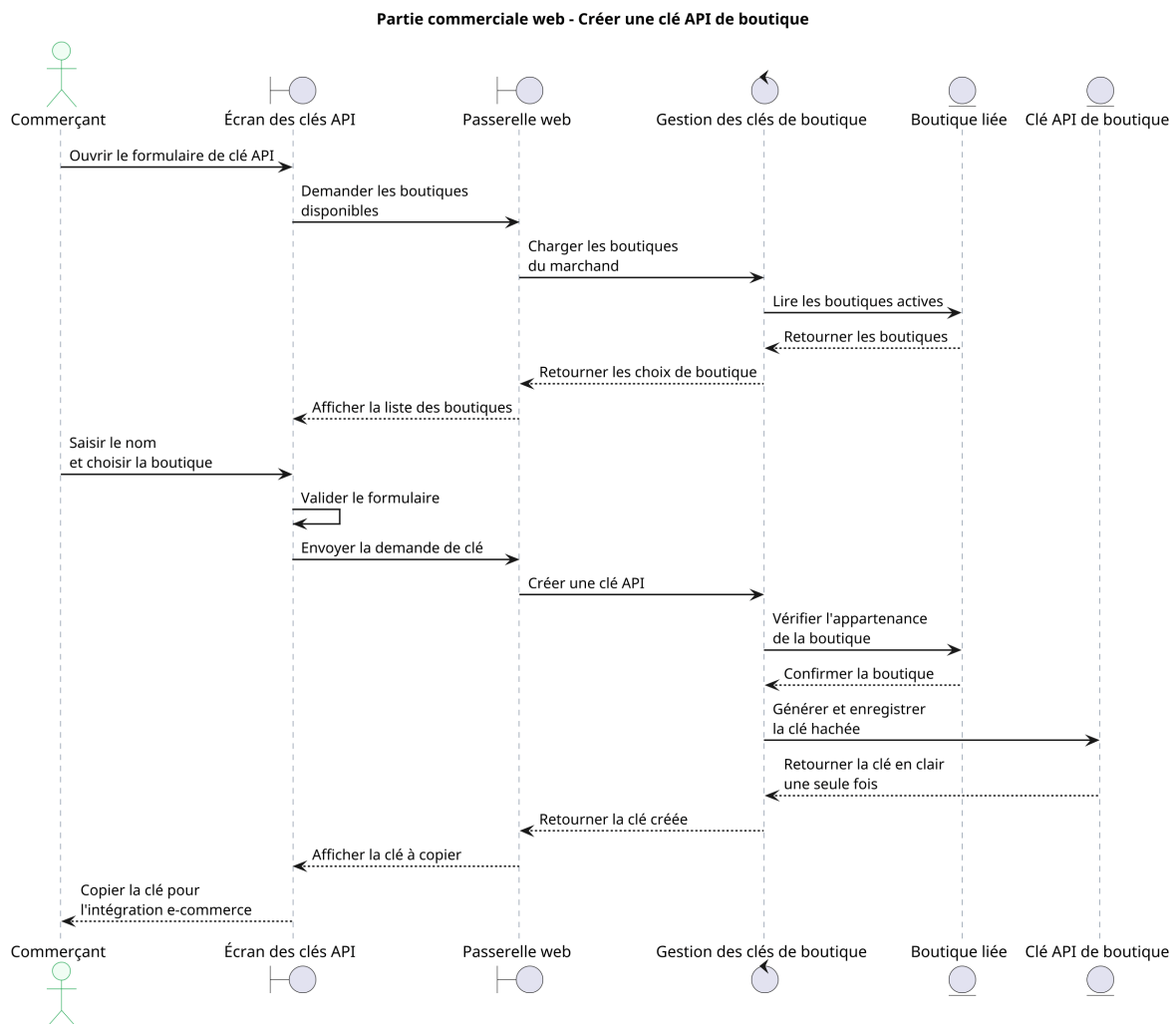


FIGURE 4.14 – Séquence de création d'une clé API de boutique

2.13 Flux de handshake

Rôle du handshake

Dans la partie web, le handshake correspond à la validation de récupération du colis chez le commerçant. Le but est de confirmer que le livreur est bien présent au point de retrait avant de faire avancer l'état de la commande.

Code utilisé

- **Code A** : OTP numérique à 6 chiffres présenté par le livreur et saisi par le commerçant dans l'interface web.

Logique technique

Au moment du retrait, le livreur présente le code A au commerçant. Le commerçant saisit ce code dans l'écran handshake marchand. La passerelle web transmet la demande au service de handshake, qui compare le code reçu avec le code actif de la commande.

Si le code est correct, la commande passe à l'état récupéré et l'interface web confirme la validation.

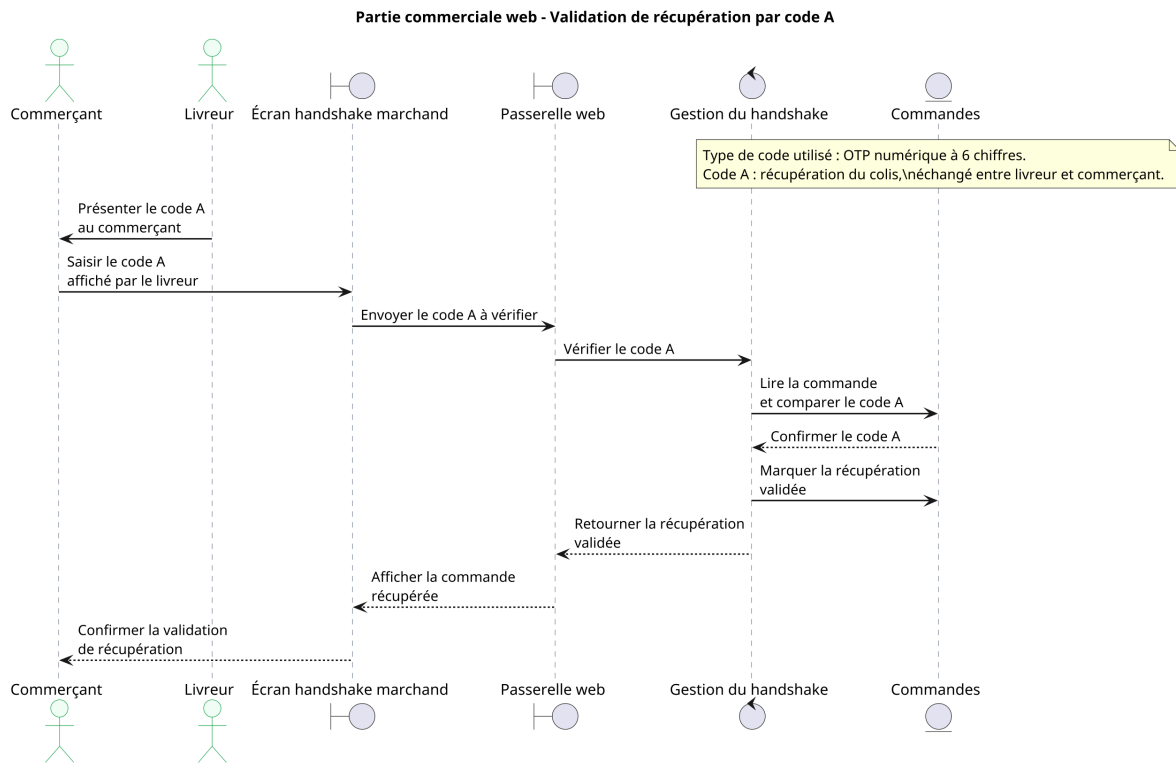


FIGURE 4.15 – Séquence web de validation de récupération avec code A

2.14 Conception de la partie livreur web

Rôle du parcours livreur web

La partie livreur web sert à créer un dossier candidat depuis le navigateur. Le parcours est découpé en étapes pour éviter un formulaire trop long et pour séparer les informations personnelles, les préférences de livraison et les documents de vérification.

- **Identité** : prénom, nom, e-mail, téléphone, avatar et mot de passe ;
- **Profil** : date de naissance, genre, ville, code postal et rue ;
- **Livraison** : ville de livraison, rayon et type de transport ;
- **Documents** : pièces justificatives et informations nécessaires au contrôle KYC.

Données et validation

Les informations sont conservées pendant le passage d'une étape à l'autre. Les champs obligatoires sont contrôlés avant progression, les fichiers sont préparés séparément, puis les références des documents sont associées au dossier livreur.

Logique technique

Le composant principal orchestre les étapes d'inscription. Le hook d'inscription garde l'état du formulaire, prépare les fichiers et appelle la passerelle web pour déposer les documents KYC. Une fois les documents prêts, la demande finale crée le profil livreur et place le dossier en attente de vérification par l'administration.

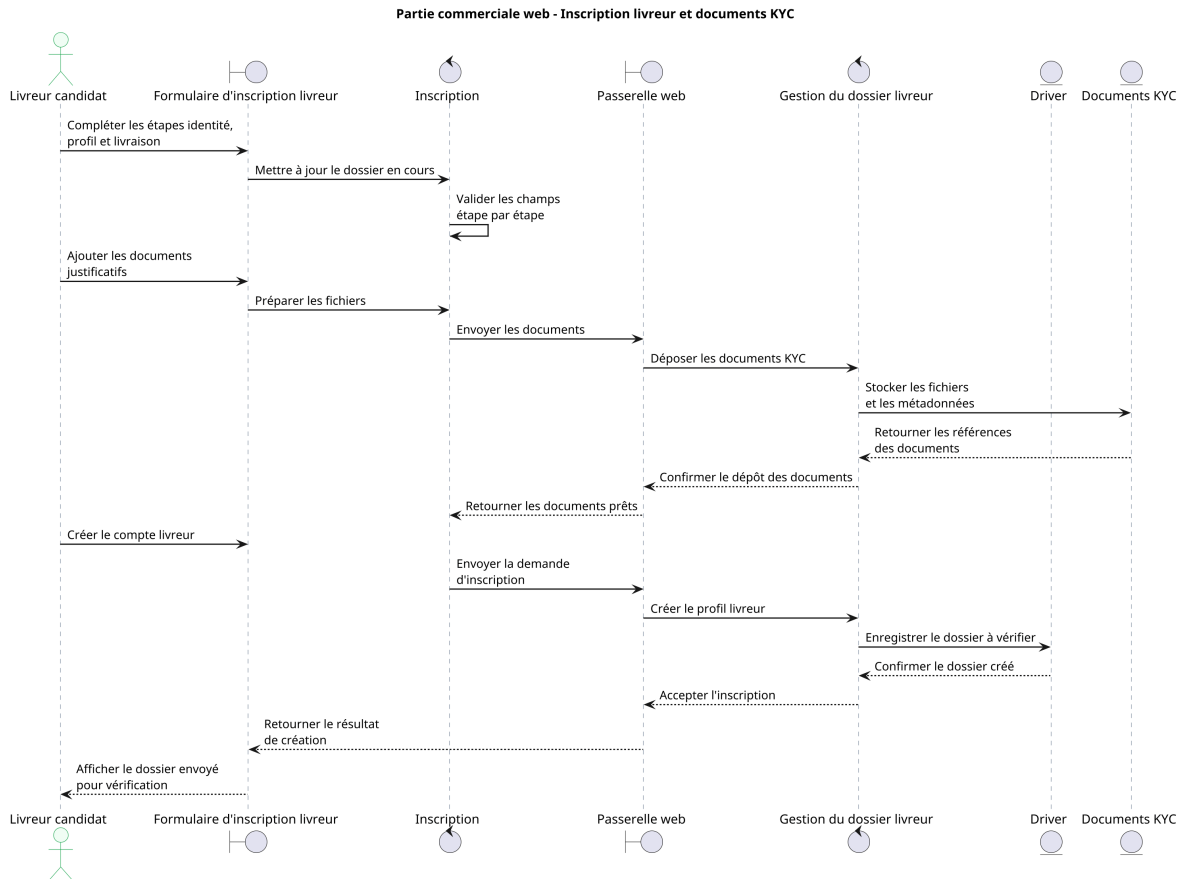


FIGURE 4.16 – Séquence d'inscription livreur et transmission des documents KYC

Chapitre 5

Application Mobile Livreur

0.1 Rôle de la partie mobile

L'application mobile constitue l'interface dédiée aux livreurs du projet GoMile. Elle leur permet de :

- créer un compte et finaliser l'activation,
- consulter et accepter des missions,
- suivre une livraison avec vérification par code,
- gérer leur profil, leur disponibilité et leurs documents (aka Know Your Customers).

Cette partie a été développée en React Native avec Expo et Expo Go pour accélérer les itérations et faciliter les démonstrations sur téléphone pendant le projet. Les builds prenant beaucoup de temps, nous les consacrons pour les démos de la soutenance.

0.2 Architecture générale et technologies

L'application mobile repose sur les briques suivantes :

React Navigation gestion des écrans d'authentification et des onglets principaux.

Expo accès aux fonctionnalités natives (média, géolocalisation, notifications) avec un cycle de développement rapide.

API backend GoMile authentification, missions, profil, vérification et opérations métier.

Modules externes `@react-native-community/datetimepicker` pour la date, composants média pour photo/document, services d'adressage pour fiabiliser les adresses.

Local storage avec Zustand

Le flux d'inscription traverse plusieurs écrans successifs. Les données doivent donc être partagées entre écrans, conservées pendant la navigation, et réutilisées au moment de l'appel API final. Pour cette raison, nous avons retenu Zustand (`useRegistrationStore`) plutôt qu'un simple objet JavaScript local à un écran. Ce choix apporte :

- un état centralisé lisible,
- des mises à jour champ par champ,
- une meilleure robustesse si l'utilisateur revient à l'étape précédente.

La description détaillée des champs arrive ensuite, pour montrer concrètement ce qui est stocké à chaque étape.

0.3 Parcours d'inscription

Point d'entrée et logique de découpage

Depuis l'écran `LoginScreen`, le bouton `Devenir livreur` ouvre le circuit d'inscription.

Le parcours a été progressivement retravaillé lors des suivis avec M. Viardot pour améliorer l'UX : simplification des écrans, réduction des frictions, repositionnement des informations administratives, et validation plus claire des champs. Le flux actuel est volontairement découpé en **trois étapes d'inscription** :

- étape 1 : identité de base,
- étape 2 : informations personnelles complémentaires,
- étape 3 : logistique de livraison.

Un ancien écran de finalisation (ancien `RegisterStep4`) existe encore dans le code, mais il ne fait plus partie du flux UX principal, car les retours d'usage ont montré qu'il vaut mieux finaliser certains éléments après création du compte.

Étape 1 : identité de base (`RegisterStep1.js`)

Champs demandés et types :

- photo de profil : image locale (URI),
- prénom : chaîne de caractères,
- nom : chaîne de caractères,
- email : chaîne de caractères au format email,
- mot de passe : chaîne de caractères,
- confirmation mot de passe : chaîne de caractères.

Modules et UX :

- Le choix de la photo passe par le module `media-picker`, qui encapsule les permissions système et la sélection de média.
- Le champ email utilise un clavier adapté à la saisie email (présence du caractère "@" facilitant la saisie).

Validation et stockage Zustand :

- Vérification des champs obligatoires.
- Vérification de la cohérence mot de passe / confirmation.
- Stockage des champs avec le hook personnalisé `useRegistrationStore` : `firstName`, `lastName`, `email`, `password`, `confirmPassword`, `avatarUrl` sont les nom de contenant Zustand que nous avons utilisé.

Étape 2 : identité complémentaire (`RegisterStep2Identity.js`)

Champs demandés et types :

- numéro de téléphone : chaîne normalisable en format FR/international,
- date de naissance : date,
- genre : valeur énumérée (Homme, Femme, Autre).

Modules et UX :

- Le numéro de téléphone utilise un clavier numérique.
- La date utilise `@react-native-community/datetimepicker`, retenu pour sa compatibilité native iOS/Android et son comportement rétractable.
- Le genre est sélectionné avec des boutons à choix explicite.

Validation et stockage Zustand :

- Champs obligatoires contrôlés avant passage à l'étape suivante.
- Stockage dans `useRegistrationStore` avec `phone`, `birthDate`, `gender` utilisés comme contenants.

Étape 3 : logistique de livraison (`RegisterStep2.js`)

Champs demandés et types :

- adresse de résidence : chaîne structurée,
- ville de livraison : chaîne,
- rayon de livraison : entier (km),
- mode de transport : enum (vélo, moto, voiture, utilitaire).

Modules et UX :

- Adresse gérée via un composant d'autocomplétion connecté au backend et à l'API d'adressage gouvernementale "`https://api-adresse.data.gouv.fr/`" pour récupérer des adresses fiables.
- Le rayon impose une saisie numérique uniquement.
- Le mode de transport est proposé sous forme de cartes de sélection pour une lecture rapide.

Normalisation et stockage Zustand :

- Normalisation du téléphone au format international attendu.
- Conversion du rayon en entier exploitable côté API.
- Mapping du transport vers les codes métier (BIKE, SCOOTER, CAR, TRUCK).
- Stockage dans `useRegistrationStore` : `address`, `city`, `zipCode`, `street`, `deliveryCity`, `deliveryRadius`, `transportType`.

Séquence technique d'inscription

Le diagramme suivant décrit le chemin complet : saisie des étapes, création du compte, envoi de l'email de vérification, attente active côté mobile, puis redirection automatique après confirmation.

0.4 Connexion et vérification email

Après lancement de l'application, l'utilisateur arrive sur `LoginScreen`. Il saisit son identifiant (email) et son mot de passe.

- Si le compte est valide, l'application ouvre directement l'espace principal.
- Si l'email n'est pas vérifié, l'utilisateur est redirigé vers `EmailVerificationScreen`.

Cet écran de vérification interroge périodiquement le backend (chaque 2000ms). En effet, nous attendons une amélioration de la part du backend pour utiliser les websockets pour améliorer les performances. Une fois le mail confirmé via le lien reçu, l'application met à jour la session et redirige automatiquement vers le tableau de bord.

0.5 Réinitialisation du mot de passe

Depuis `LoginScreen`, le bouton `Mot de passe oublié` ouvre le flux de réinitialisation :

- saisie du mail du livreur
- envoi d'une demande de reset,

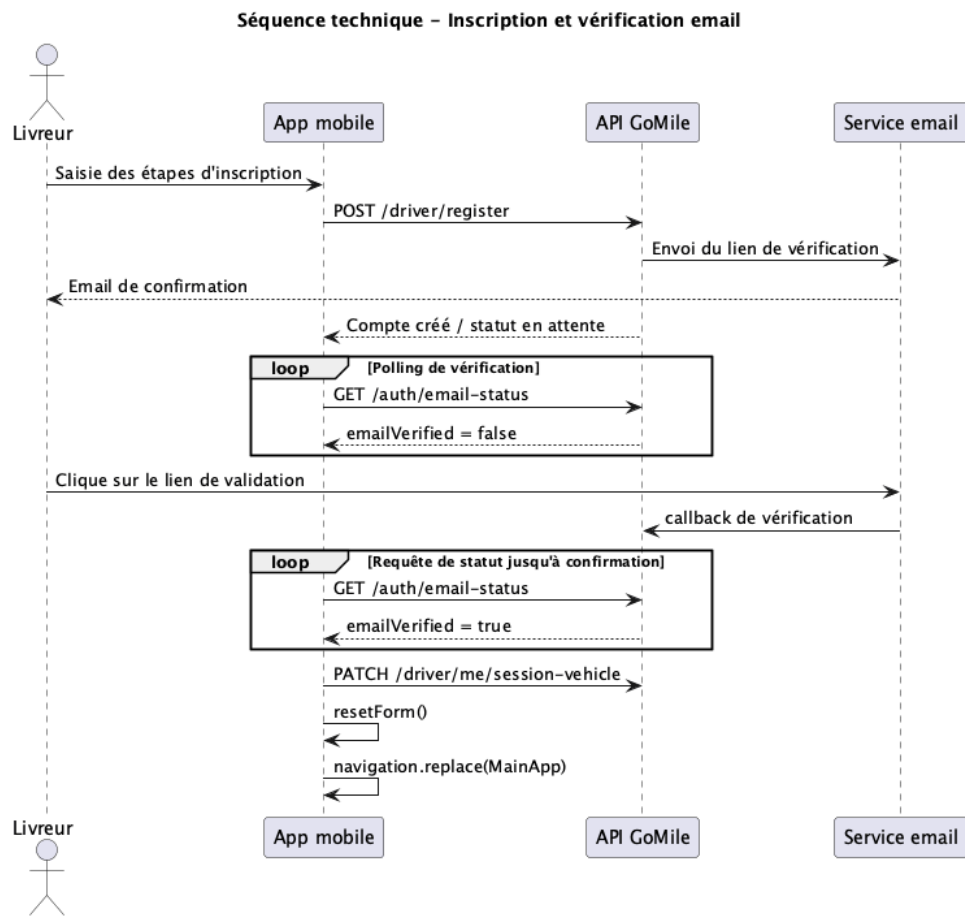


FIGURE 5.1 – Séquence technique d'inscription avec vérification email

- réception d'un email,
- définition d'un nouveau mot de passe,
- retour à la connexion.

Cette séparation permet de garder un écran de connexion simple tout en offrant un parcours de récupération complet.

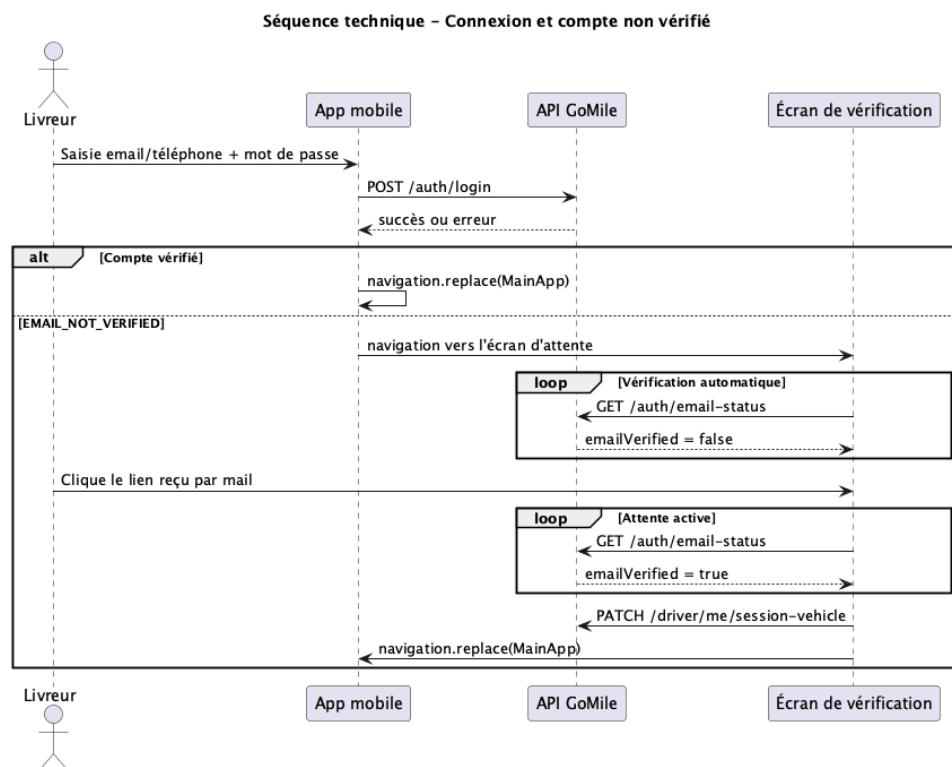


FIGURE 5.2 – Séquence technique de connexion avec gestion de l'email non vérifié

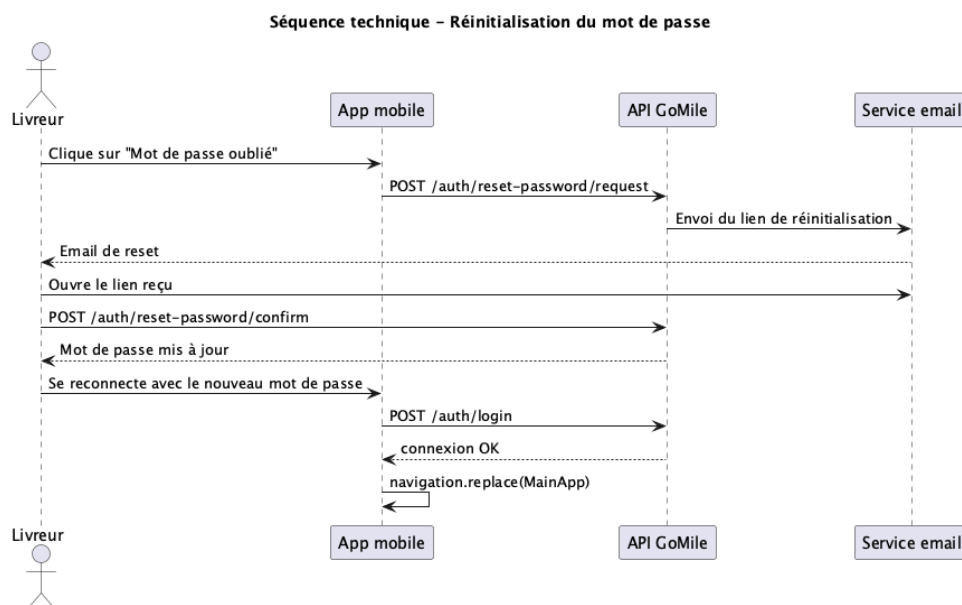


FIGURE 5.3 – Séquence technique de réinitialisation du mot de passe

0.6 Diagramme d'état global authentification

Le diagramme d'état ci-dessous résume les transitions entre les états principaux : connexion, inscription, attente de vérification email, accès au dashboard, et retour après déconnexion ou reset du mot de passe.

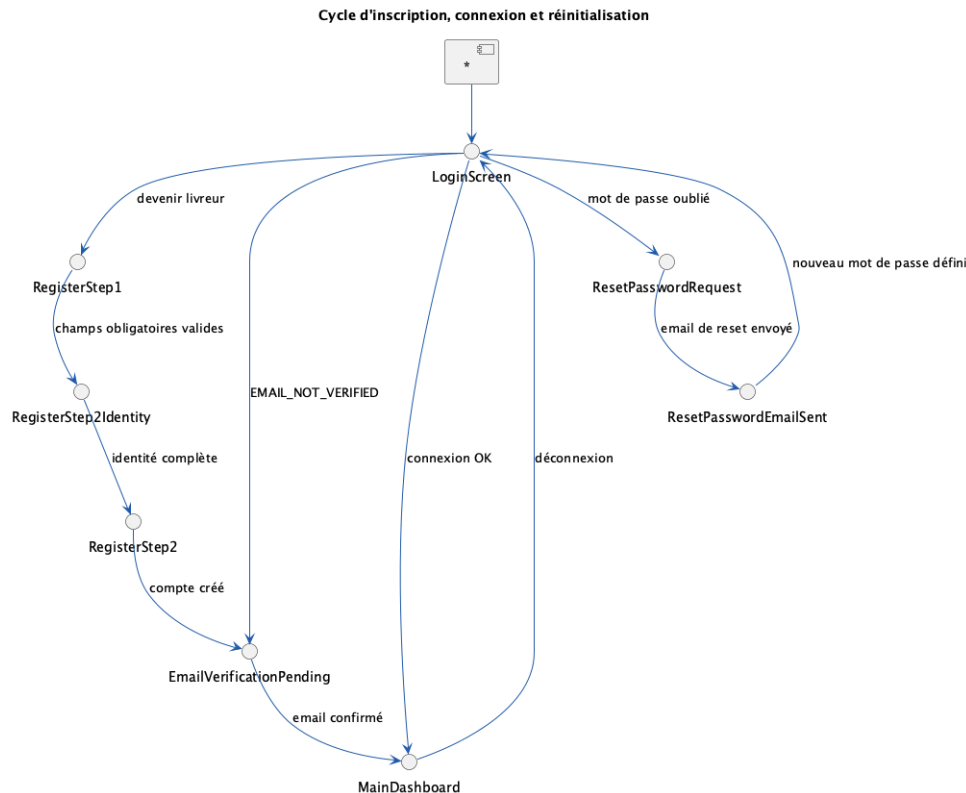


FIGURE 5.4 – Diagramme d'état global du cycle inscription-connexion-reset

0.7 Décisions UX abandonnées ou déplacées

Les évolutions du parcours ont conduit à déplacer plusieurs éléments hors du tunnel d'inscription principal :

- Les documents KYC ne sont plus demandés pendant les trois étapes d'inscription : ils sont traités après création du compte, depuis le profil.
- L'ancien écran de finalisation (ancien **RegisterStep4**) reste présent dans le code, mais n'est plus intégré au flux UX principal.
- La vérification du numéro de téléphone (OTP) n'est pas faite pour le moment pour des questions de budget car nous utilisons Twilio qui peut vite être couteux au vu de nos tests quotidiens

0.8 Navigation et écrans post-connexion

Architecture de navigation principale (MainTabs)

Une fois connecté, le livreur arrive sur `MainTabs.js`, qui constitue le cœur de l'application post-authentification. Cette structure repose sur :

React Navigation Bottom Tab Navigator organisation de quatre onglets principaux accessibles en permanence au bas de l'écran.

Stack Navigator implicite chaque onglet peut avoir ses propres écrans enfants (ex : `MissionDetailsScreen` accessible depuis `MissionsScreen`).

Push Notifications configuration des notifications à ce niveau, avec enregistrement du token et liaison `WebSocket`.

Suivi GPS mise à jour de la position du livreur à intervalle régulier pour enrichir les suggestions de missions.

WebSocket (socket.io) connexion établie ici pour recevoir les événements en temps réel (nouvelles missions, changements de statut).

Action rapide globale (très utile, et dont nous sommes beaucoup fier!) un bouton flottant de mission apparaît au-dessus des onglets dès qu'il existe au moins une mission active ; il renvoie directement vers `MissionFocusScreen` depuis n'importe quel onglet.

Header personnalisé avec toggle en ligne/hors ligne

Le header `Header.js` affiche :

- Le titre de la page courante (ex : «TABLEAU DE BORD», «MISSIONS»).
- Un indicateur **EN LIGNE** / **HORS LIGNE** avec un bouton switch.
- Lorsque le livreur bascule **en ligne**, il devient visible dans les requêtes de mission ; **hors ligne**, aucune mission ne lui est proposée.
- La transition repose sur un appel à `toggleDriverAvailability()` (endpoint `PATCH /driver/me/availability`).
- Sécurité : un livreur ne peut pas se mettre hors ligne s'il a une mission active en cours.

Magasin d'informations partagées : `useAvailabilityStore` (simple toggle Zustand) maintient l'état `isOnline` en mémoire de session.

Écran 1 : Tableau de Bord (Dashboard / TableauDeBord.js)

Purpose et composition

Le Tableau de Bord est l'écran d'accueil principal post-connexion. Il présente :

- **Carte interactive Google Maps** affichant :
 - La position actuelle du livreur (marqueur bleu).
 - Les missions disponibles à proximité (marqueurs rouges).
 - Un cercle de rayon 1 km autour de la position courante.
- **Widget de statistiques rapides** :
 - Gains du jour (format : `XX.XX€`).
 - Nombre de courses effectuées aujourd'hui.
- **Overlay de missions proposées** : défilement horizontal des cartes mission les plus proches, avec bouton «Voir détails».

Logique de localisation et mise à jour

À l'ouverture du Dashboard, on :

1. Demande de permission de géolocalisation (`Location.requestForegroundPermissionsAsync`).
2. Récupère la position courante avec précision `Balanced` sous condition de statut `onLine` du livreur.
3. Appelle immédiatement `updateDriverLocation(lat, lng)` pour synchroniser la position backend (nécessaire pour la requête géospatiale `ST_DWithin`).
4. Charge le dashboard via `getDriverDashboard()` pour les statistiques du jour.
5. Lance le hook de polling `useMissionsPoller( fetchAll: false )` qui va rafraîchir les missions disponibles au focus une fois qu'il y a de nouvelle mission avec les websockets.

Intégration des stores et hooks

Stores utilisés :

- `useAvailabilityStore` : lecture de `isOnline` pour afficher/masquer les missions.
- `useMissionStore` : lecture de `availableMissions` pour afficher les marqueurs et les cartes.

Hooks :

- `useMissionsPoller( fetchAll: false )` : fetch unique des missions disponibles au premier focus de l'écran et au passage en ligne.

Endpoints API appelés :

- `getDriverDashboard()` : GET `/driver/me/dashboard` -> retourne `{isOnline, todayEarnings, todayTrips}`.
- `updateDriverLocation(lat, lng)` : PATCH `/driver/me/location` -> envoie la position actuelle.
- `getAvailableMissions()` : GET `/driver/me/missions/available` -> retourne tableau de missions.

Interaction utilisateur

- **Tap sur une carte mission** : ouvre `MissionDetailsScreen` avec les détails complets.
- **Tap sur le bouton GPS** : recentre la carte sur la position courante.
- **Affichage hors ligne** : les missions ne sont pas visibles ; message : «Les missions proposées sont masquées hors ligne».

0.9 Écran enfant : Détails Mission (`MissionDetailsScreen`)

Purpose et composition

Cet écran affiche les détails complets d'une mission sélectionnée depuis le Tableau de Bord ou depuis l'écran Missions. L'utilisateur peut y accepter ou refuser la mission.

Contenu affiché :

- Carte avec trajet pick-up → destination (Polyline).
- Nom du commerce, adresse pick-up, zone de livraison.
- Rémunération, distance estimée, durée.
- Boutons : REFUSER et ACCEPTER.

Logique d'acceptation et refus

Accepter :

1. Appel API `acceptMission(missionId)` : POST `/driver/me/missions/{id}/accept`.
2. Dispatch action Zustand `acceptMission(missionCard)` qui ajoute la mission à `missionQueue`.
3. Navigation vers `MissionFocusScreen` pour débiter le suivi GPS en direct.

Refuser :

1. Alerte de confirmation : « Cette mission sera retirée de ta liste ».
2. Dispatch action Zustand `declineMission(missionId)` qui :
 - Ajoute l’ID à `declinedMissionIds`.
 - Filtre la mission hors de `availableMissions`.
3. Navigation arrière.

Calcul du trajet et géocodage

À l’ouverture :

- Récupère la position courante du livreur.
- Appelle `getDrivingRoute([currentPos, pickupPos, dropoffPos])` pour obtenir la route optimale.
- Affiche distance formatée et durée estimée.

Endpoints API :

- `acceptMissionApi(missionId)` : POST acceptation.
- `getDrivingRoute()` : calcul route (utilise service Google Maps ou routeur local).

0.10 Diagramme de flux : Navigation Tableau de Bord → Détails Mission → Focus

FIGURE 5.5 – Séquence technique : le livreur arrive sur le Tableau de Bord, consulte une mission via MissionDetailsScreen, puis l'accepte pour passer en MissionFocusScreen et débiter le suivi GPS. Les stores Zustand maintiennent l'état entre chaque transition.



0.11 Diagramme d'état : Cycle de vie d'une mission acceptée

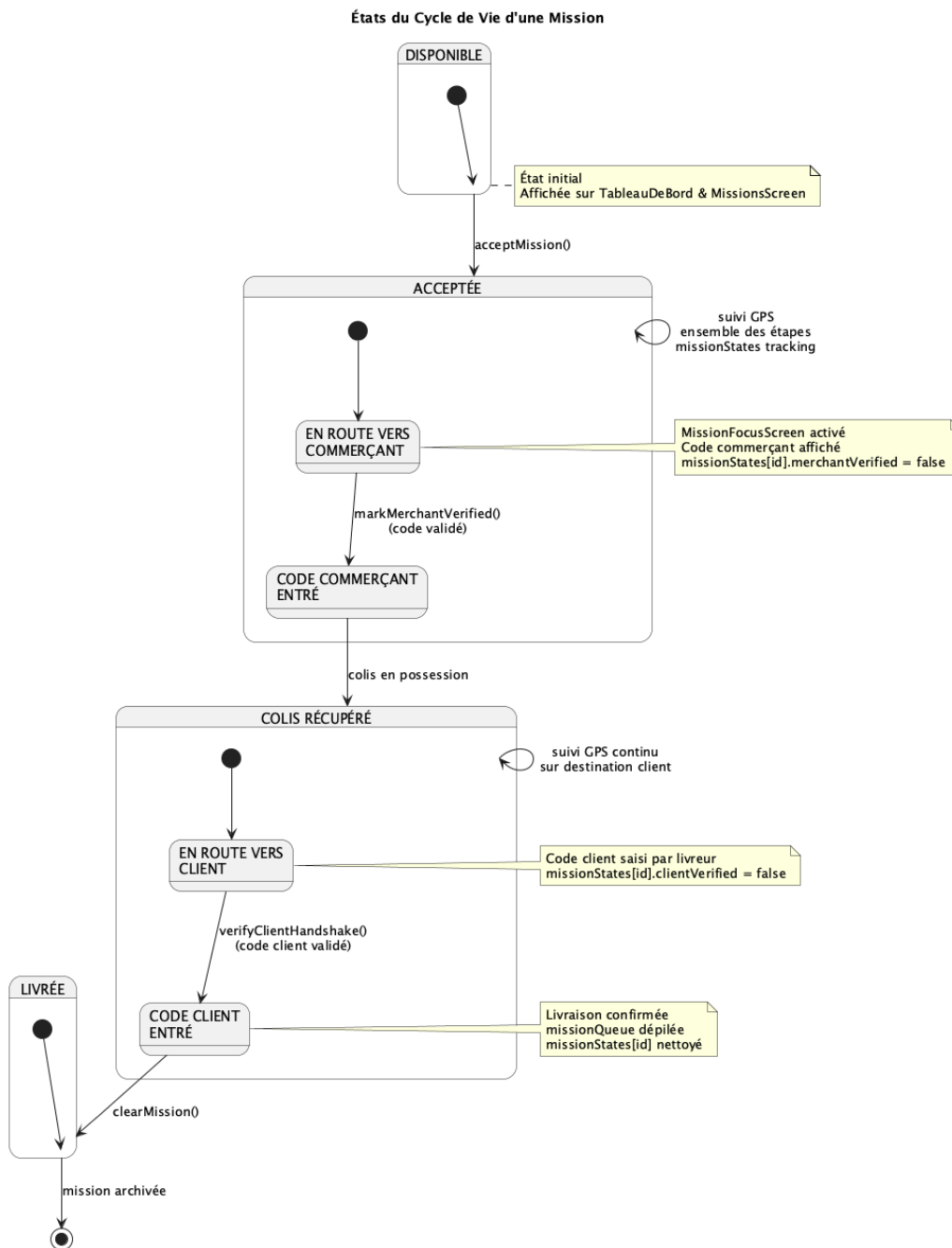


FIGURE 5.6 – États d'une mission : disponible → acceptée (EN ROUTE vers commerçant) → récupérée (RÉCUPÉRÉ colis, EN ROUTE vers client) → livrée (LIVRÉ). Les codes marchand et client actent le passage entre états.

0.12 Écran 2 : Missions (MissionsScreen)

Purpose et organisation par onglets

Cet écran centralise la gestion des missions sous trois onglets logiques :

Notifications Missions résumé du nombre de missions disponibles + horodatage du dernier rafraîchissement.

Onglet 1 : Missions en cours toutes les missions acceptées (état **active** dans `missionQueue`), avec bouton «REPRENDRE LA MISSION» redirigeant vers `MissionFocusScreen`.

Onglet 2 : Missions à proximité liste complète des missions disponibles avec bouton «VOIR LES DÉTAILS» ouvrant `MissionDetailsScreen` pour consultation avant acceptation.

Onglet 3 : Historique historique des livraisons (archivées) avec magasins et rémunérations.

Comportement selon le statut en ligne :

- **En ligne** : affichage des trois onglets + bannau informatif.
- **Hors ligne** : bannau unique expliquant que les missions ne sont pas disponibles.

Stores et intégration Zustand

Store `useMissionStore` - lectures :

- `availableMissions` : missions proposées à proximité (GET `/driver/me/missions/available`).
- `activeMissions` : missions en cours acceptées (GET `/driver/me/missions/active`).
- `historyMissions` : missions archivées (GET `/driver/me/missions/history`).
- `lastRefreshAt` : timestamp du dernier fetch.
- `isLoadingMissions` : indicateur de chargement.

Store `useMissionStore` - actions :

- `acceptMission(mission)` : ajoute à `missionQueue` et définit comme focalisée.
- `declineMission(missionId)` : ajoute à `declinedMissionIds`, filtre hors de `availableMissions`.
- `fetchAvailableMissions()` : appel backend pour missions disponibles.
- `fetchAllMissions()` : appel bundle pour tous les états (disponibles + actives + historique).

Polling et refresh

Le hook `useMissionsPoller( fetchAll: true )` gère :

- Fetch unique au premier focus de `MissionsScreen`.
- Fetch au passage du livreur en ligne.
- Bouton «**RAFRAÎCHIR**» manuel appelant `refresh()`.

Nouveau système (remplace le polling périodique) :

- Les nouvelles missions arrivent via `WebSocket` (événement `order:new`) reçu dans `MainTabs`.
- Le hook ne sert plus au polling périodique, mais au chargement initial et refresh manuel.
- Gain : réduction de la charge réseau, mises à jour en temps réel via événements.

Navigation et actions utilisateur

- Si on clique sur «**VOIR LES DÉTAILS**» : ça ouvre `MissionDetailsScreen` pour consultation et décision.
- Clique sur «**REPRENDRE LA MISSION**» : on appelle `acceptMission()`, navigue immédiatement vers `MissionFocusScreen`.
- Clique «**RAFRAÎCHIR**» : on appelle `refresh()` du hook de polling.

0.13 Écran enfant : Focus Mission (`MissionFocusScreen`)

Purpose : suivi GPS en direct et vérification codes

Cet écran est le cœur de la livraison. Il affiche :

- **Carte interactive** avec :
 - Marqueur bleu : position actuelle du livreur (mise à jour en temps réel).
 - Marqueurs rouges : toutes les missions en cours (pick-up en rouge, livraison en rose).
 - Polyline : trajet calculé vers la cible courante (malheureusement `react-native-maps` a des performances limitées au vu des tests réalisés et en comparaison de l'appli `googleMaps`)
- **Panneau bas** (collapsible) contenant :
 - Sélecteur de mission (si plusieurs en cours).
 - Code commerçant (affiché, non saisi).
 - Bouton de validation commerçant.

- Champ saisie code client (après validation commerçant).
- Bouton de validation client.

Logique de vérification : deux étapes

Étape 1 : Vérification commerçant

1. Livreur arrive au point de pick-up.
2. Code commerçant est affiché via `getMissionPickupCode(missionId)`.
3. Commerçant valide le code via son interface (pas de saisie livreur ici).
4. Livreur tape le bouton «VALIDER IDENTIFICATION COMMERÇANT».
5. Action `markMerchantVerified(missionId)` active l'étape suivante.
6. État : `missionStates[id].merchantVerified = true`.

Étape 2 : Vérification client

1. Livreur arrive au point de livraison.
2. Champ de saisie code client devient actif (6 chiffres).
3. Client fournit le code (qu'il a reçu via SMS).
4. Livreur saisit le code.
5. Appel API `verifyClientHandshake(missionId, code)` : POST `/driver/me/missions/{id}/handshake/client/verify`.
6. Si succès : action `markClientVerified(missionId)`.
7. Alerte : «Mission terminée, Livraison confirmée!».
8. Action `clearMission(missionId)` : dépile de `missionQueue`, met à jour `focusedMissionId`.

Suivi GPS continu

À l'ouverture du `MissionFocusScreen` :

1. Demande permission géolocalisation.
2. Lance `watchPositionAsync()` : mise à jour position toutes les 5 secondes (distance minimale : 10 m).
3. À chaque nouvelle position :
 - Met à jour `currentPosition` (état local).
 - Recalcule la route vers la cible courante via `getDrivingRoute()`.
 - Affiche distance et durée estimée.
4. Arrêt au démontage du composant.

Sélecteur de missions (si plusieurs en file)

Si `missionQueue.length > 1` :

- Affiche composant `MissionPicker` permettant de basculer entre missions.
- À chaque sélection, `setFocusedMission(id)` met à jour `focusedMissionId`.
- La carte et le panneau se réajustent instantanément ! (magie de `useEffect()`)
- Affichage du statut de chaque mission : EN ROUTE, RÉCUPÉRÉ, LIVRÉ.

Endpoints API utilisés

- `getMissionPickupCode(missionId)` : GET `/driver/me/missions/{id}/handshake/pickup-code`
→ code commerçant.
- `verifyClientHandshake(missionId, code)` : POST
`/driver/me/missions/{id}/handshake/client/verify` → validation livraison.
- `getDrivingRoute([coords])` : calcul route.

0.14 Diagramme de flux : Acceptation mission → Focus → Vérification codes

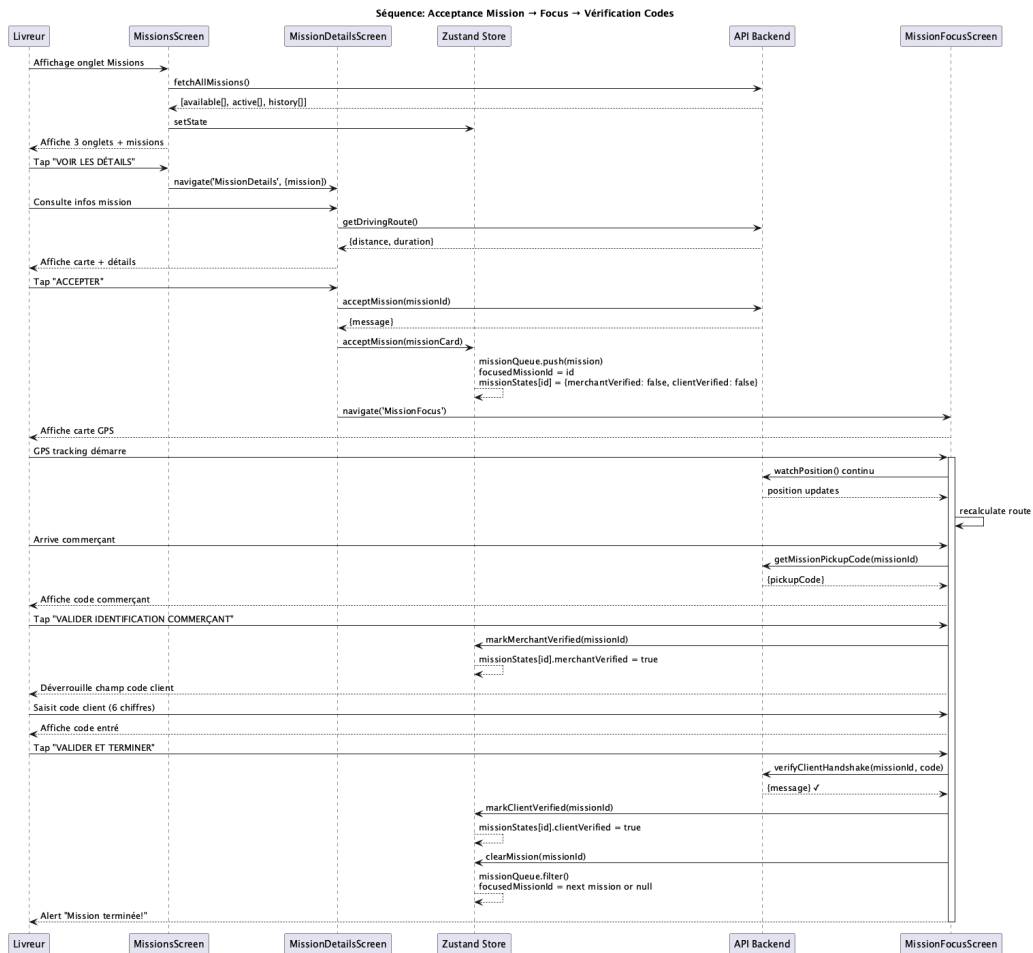


FIGURE 5.7 – Séquence technique : depuis MissionsScreen, le livreur accepte une mission via MissionDetailsScreen. La mission s’ajoute à la file (missionQueue). MissionFocusScreen s’affiche avec suivi GPS. Codes commerçant et client sont vérifiés séquentiellement, puis la mission est archivée.

0.15 Diagramme d'état : Transition états mission dans MissionFocusScreen

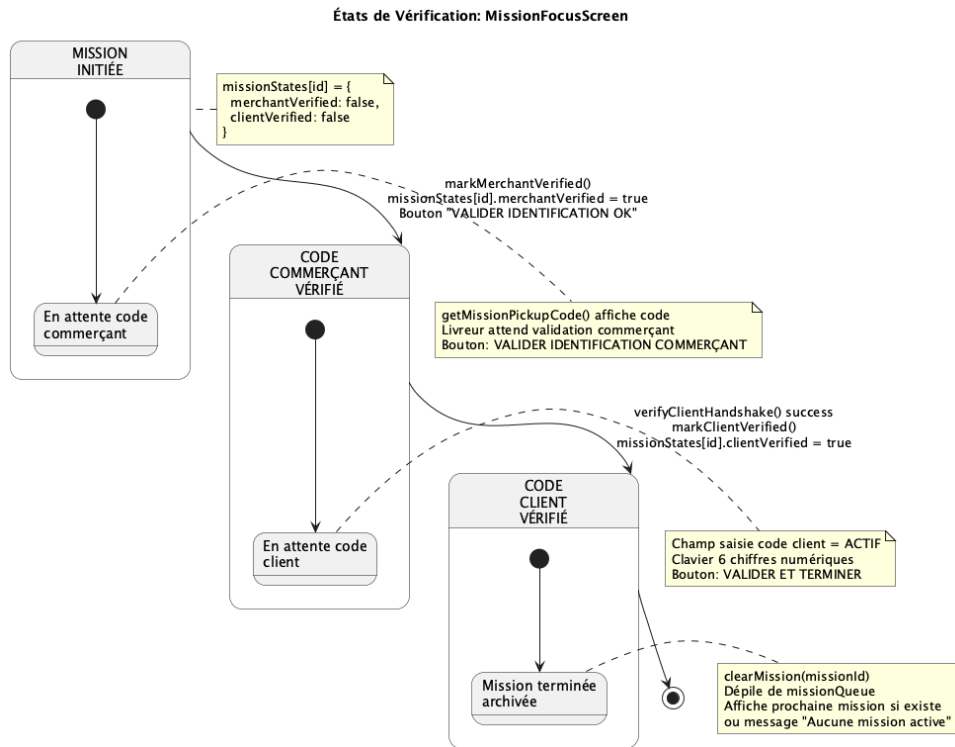


FIGURE 5.8 – États de vérification dans MissionFocusScreen : INITIÉE → COMMERÇANT VÉRIFIÉ (merchantVerified=true) → CLIENT VÉRIFIÉ (clientVerified=true) → TERMINÉE. À chaque étape, le panneau se met à jour pour afficher le code suivant ou le message de fin.

0.16 Écran 3 : Portefeuille (WalletScreen)

Purpose : gestion du solde et retrait

Le Portefeuille affiche les gains du livreur et permet les demandes de virement. Composants :

- **Carte solde principal** : affiche le solde disponible et le montant en attente de paiement.
- **Historique des transactions** : liste des crédits et débits avec dates, montants, statuts.
- **Bouton «Demander un virement»** : ouvre modal de retrait.
- **Modal de retrait** : saisie montant ou boutons rapides

Modal de retrait

À l'ouverture du modal :

1. Affiche solde disponible.
2. Champ saisie montant (`TextInput`, clavier décimal).
3. Boutons rapides calculant pourcentages du solde :
 - 25%, 50%, 75%, 100% pré-calculs.
 - Champ se remplit automatiquement.
4. Validation et vérification :
 - Montant doit être > 0 .
 - Montant doit être $\leq$ solde disponible.
5. Appel API `requestWithdrawal(amount)` : POST `/driver/me/wallet/withdrawals`.
6. Sur succès : alerte confirmation + rechargement du solde.

Endpoints API

- `getWallet()` : GET `/driver/me/wallet` $\rightarrow$ `{balance, pendingAmount, currency}`.
- `getWalletEntries()` : GET `/driver/me/wallet/entries` $\rightarrow$ historique transactions.
- `requestWithdrawal(amount)` : POST `/driver/me/wallet/withdrawals` $\rightarrow$ demande virement.

0.17 Diagramme de flux : Wallet (solde → retrait → confirmation)

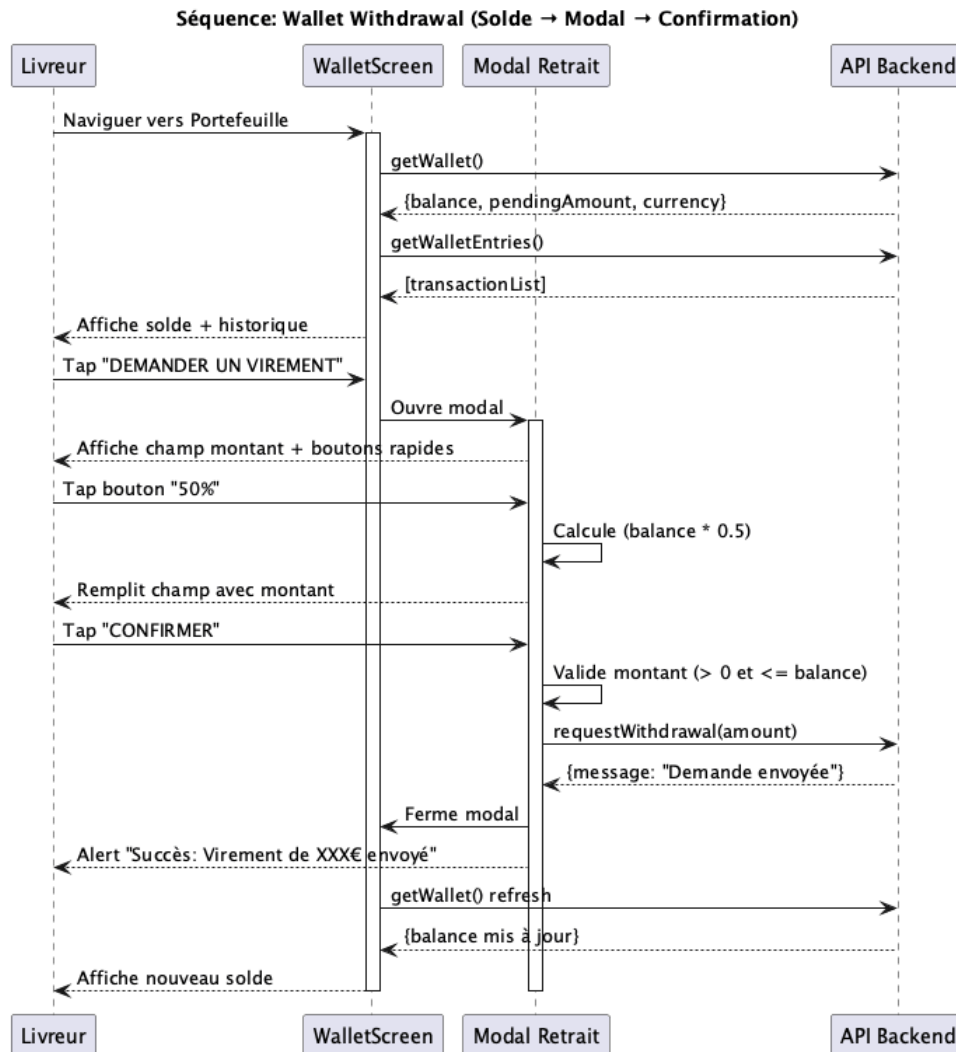


FIGURE 5.9 – Séquence : livreur consulte solde, ouvre modal retrait, calcule montant (rapide ou saisie), valide. API confirme, solde est mis à jour.

0.18 Écran 4 : Profil (ProfileScreen)

Purpose : gestion identité et KYC

Le Profil réunit les informations personnelles du livreur et l'accès au parcours KYC.
Composants :

- Carte identité principale :
 - Avatar circulaire (stocké sur Amazon AWS S3).
 - Nom et prénom.

- Note moyenne (rating) et nombre total de courses.
- Véhicule actuel (BIKE, SCOOTER, etc.).
- Numéro de code GoMile unique.
- Statut en ligne (ONLINE / OFFLINE).
- **Sélecteur de véhicule** : change le type de véhicule actif pour la session via `updateSessionVehicle(vehicleType)`.
- **Widget KYC** : affiche le statut de vérification (NOT_SUBMITTED, PENDING, APPROVED, REJECTED) avec badge coloré et CTA vers KYCScreen.
- **Section parrainage** : affiche code de parrainage et bouton pour partager avec amis.
- **Bouton déconnexion** : termine la session, clear tokens, retour écran LoginScreen.

Statut KYC sur ProfileScreen

Chaque état affiche une configuration visuelle distincte :

- **NOT_SUBMITTED** : badge orange, icône alerte, message «Ajoute tes documents», CTA «Compléter mon KYC».
- **PENDING** : badge bleu, icône horloge, message «En cours de vérification», CTA «Voir mes documents».
- **APPROVED** : badge vert, icône checkmark, message «Compte vérifié», CTA «Voir les détails».
- **REJECTED** : badge rouge, icône X, message «Documents refusés», CTA «Corriger mes documents».

Tap sur le widget : redirection vers KYCScreen avec le statut courant.

Endpoints API

- `getDriverProfile()` : GET `/driver/me/profile` → infos perso.
- `updateSessionVehicle(vehicleType)` : PATCH `/driver/me/session-vehicle`.
- `getMyKycStatus()` : GET `/driver/me/kyc` → statut KYC.
- `getMyReferral()` : GET `/driver/me/referral` → code parrainage.
- `logout()` : clear tokens (local storage).

0.19 Diagramme d'état : Profil et transitions KYC

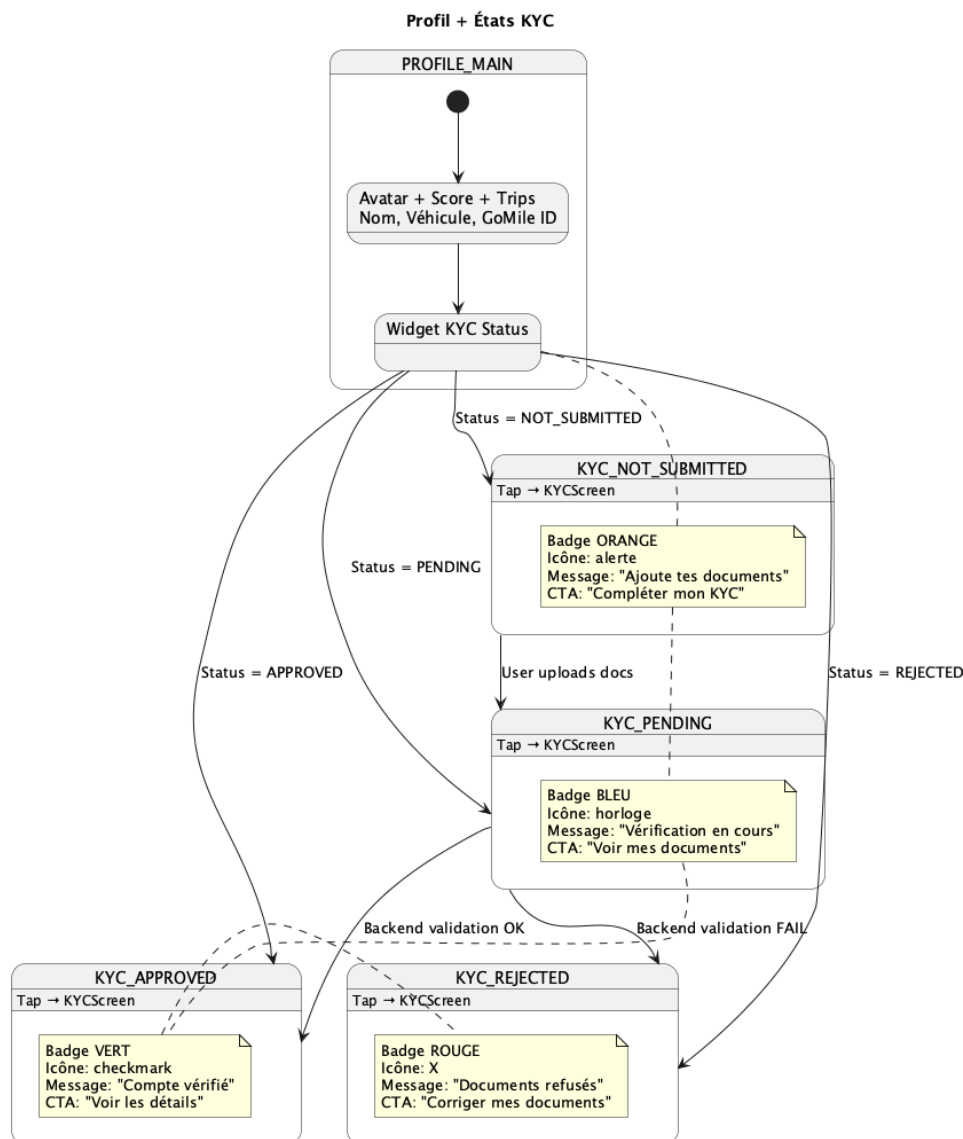


FIGURE 5.10 – État du profil : affichage des infos de base. Widget KYC avec 4 états colorés (NOT_SUBMITTED → PENDING → APPROVED ou REJECTED). Tap sur widget navigue vers KYCScreen.

0.20 Écran enfant : KYC (KYCScreen)

KYCScreen est l'écran dédié à la vérification d'identité et à la gestion des documents. C'est ici que le livreur téléverse les pièces justificatives requises.

Slots et types de documents

GoMile demande 5 emplacements (slots) de documents (Cette partie est susceptible de beaucoup changer car nous avons pas fini de discuter sur la notion de KYC avec les collègues) :

IDENTITY accepte «**Carte nationale d'identité**» (CNI) OU «**Passeport**». Un seul des deux suffit.

DRIVING_LICENSE permis de conduire.

REGISTRATION_CARD carte grise (immatriculation véhicule).

RIB relevé bancaire (IBAN/RIB pour virements).

JUSTIFICATIF_DOMICILE justificatif de domicile de moins de 3 mois.

L'utilisateur peut téléverser des fichiers images (PNG, JPEG) ou PDF.

États de vérification d'un document

Chaque document dans un slot a quatre états possibles :

Non fourni aucun fichier uploadé ; puce grise ; CTA «Ajouter document».

En attente d'examen document uploadé mais pas encore vérifiée par backend ; icône horloge bleu ; puce bleu.

Vérifié backend a validé le document ; icône checkmark vert ; puce vert ; badge «Vérifié ».

Refusé backend a rejeté le document ; icône X rouge ; puce rouge ; badge «Refusé — à corriger» + raison de rejet.

Statut KYC global

Le statut global du compte KYC a aussi quatre états :

NOT_SUBMITTED aucun document n'a été soumis ; affichage : badge «KYC à compléter», sous-titre «Téléverse tes documents», icône alerte.

PENDING au moins un document soumis, en attente de vérification backend ; badge «Vérification en cours», sous-titre «Examen sous 48h», icône horloge.

ACCEPTED tous les documents requis ont été validés ; badge «Compte vérifié», sous-titre «Tu as accès à tous les missions», icône checkmark vert.

REJECTED un ou plusieurs documents refusés ; badge «Documents refusés», sous-titre «Corrige-les et resoumets», icône X rouge.

Workflow d'upload de document

Étape 1 : Sélection du fichier

1. Livreur tap sur slot vide ou sur document existant pour modifier.
2. Modal : choix «Caméra», «Galerie photo», ou «Fichier».
3. Livreur sélectionne source et choisit/prend le fichier.
4. Module `media-picker` : applique permissions, retourne URI du fichier.

Étape 2 : Presignage URL S3

1. Appel API `presignDocument(filename, contentType)` : POST `/driver/me/documents/presign`.
2. Backend retourne : `{uploadUrl, fileUrl}`.
3. `uploadUrl` est une URL S3 signée (temporaire, 5 min).
4. `fileUrl` est l'URL finale du fichier stocké.

Étape 3 : Upload S3 direct

1. Utilise `uploadFileToPresignedUrl(uploadUrl, file, contentType)`.
2. Envoie fichier directement vers S3 via l'URL signée (bypass backend, plus rapide).
3. Indicateur loading : «Envoi en cours...».
4. Sur succès : puce de l'état passe à «En attente d'examen» (bleu).

Étape 4 : Liaison document au slot

1. Appel API `createMyDocument(type, fileUrl)` : POST `/driver/me/documents`.
2. Backend enregistre le document lié au slot.
3. État du document : `{id, type, url, verified: false, rejectionReason: null, createdAt}`.

Étape 5 : Soumission KYC (optionnel)

1. Livreur voit tous les documents uploadés.
2. Bouton «SOUMETTRE MA DEMANDE KYC» : appel API `submitMyKyc()` : POST `/driver/me/kyc`.
3. Statut passe à PENDING ; backend commence vérification.

Suppression document

Livreur peut supprimer un document via bouton `deleteDocument(documentId)` :
DELETE /driver/me/documents/{id}.

La puce revient à «Non fourni».

Endpoints API pour KYC

- `getMyKycStatus()` : GET /driver/me/kyc → {status: 'NOT_SUBMITTED' | 'PENDING' | 'ACCEPTED' | 'REJECTED', latestSubmission}.
- `getMyDocuments()` : GET /driver/me/documents → liste des documents uploadés.
- `presignDocument(filename, contentType)` : POST /driver/me/documents/presign → {uploadUrl, fileUrl}.
- `uploadFileToPresignedUrl(uploadUrl, file, contentType)` : PUT vers S3 (hors API GoMile).
- `createMyDocument(type, fileUrl)` : POST /driver/me/documents → enregistre document.
- `deleteDocument(documentId)` : DELETE /driver/me/documents/{id}.
- `submitMyKyc()` : POST /driver/me/kyc → soumet au backend pour vérification.

GoMile

Diagramme de flux : Upload document KYC complet

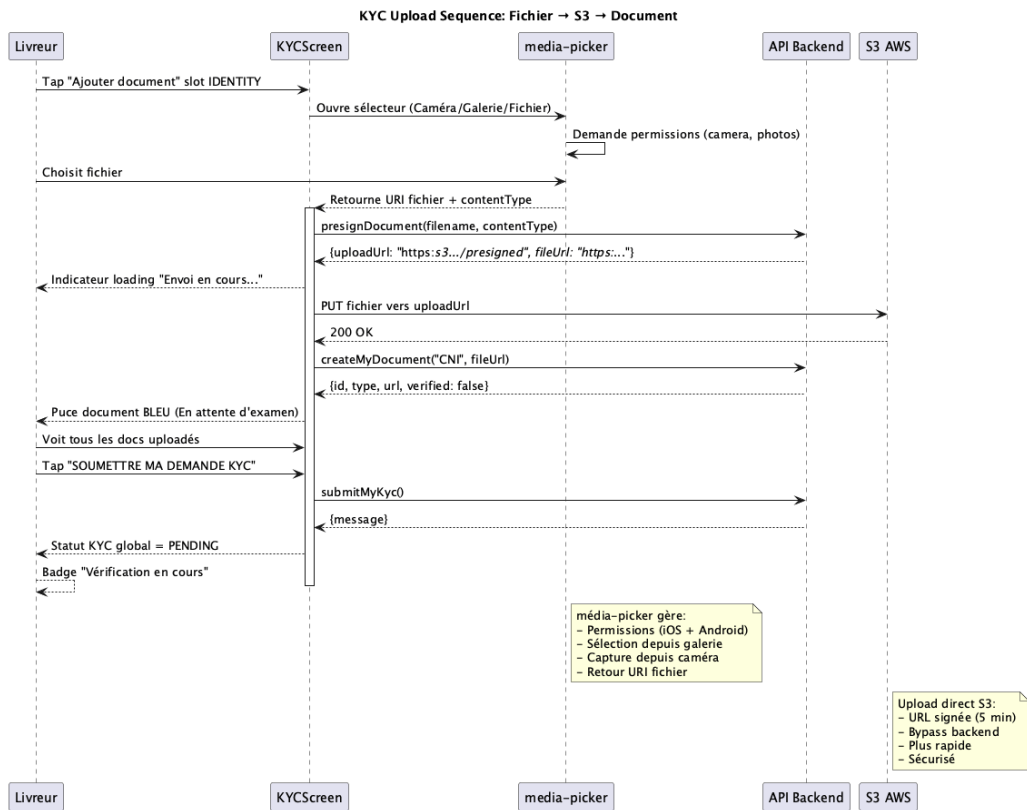


FIGURE 5.11 – Séquence technique : livreur sélectionne fichier → presignage URL S3 → upload direct S3 → création document backend. État passe à «En attente d'examen». Soumission optionnelle verrouille les modifications.

Diagramme d'état : KYC global + transitions documents

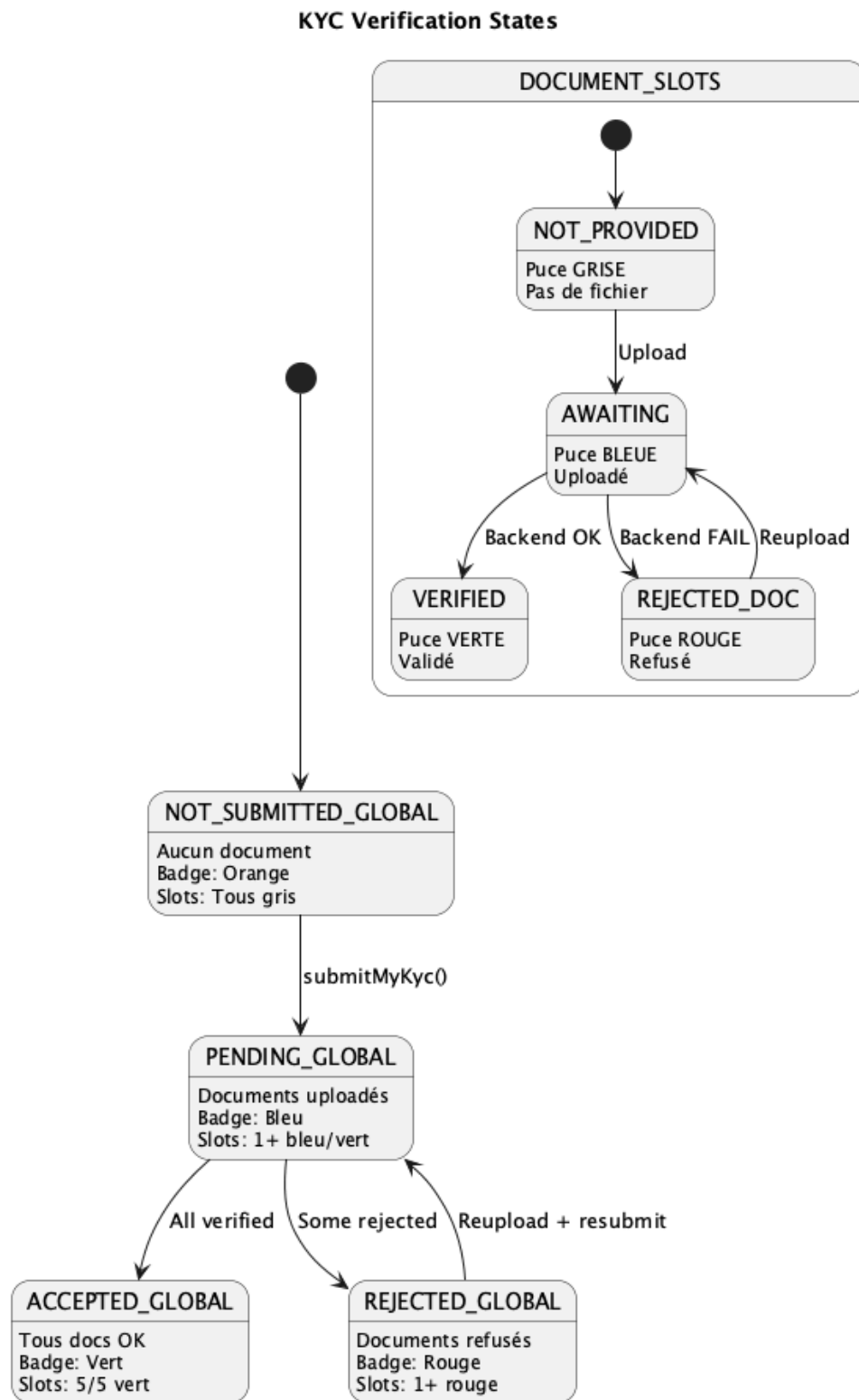


FIGURE 5.12 – États : NOT_SUBMITTED → PENDING (documents uploadés) → ACCEPTED (tous vérifiés) ou REJECTED (révision nécessaire). Chaque slot a ses propres états (Non fourni, En attente, Vérifié, Refusé). Livreur peut reupload et resubmit.

0.21 Services et stores partagés

Choix d'architecture pour la persistance des missions

Au début du projet, l'état des stores Zustand était partiellement sauvegardé localement dans un fichier JSON sur le téléphone. L'idée était simple : si l'application était relancée, on relisait cette sauvegarde locale pour reconstruire l'état de navigation et de mission. Cette approche pouvait sembler pratique pour un prototype, mais elle introduisait un risque important : si le téléphone du livreur s'éteignait, redémarrait, ou perdait son stockage temporaire pendant une livraison, l'application pouvait restaurer une mission qui n'existait plus réellement côté serveur. On se retrouvait alors avec une *mission fantôme* : l'interface affichait une livraison en cours alors que le backend ne pouvait plus garantir la cohérence de cet état.

Après concertation avec l'équipe, nous avons donc décidé de déplacer la source de vérité vers le backend. Désormais, les stores Zustand restent utiles pour l'interface et la synchronisation en mémoire de session, mais l'état métier critique est reconstruit à partir des API serveur au démarrage et lors des rafraîchissements. Ce choix rend l'application plus robuste face aux pertes de contexte, supprime les divergences entre téléphone et serveur, et garantit qu'une mission en cours reste traçable même si le mobile se coupe brutalement ou devient indisponible.

En pratique, cela signifie que : 1) le téléphone n'est plus responsable de conserver durablement l'état d'une mission ; 2) le backend garde l'historique, les statuts et les vérifications ; 3) au relancement, l'application interroge le serveur pour reconstruire un état fiable au lieu de faire confiance à une sauvegarde locale possiblement obsolète.

useMissionStore (Zustand)

Le store central pour la gestion des missions :

missionQueue tableau des missions acceptées en cours de livraison.

focusedMissionId ID de la mission actuellement suivie (MissionFocusScreen).

missionStates objet : {[missionId]: {merchantVerified, clientVerified}}.

availableMissions missions proposées à proximité.

activeMissions missions acceptées (alias de missionQueue pour l'affichage).

historyMissions missions archivées/livrées.

lastRefreshAt timestamp dernier fetch.

isLoadingMissions indicateur chargement.

Actions principales :

- `acceptMission(mission)` : ajoute à `missionQueue`, définit `focused`.
- `declineMission(missionId)` : ajoute à `declinedMissionIds`.
- `markMerchantVerified(id)` : marque commerçant vérifié.
- `markClientVerified(id)` : marque client vérifié.
- `clearMission(id)` : dépile mission, met à jour `focusedMissionId`.
- `setFocusedMission(id)` : bascule mission focalisée.
- `fetchAvailableMissions()` : GET API missions disponibles.
- `fetchAllMissions()` : GET API tous les états (disponibles + actives + historique).
- `initStore()` : restaure missions actives au démarrage.

useAvailabilityStore (Zustand)

Store simple pour le statut en ligne/hors ligne :

- `isOnline` : booléen état courant.
- `setOnlineStatus(boolean)` : action mise à jour.
- `toggleOnlineStatus()` : toggle action.

Intégration : tout écran lit `isOnline` pour afficher/masquer les éléments mission-dépendants.

useMissionsPoller (Hook personnalisé)

Hook gérant le fetch des missions avec filtrage par écran :

- Paramètre `{ fetchAll: boolean }`.
- `fetchAll = false` : dashboard uniquement missions disponibles (rapide).
- `fetchAll = true` : MissionsScreen charge tous les états.
- Fetch unique au premier focus de l'écran OU passage en ligne.
- Bouton refresh manuel via retour action `{ refresh() }`.
- Les nouvelles missions arrivent aussi via WebSocket (events `order:new` dans Main-Tabs).

socketService (WebSocket)

Connexion socket.io au backend pour événements temps réel :

`connectSocket(token)` établit connexion ; emmet `join-driver` avec `driverId`.

Events reçus — `order:new` : nouvelle mission disponible → fetch + notification push locale.

— `driver:status` : changement statut driver côté serveur (ex : forcer offline).

disconnectSocket() ferme connexion (logout ou app termination).

Socket reste actif pendant toute la durée de session (MainTabs et enfants).

driver-client.ts (API endpoints)

Tous les appels API sont centralisés ici. Endpoints post-connexion :

- `getDriverDashboard()` : GET /driver/me/dashboard.
- `toggleDriverAvailability()` : PATCH /driver/me/availability.
- `updateDriverLocation(lat, lng)` : PATCH /driver/me/location.
- `getAvailableMissions()` : GET /driver/me/missions/available.
- `acceptMission(id)` : POST /driver/me/missions/{id}/accept.
- `getActiveMissions()` : GET /driver/me/missions/active.
- `getMissionHistory()` : GET /driver/me/missions/history.
- `getMissionPickupCode(id)` : GET /driver/me/missions/{id}/handshake/pickup-code.
- `verifyClientHandshake(id, code)` : POST /driver/me/missions/{id}/handshake/client/verify.
- `getWallet()` : GET /driver/me/wallet.
- `getWalletEntries()` : GET /driver/me/wallet/entries.
- `requestWithdrawal(amount)` : POST /driver/me/wallet/withdrawals.
- `getDriverProfile()` : GET /driver/me/profile.
- `updateSessionVehicle(vehicleType)` : PATCH /driver/me/session-vehicle.
- `getMyKycStatus()` : GET /driver/me/kyc.
- `getMyDocuments()` : GET /driver/me/documents.
- `presignDocument(filename, contentType)` : POST /driver/me/documents/presign.
- `createMyDocument(type, fileUrl)` : POST /driver/me/documents.
- `deleteDocument(id)` : DELETE /driver/me/documents/{id}.
- `submitMyKyc()` : POST /driver/me/kyc.

Gestion d'erreurs centralisée : redirection si token invalide, parsing des erreurs backend, retry logic.

Récapitulatif : flux complet d'une session livreur

1. Connexion : LoginScreen → EmailVerificationScreen → MainTabs.
2. MainTabs initialise : notifications push, WebSocket, position GPS, fetch missions initiales.
3. Livreur navigue entre onglets (TableauDeBord, Missions, Portefeuille, Profil).
4. Accepte une mission → MissionDetailsScreen → MissionFocusScreen.
5. Suivi GPS en direct, codes vérifiés séquentiellement.
6. Mission livrée → archivée, gains crédités au portefeuille.
7. Livreur consulte portefeuille, peut demander retrait.
8. Livreur manage profil, KYC, données personnelles.
9. Déconnexion : suppression tokens, fermeture WebSocket.



Chapitre 6

Plugin WooCommerce

Le projet inclut une extension WooCommerce permettant de connecter une boutique WordPress à l'infrastructure GoMile. Ce plugin joue le rôle d'interface entre le site e-commerce du commerçant et l'API de livraison. Il permet à une commande WooCommerce d'être transformée automatiquement en demande de livraison GoMile.

L'objectif du plugin est de rendre l'intégration utilisable par un commerçant sans modifier le code de sa boutique. Une fois installé et configuré, le plugin ajoute une méthode de livraison dédiée, calcule le coût de livraison, transmet les commandes à l'API GoMile et met à jour leur statut grâce aux webhooks.

1 Rôle dans l'architecture GoMile

Le plugin intervient côté commerçant. Il complète les autres composants du projet : l'API backend, l'interface web d'administration et l'application mobile livreur. Sa responsabilité est limitée à l'intégration WooCommerce :

- proposer la livraison GoMile dans le panier et le checkout ;
- demander un devis de livraison à l'API GoMile ;
- créer une livraison GoMile lorsqu'une commande WooCommerce est validée ;
- annuler une livraison si la commande WooCommerce est annulée ;
- recevoir les webhooks GoMile pour synchroniser les statuts de livraison.

2 Fonctionnement général

Le plugin ajoute une méthode d'expédition appelée **Gomile Delivery**. Cette méthode peut être activée dans les zones d'expédition WooCommerce, comme illustré ci-dessous.

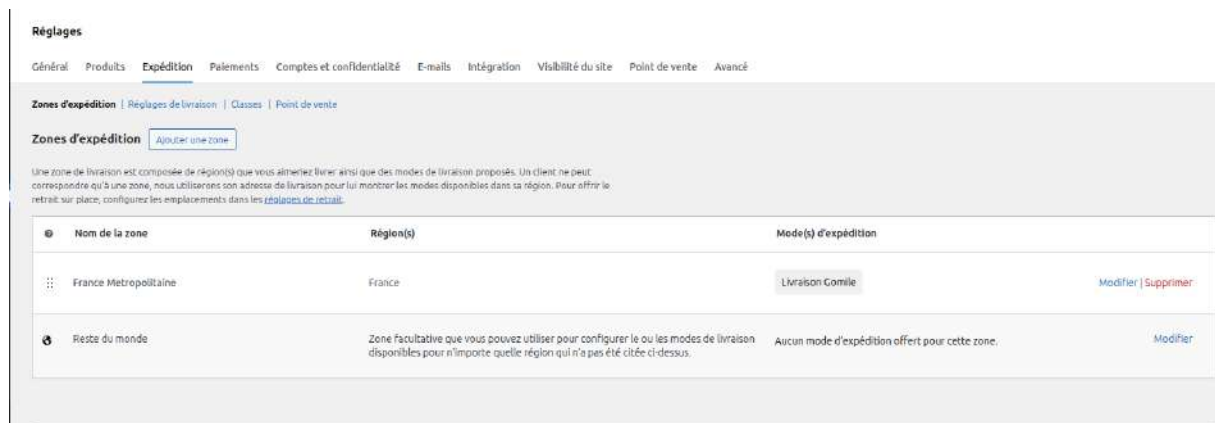


FIGURE 6.1 – Méthode de livraison GoMile activée dans une zone d'expédition WooCommerce.

Lors du calcul des frais de livraison, WooCommerce appelle cette méthode. Le plugin construit alors une requête de devis contenant l'adresse d'enlèvement, l'adresse client et le poids estimé du panier. Le tarif retourné par l'API est ensuite affiché dans WooCommerce.

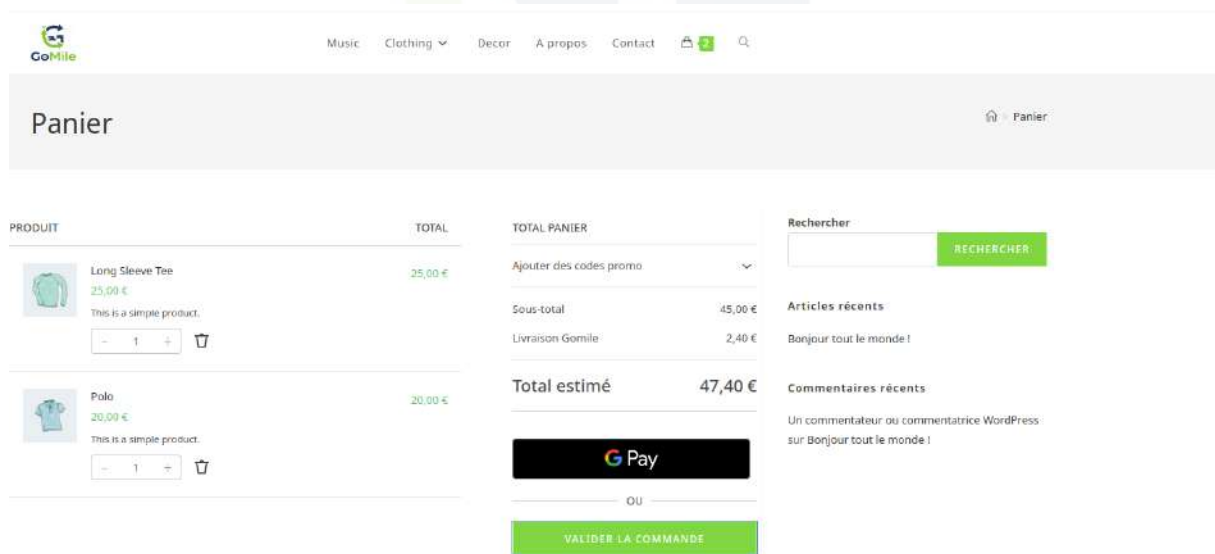


FIGURE 6.2 – Affichage de la livraison GoMile et du tarif calculé dans le panier.

La méthode de livraison reste visible pendant la validation de commande. Le client peut ainsi confirmer son achat avec l'option **Livraison Gomile** sélectionnée.

The screenshot displays the 'Validation de la commande' (Order Validation) page for GoMile. The page is divided into several sections:

- Header:** Includes the GoMile logo and navigation links for Music, Clothing, Decor, A propos, and Contact.
- Payment Section:** Features a 'Validation de commande express' button with the GoMile logo and a link to 'Ou continuez ci-dessous'.
- Coordonnées (Contact Information):** Includes an email field with 'john.doe@gmail.com' and a note: 'Vous validez votre commande en tant qu'invité.'
- Adresse de livraison (Shipping Address):** Displays the address 'John Doe, 361 allée de berlioz, 38400 SAINT-MARTIN-D&RSQUO;HÈRES, France, 0620651777' with a 'Modifier' (Edit) link.
- Options de livraison (Shipping Options):** Shows a radio button selected for 'Livraison Gomile' at a cost of 2,40 €.
- Résumé de la commande (Order Summary):** Lists items: 'Long Sleeve Tee' (25,00 €) and 'Polo' (20,00 €). It also shows a 'Sous-total' of 45,00 €, 'Livraison Gomile' of 2,40 €, and a 'Total' of 47,40 €.

FIGURE 6.3 – Sélection de la livraison GoMile lors de la validation de commande.

Après validation d'une commande utilisant cette méthode de livraison, le plugin envoie les informations nécessaires à l'API GoMile : nom du client, téléphone, adresse de livraison, poids et référence de commande WooCommerce. Cette référence permet ensuite de retrouver la commande lors des notifications envoyées par webhook.

3 Choix techniques

Le plugin est développé en PHP, conformément à l'écosystème WordPress. Il utilise les hooks WooCommerce pour s'intégrer au checkout, aux méthodes d'expédition et aux changements de statut de commande. Les appels externes passent par l'API HTTP WordPress, ce qui évite l'utilisation directe de cURL et respecte les pratiques recommandées par WordPress.

La configuration du plugin est centralisée dans une page d'administration. Elle permet de renseigner l'URL de l'API, la clé API, les endpoints utilisés, le secret webhook et l'adresse d'enlèvement.

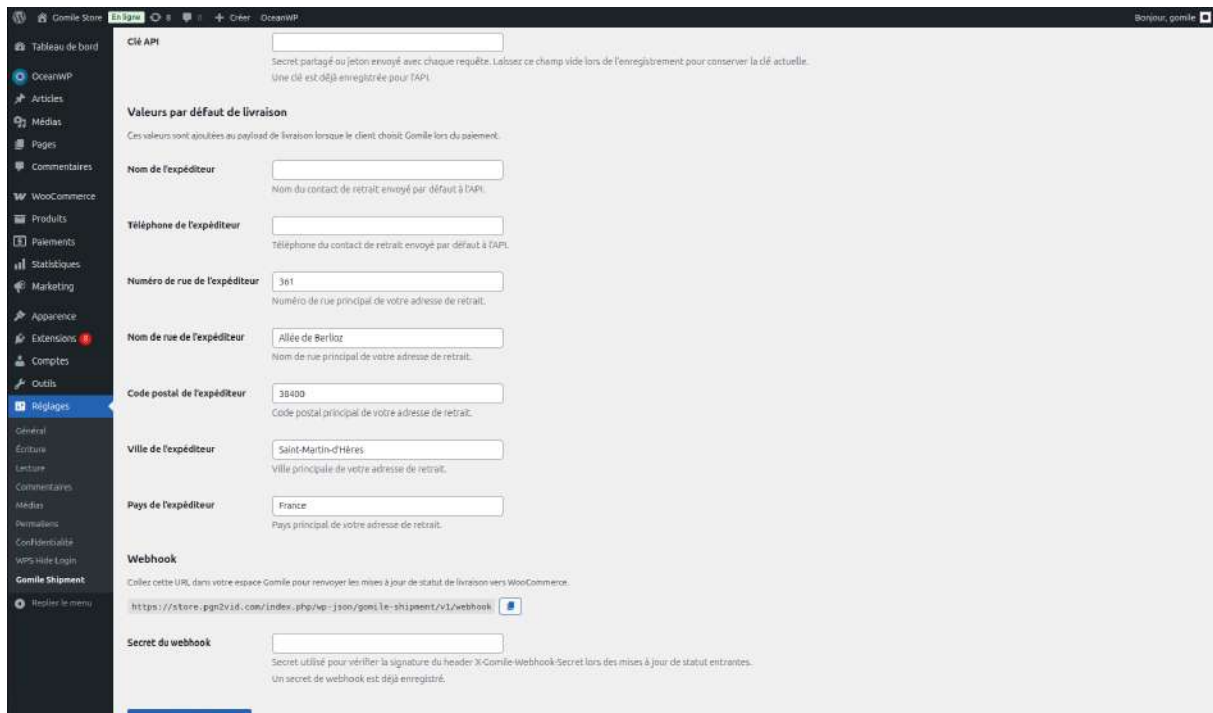


FIGURE 6.4 – Page de configuration du plugin GoMile dans l’administration WordPress.

4 Sécurité et synchronisation

Les webhooks reçus depuis GoMile sont protégés par une signature HMAC-SHA256. Le plugin vérifie cette signature avant d’accepter une mise à jour de statut. Les informations liées à la livraison sont stockées dans les métadonnées de la commande WooCommerce : statut GoMile, identifiant de livraison, URL de suivi et dernière erreur éventuelle.

En complément des métadonnées, le plugin ajoute des notes dans la commande WooCommerce. Ces notes permettent au commerçant de suivre les événements importants : création de la commande GoMile, code de livraison, frais estimés, distance calculée et mises à jour reçues par webhook.



FIGURE 6.5 – Notes de commande WooCommerce générées par le plugin GoMile.

Le plugin intègre également une logique d’idempotence afin d’éviter la création de livraisons en double lorsqu’un même hook WooCommerce est déclenché plusieurs fois.

5 Documentation technique

La documentation détaillée du plugin est fournie dans le document **guide-plugin.pdf**. Ce guide décrit l’architecture interne, les fichiers principaux, les hooks WordPress et WooCommerce utilisés, les flux API, les métadonnées de commande, les tests unitaires et

les diagrammes de séquence.



Chapitre 7

Utilisation de l'intelligence artificielle pendant le projet

Au cours du développement du projet GoMile, nous avons eu recours à des outils d'intelligence artificielle afin d'améliorer notre productivité et de faciliter l'avancement du projet. Dans un contexte où le temps disponible était limité et où le périmètre fonctionnel était large, l'utilisation de l'IA nous a permis de gagner du temps sur certaines tâches, tout en nous aidant à mieux structurer nos idées.

L'IA a notamment été utilisée comme outil d'assistance à la conception. Elle nous a aidés à visualiser certaines idées, par exemple en générant des images ou des représentations graphiques qui auraient autrement nécessité une création manuelle plus longue. Cela nous a permis d'explorer plus rapidement différentes propositions visuelles et de mieux communiquer certaines intentions d'interface ou de fonctionnement au sein de l'équipe.

Nous avons également utilisé l'IA comme support lors de l'implémentation de certains tests, notamment les tests de type black-box et les tests de flux. Ces tests étant importants pour vérifier le comportement global de l'application, l'IA nous a permis d'accélérer leur rédaction et de couvrir plus rapidement plusieurs scénarios utilisateurs. Cette aide a été particulièrement utile face aux contraintes de temps rencontrées pendant le projet.

Cependant, nous reconnaissons que l'IA reste un outil d'assistance et ne remplace pas le travail d'analyse, de validation et de décision réalisé par l'équipe. Les propositions générées ont toujours dû être relues, adaptées et vérifiées afin de s'assurer qu'elles correspondaient à notre architecture, à nos choix techniques et aux exigences du projet. En particulier, les décisions concernant la structure du système, la sécurité, les flux métier et l'intégration entre les différents modules sont restées sous notre responsabilité.

Ainsi, l'utilisation de l'intelligence artificielle a représenté un appui précieux dans le développement de GoMile. Elle nous a permis de progresser plus efficacement, tout en nous obligeant à garder un regard critique sur les résultats produits. Cette expérience nous a

montré que l'IA peut être un outil très utile dans un projet informatique, à condition d'être utilisée de manière contrôlée, réfléchie et complémentaire au travail des développeurs.



Chapitre 8

Conclusion

Ce projet GoMile nous a permis de travailler sur une application complète, proche d'un cas réel d'entreprise, en combinant plusieurs aspects essentiels du développement informatique : conception d'architecture, développement backend, interfaces web, application mobile, intégration WooCommerce, gestion des utilisateurs, sécurité, tests et déploiement.

À travers ce projet, nous avons appris à construire une plateforme cohérente composée de plusieurs modules qui communiquent entre eux : une application mobile pour les livreurs, un espace web pour les commerçants, une interface d'administration, ainsi qu'une API centrale assurant la logique métier. Cette expérience nous a aussi permis de mieux comprendre les contraintes liées à un système de livraison réel, comme la gestion des missions, le suivi des commandes, la vérification KYC, les paiements, les notifications et la sécurisation des échanges.

Au-delà de l'aspect technique, ce projet a été une excellente opportunité pour améliorer notre organisation en équipe. Nous avons dû répartir les tâches, coordonner les différentes parties du système, résoudre des problèmes d'intégration et maintenir une vision globale du produit final. Cela nous a permis de progresser non seulement en développement logiciel, mais aussi en communication, en autonomie et en gestion de projet.

Même si certaines améliorations restent possibles, notamment en matière d'optimisation, d'expérience utilisateur et de couverture de tests, le résultat obtenu constitue une base solide pour une plateforme fonctionnelle et évolutive. Ce projet représente donc une expérience très enrichissante, qui nous a permis de mettre en pratique les connaissances acquises à l'ENSIMAG tout en découvrant les exigences concrètes d'un projet applicatif complet.

En conclusion, GoMile a été pour nous une occasion précieuse d'apprendre, d'expérimenter et de développer des compétences techniques et humaines importantes. Nous retenons de ce projet une meilleure compréhension du développement d'un système d'information complet, ainsi qu'une expérience collective formatrice et motivante pour la suite de notre

parcours.

1 Annexe : vocabulaire technique

- **Espace administrateur** : interface réservée aux personnes qui suivent les comptes, les livreurs et les validations internes.
- **Espace commerçant** : interface utilisée par un marchand pour gérer ses boutiques, commandes et clés d'intégration.
- **Parcours livreur** : formulaire web permettant à un livreur candidat de créer son dossier et d'envoyer ses documents.
- **Application web Next.js** : partie visible par l'utilisateur, composée d'écrans et de composants d'interface.
- **Passerelle web sécurisée** : couche serveur de Next.js qui centralise les appels au backend, les cookies, les sessions et les erreurs.
- **Services métier GoMile** : backend qui applique les règles métier : comptes, boutiques, commandes, clés API, KYC et handshake.
- **Données et cache** : stockage persistant et temporaire utilisé par les services métier.
- **Clé API** : clé utilisée par une boutique ou une intégration e-commerce pour appeler GoMile de manière authentifiée.
- **OTP** : code numérique temporaire. Les codes de handshake ont 6 chiffres.
- **Code A** : code OTP utilisé entre le livreur et le commerçant pour confirmer la récupération du colis.
- **KYC** : vérification des documents et informations d'un livreur avant validation de son dossier.
- **Service externe** : système utilisé par GoMile mais séparé de l'application, par exemple l'envoi d'e-mails ou de SMS.